ABACUS / IMPLEMENTATION 1.0

Technical report

Architecture, physics, data provenance, verification scope and complete source appendix

16 SEPTEMBER 2026

Implementation and recorded host checks. MATLAB, GUI, Simulink and ML training have not executed on the delivery host.

ABACUS implementation technical report

Aircraft Engine Performance Analysis, Digital Twin, Machine Learning and Intelligent Engine Selection System

Version 1.0 | 16 September 2026

This report records the architecture, physical equations, provenance policy, implementation choices, testing plan and actual host evidence. It is not an experimental validation thesis. The final appendix contains the delivered MATLAB source for review and traceability; it is explicitly source documentation, not additional experiments.

Aircraft Engine Digital Twin - ABACUS

Academic steady-state software Digital Twin for engine performance analysis, comparison, constrained mission selection and physics-based ML surrogates. This implementation follows the attached ABACUS v1.1 specification; every deliberate correction or scope extension is recorded in CHANGELOG.md.

Delivery status

All 15 requested implementation areas have source, integration interfaces, documentation and relevant tests. This is an implementation package awaiting MATLAB runtime acceptance, not a claim that all phases have passed. The executed host audit passed 28 checks, including real SQLite schema, seed, transaction, provenance and integrity checks. The 64 MATLAB test functions have not been executed in this environment. MATLAB, GUI, Simulink and ML training results are not supplied as successful results.

The default database contains the PDF's 11 engines and 192 populated numerical parameters with matching provenance. All engine inputs copied from the secondary specification or introduced as assumptions retain Low confidence. No primary manufacturer data was independently verified during this implementation. Missing TSFC values remain NULL. The Kaveri warning is mandatory. Two optional piston entries implement the additional user request without changing the PDF's default catalogue.

Requirements

- MATLAB R2023b or newer.
- Database Toolbox, for the native SQLite interface, sqlread and sqlwrite.
- Statistics and Machine Learning Toolbox for RF, GPR and ANN training. The ANN implementation uses fitrnet with a separate network for each output; Deep Learning Toolbox is not required.
- Simulink only for the three Simulink wrappers.
- A graphical MATLAB desktop for the eight-tab app. A JVM is used for SHA-256 fingerprints.

No Python, Node.js, external SQLite JDBC driver or network connection is needed to run the MATLAB application. The optional host auditor uses Node.js 24 or newer with its built-in SQLite module.

Start

Unzip, open MATLAB, and set Current Folder to the extracted Aircraft_Engine_Digital_Twin directory. Then run:

```
run_project('gui')
```

That command calls setup_project, adds the authoritative folders, checks MATLAB version, and initializes/seeds the database if necessary. Existing compatible data is preserved. The GUI is a programmatic AppBase application in gui/EngineAnalyser_v1.m; it is not an App Designer .mlapp Design View file.

For an explicit installation and acceptance sequence:

```
p = setup_project;
db_initialize(p.database_file);
db_seed(p.database_file);
installation = verify_project;
tests = run_all_tests;
validation = run_project('validate');
app = run_project('gui');
```

Do not call a runtime phase accepted until its actual tests and reference review have been completed. Read FINAL_VERIFICATION_REPORT.md before presenting results.

Reproducible workflows

```
demo = run_project('demo');              % Single point, comparison, mission, HTML report
grid = run_project('grid');              % Architecture grids; may take time
db_seed_piston_extension;               % Explicitly add the two optional piston engines
models = run_project('simulink');         % Build actual .slx files with licensed Simulink
simout = run_simulink_model;             % Execute the turbojet wrapper
training = run_project('train');         % Requires accepted, current physics validation
```

The default ML gate is intentionally closed: the supplied secondary reference ratings are unverified. See docs/ML_WORKFLOW.md for the source-review and validation requirements. L5 falls back to physics with an explicit warning when an accepted model is unavailable. No pretrained MAT files, fabricated accuracy values, measured engine outputs or generated validation certificates are included.

Structure

Folder: database | Purpose: Reproducible SQL, production/test SQLite files, provenance and reference policy
Folder: matlab/setup | Purpose: Native SQLite access, seeding, constants and typed persistence
Folder: matlab/atmosphere | Purpose: Three-layer ISA with explicit altitude convention
Folder: matlab/engines | Purpose: Common cycle components and piston/TJ/TF/TP adapters
Folder: matlab/validation | Purpose: Input/output checks, reference validation, sensitivity and regression
Folder: matlab/digital_twin | Purpose: Shared coordinator and architecture dispatch
Folder: matlab/performance | Purpose: Architecture grids and validity-aware evaluation
Folder: matlab/comparison, matlab/mission | Purpose: Comparisons and constraint-first mission scoring
Folder: matlab/ml | Purpose: Dataset, splitting, four surrogate families, prediction and metrics
Folder: matlab/reporting, matlab/utilities | Purpose: Tables, plots, HTML reports, paths and fingerprints
Folder: gui, simulink | Purpose: Eight-tab app and three steady-state model wrappers
Folder: tests, scripts | Purpose: MATLAB tests, phase runner, static/host audit tools
Folder: docs, reports | Purpose: Manuals, technical report, specification and actual verification evidence
Folder: results, plots, tables, ml_models | Purpose: Runtime output destinations; no invented results

Scope

Uninstalled, constant-property, separate-exhaust, steady-state models. No afterburner, compressor maps, transient dynamics, surge/stall, cooling/bleed, inlet distortion, detailed

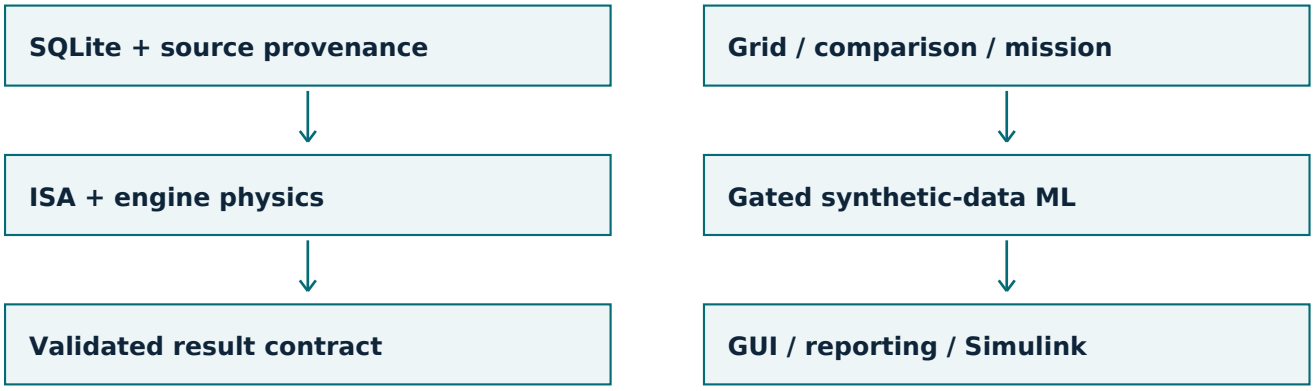
propeller map, flight-range solver, engine degradation, hardware interface or certification claim. Rated thrust/power is never used to force model outputs to match a target. Mission results compare catalogue candidates under an academic model and do not establish aircraft suitability.

See docs/USER_MANUAL.md, docs/ARCHITECTURE.md, docs/ASSUMPTIONS.md, docs/DATA_PROVENANCE.md, docs/TESTING.md and docs/LIMITATIONS.md. The PDF technical report includes a labelled source-code appendix, not experimental results.

Architecture and contracts

One authoritative backend

Database rows load through DB_Interface and db_get_engine. The latter constructs an engine structure and checks matching numerical provenance. flight_condition standardizes altitude convention and SI conditions. simulate_engine validates the inputs and dispatches to Piston_Model, TJ_Model, TF_Model or TP_Model. Gas turbines share gas_turbine_cycle and its component functions. The coordinator, comparison, grids, mission logic, GUI and Simulink all call this same backend.



The same backend serves each interface. Reference review and actual physics tests gate synthetic-data training.

Public interfaces

Entry: setup_project | Inputs: Optional database path | Output / side effect: Adds project folders; initializes and seeds compatible DB
Entry: flight_condition | Inputs: Altitude m, Mach, throttle, optional convention/afterburner | Output / side effect: Standard condition including geometric and geopotential altitude
Entry: ISA_model | Inputs: Altitude m, Mach, convention | Output / side effect: Static/total atmosphere, velocity, gravity and viscosity
Entry: db_get_engine | Inputs: DB handle, Engine_ID | Output / side effect: Parameters, provenance, architecture, warnings and fingerprint
Entry: simulate_engine | Inputs: Engine structure, flight structure | Output / side effect: Standard result; invalid evaluations carry Valid=false and diagnostics
Entry: DT_Coordinator.run | Inputs: ID, condition, L1/L4/L5, persistence flag | Output / side effect: Physics or accepted surrogate result
Entry: DT_Coordinator.sweep | Inputs: ID, geopotential altitudes, Mach, throttle, persistence | Output / side effect: L2 table, retaining invalid points
Entry: DT_Coordinator.map | Inputs: ID, persistence | Output / side effect: L3 architecture grid table
Entry: compare_engines | Inputs: Coordinator, IDs, condition, metric | Output / side effect: Comparable results and sorted metric table

Entry: EngineSelector | Inputs: Coordinator, mission ID/structure, IDs, persistence | Output / side effect: Feasible/eliminated candidates, reasons and contributions
Entry: validate_physics | Inputs: Database path | Output / side effect: Executed internal tests, Track A report, current acceptance certificates
Entry: ml_train_surrogates | Inputs: Coordinator, ID, optional current dataset | Output / side effect: Actual metrics and MAT bundles, only after validation gate
Entry: generate_report | Inputs: Result, comparison, selection, ML metrics | Output / side effect: HTML analysis report using available session evidence

Data contract

SI is authoritative: force N, pressure Pa, absolute temperature K, mass kg, fuel kg/s, shaft power kW, TSFC kg/(N s), BSFC kg/kWh, altitude m, efficiencies dimensionless. Fields are named with their units. Jet Thrust_N and propeller EquivalentThrust_N are distinguished; jet-only TSFC remains unavailable for turboprop/piston. ShaftPower_kW for a turboprop is power delivered after the gearbox. Station tables contain total temperature/pressure; nozzle diagnostics separately retain static exit quantities.

The result also carries Engine_ID, engine name/architecture, condition, Data_Type, Prediction_Source, confidence, validity, warnings, version/fingerprint and station/component diagnostics. NaN represents unavailable numerical values in MATLAB and becomes SQL NULL. Physical values are never silently substituted for missing values. A comparison rejects an incompatible metric instead of ranking unavailable values.

Digital Twin levels

L1 is a single-point physics evaluation. L2 is an altitude sweep. L3 is the specified performance grid. L4 adds persistent results and logs. L5 attempts a current, accepted surrogate within its domain and explicitly falls back to physics if it is missing, stale or inapplicable. These levels describe software capability; they do not imply a sensor-connected or calibrated physical engine.

Persistence and failure paths

Every connection enables foreign keys. The native SQLite interface uses typed table writes, a closed identifier list and AutoCommit off/commit/rollback for atomic operations. Trusted distributed schema/seed scripts are the only SQL text executed from files. Reference rows are immutable and simulation logs append-only. Invalid operating points are retained in evaluation tables and failure metadata; no nonphysical result is eligible for ML training. An unknown engine ID is logged with a NULL foreign key and the original ID in input metadata when persistence is requested.

The fingerprint covers current code and engine/provenance inputs. Changes invalidate stored validation and surrogate use. It is a consistency check, not a cryptographic signature or protection against deliberate alteration of a local workspace.

Integration review limits

reports/HOST_AUDIT_RESULTS.json records the static dependency inventory, callback dispatch registry, filename uniqueness, entry-name matching and model references. MATLAB's

scripts/audit_dependencies.m uses requiredFilesAndProducts on the actual installation to resolve toolbox dependencies. Dynamic callback dispatch and MATLAB API behaviour still require runtime tests; static text inspection cannot prove all variables, argument types or paths correct.

Physical assumptions and equation ledger

Atmosphere

ISA is evaluated on geopotential altitude h from 0 to 32000 m. GUI altitude H is geometric. Convert h=Re*H/(Re+H), Re=6356766 m. Constants: T0=288.15 K, P0=101325 Pa, g0=9.80665 m/s2, R=287.058 J/(kg K), gamma=1.4. Lapse layers are -0.0065 K/m through 11 km, isothermal through 20 km, and +0.001 K/m through 32 km. Each layer's base pressure is propagated from the previous layer. Density=P/(R*T), sound speed=sqrt(gamma*R*T); total conditions use isentropic Mach relations. Gravity and Sutherland viscosity are included. No extrapolation or weather departure is automatic.

Gas turbine stations

Turbojet stations are 0,2,3,4,5,8. Turbofans add fan exit 2.5 and bypass nozzle 18. Turboprops add free power-turbine exit 7. All tabulated station temperatures/pressures are total quantities. Fuel increases core gas mass through the combustor. A non-ideal convergent nozzle returns exit Mach, static pressure, area, velocity, choking state and pressure thrust; expanded-equivalent velocity is used only for the energy bookkeeping stated in code.

Compression: T_out=T_in*[1+(PR^((gamma-1)/gamma)-1)/eta_c]. Specific compressor work is cp_air*(T_out-T_in). Burner fuel-air ratio follows (cp_gas*T4-cp_air*T3)/(eta_b*LHV-cp_gas*T4), with positive denominator and temperature-rise checks. The gas-generator turbine supplies compressor plus fan demand through mechanical efficiency; turbine pressure follows the efficiency-consistent temperature relation. Net thrust sums exit momentum and pressure thrust and subtracts incoming air momentum. Fuel momentum at inlet is not added because onboard fuel has no separate incoming freestream flux.

cp is derived from each gas R and gamma; cp_air and cp_gas therefore need not equal the PDF's rounded independent constants. Cold gas: gamma=1.4, R=287.058. Hot gas: gamma=1.33, R=287.4. LHV for the core gas-turbine model is 43.2 MJ/kg.

Off-design conditions

Throttle theta linearly reduces OPR toward 1 and fan PR toward 1. TIT is T3+theta*(design TIT-T3). Corrected inlet mass flow scales with total pressure/sqrt(total temperature), multiplied by 0.35+0.65*theta. Supersonic ram recovery uses 1-0.075*(Mach-1)^1.35, multiplied by the stored inlet recovery. These laws are labelled assumptions. The design mass-flow values do not supply compressor maps or a physically complete spool match. Low throttle below 0.5 is warned. Exactly zero throttle returns an engine-off result with zero output/fuel, no windmilling model and no fabricated fuel-specific consumption.

Turbofan

The total inlet airflow divides into core=total/(1+BPR) and bypass=total-core. The fan acts on both streams. OPR denotes total fan-times-core compression, so core PR=OPR/fan PR. Turbine work per unit core air is compressor work+(1+BPR)*fan work. Separate core and bypass nozzles can choke independently. No mixer, afterburner or multi-spool map is supplied.

Turboprop and propeller

After gas-generator work balance, a free power turbine expands to an assumed total pressure 1.05 times ambient. Its flow includes fuel mass. Shaft power passes through a stored gearbox efficiency. ShaftPower_kW is the delivered propeller-shaft power. Residual jet thrust is calculated from the actual remaining total pressure/temperature. Equivalent thrust equals propeller thrust plus residual jet net thrust. BSFC uses delivered shaft power.

At rest, ideal actuator-disc thrust scales as (2*rho*A)^(1/3)*(eta_prop*P)^(2/3). A smooth blend reaches eta_prop*P/V at Mach 0.05, using an algebraically finite expression at V=0. The propeller model is an academic static/cruise bridge, not a blade-element or measured propeller map. Diameter and RPM are explicit assumptions; tip Mach is diagnostic.
ESHP=(P_prop+F_jet*V/eta_prop)/745.6998716, with powers in W.

Optional piston model

Naturally aspirated airflow is rho*displacement*RPM/(30*strokes)*volumetric_efficiency*throttle. A four-stroke engine therefore has one intake per two revolutions. Fuel=air/AFR, brake power=fuel*LHV*brake_thermal_efficiency, torque=power/angular_speed. Compression ratio defines an ideal Otto efficiency bound; the supplied brake efficiency must remain lower. No manifold pressure map, supercharging, friction decomposition, transient combustion or manufacturer calibration is claimed. Both catalogue records and their parameters are opt-in Assumed/Low academic entries.

Envelope and grid policy

Engine envelopes follow the PDF Table 18.5 academic domains. They are not aircraft operating approvals. Turboprop altitude is 9000 m despite the conflicting PW127G note of 7620 m. Turbojets and low-BPR fans use 21x9x6=1134 grid points; high-BPR fans and turboprops use 21x8x6=1008. High-BPR altitude knots include 10668 and 11000 m. Supersonic grid entries outside an engine's limit are retained as invalid and excluded from ML. Grid cardinality is a configured evaluation count, not a claim that all points are physically valid.

Reference interpretation

Internal balance checks establish consistency of the implemented equations. Track A compares at matching source conditions, respecting dry/afterburner distinction and source-review status. Track B measures surrogate approximation to generated physics. Passing one track does not prove the other. Assumed mass flows and efficiencies may produce substantial errors against secondary ratings, especially in turboprop power; the implementation does not hide those errors by scaling outputs to ratings.

Data provenance

The default seed is an explicitly labelled academic transcription of the user-provided ABACUS v1.1 blueprint, supplemented by disclosed assumptions. No engine parameter in this delivery was independently verified against its original manufacturer/type-certificate source. An engine name or a citation appearing in the blueprint is not itself primary-source verification.

The engine catalogue contains:

ID: 1 | Engine: J79-GE-17A | Architecture: Turbojet | Confidence: Low

ID: 2 | Engine: SNECMA Atar 9C | Architecture: Turbojet | Confidence: Low

ID: 3 | Engine: R-25-300 | Architecture: Turbojet | Confidence: Low

ID: 4 | Engine: CFM56-5B4 | Architecture: Turbofan_HighBPR | Confidence: Low

ID: 5 | Engine: CF6-80C2B1F | Architecture: Turbofan_HighBPR | Confidence: Low

ID: 6 | Engine: F100-PW-229 | Architecture: Turbofan_LowBPR | Confidence: Low

ID: 7 | Engine: SNECMA M88-2 | Architecture: Turbofan_LowBPR | Confidence: Low

ID: 8 | Engine: GTX-35VS Kaveri | Architecture: Turbofan_LowBPR | Confidence: Low

ID: 9 | Engine: PT6A-67D | Architecture: Turboprop | Confidence: Low

ID: 10 | Engine: PW127G | Architecture: Turboprop | Confidence: Low

ID: 11 | Engine: AE 2100D3 | Architecture: Turboprop | Confidence: Low

Every one of the 192 populated numerical core engine parameters has a Parameter_Provenance record whose value matches the parameter table, with unit, source category, reference, notes and confidence. Estimated means a secondary/transcribed academic value; Assumed means an explicit modelling assumption. Neither means measured. The 13 Performance_Data reference rows include excluded afterburning targets for traceability; they are all UnverifiedSecondary with Reference_Verified=0. Missing measured TSFC is NULL, not an invented zero or fitted number. Defaults remain Low confidence in derived outputs.

The optional seed adds Lycoming O-360-A1A and Continental IO-550-B. The user requested their inclusion, but the primary PDF's core does not include pistons. Their supplied numerical values are explicit academic assumptions and not manufacturer-confirmed specifications.

Source-review process

Read the original public source and verify the exact variant, configuration, dry/afterburning status, altitude convention, atmospheric condition, Mach, throttle/rating, units and measurement/rating status. Record its exact URL, title/page, conversions and reviewer note. db_add_reference appends a new immutable row after these explicit inputs; it records the caller's review, not an automated independent verification. Do not update or delete an old reference to erase a discrepancy.

Reference_Kind is Measured, PublishedRating or EstimatedTarget for newly reviewed inputs. EstimatedTarget retains Low confidence and the statement that it is not measured performance. Kaveri accepts only EstimatedTarget under this project policy. Re-run validate_physics, inspect actual errors and the applicable tolerance, then inspect each engine's fingerprinted certificate. A reviewed source alone does not make an inaccurate model pass.

Confidence and labels

Reference identifies the origin of a comparison target, not its truth. Simulated is reserved for physics data without estimated parameter status; supplied Low-confidence seeds produce Simulated_Estimated. ML_Predicted marks surrogate outputs while retaining underlying source confidence and the surrogate's actual validation metadata. The L5 prediction source is always visible; a fallback remains Physics.

Kaveri displays: ESTIMATED / ACADEMIC MODEL - NOT MANUFACTURER-CONFIRMED PERFORMANCE. These results do NOT represent actual Kaveri engine performance.

Reference conflicts retained

CF6 variant naming, PT6A power, AE2100 rounding, PW127 altitude, Kaveri dry/afterburner targets and atmosphere checkpoints are resolved in CHANGELOG.md. Preserve these notes when adding stronger sources. Parameter changes must update matching provenance and trigger revalidation; do not silently promote a confidence flag.

SQLite database

engine_analysis.db is the seeded application database; engine_test.db is a separate seeded inspection copy. Automated MATLAB tests create disposable databases and do not mutate the production catalogue. schema.sql and seed_data.sql reproduce the 11-engine core. seed_piston_extension.sql is optional. PRAGMA user_version is 2.

The 11 required tables are Manufacturers, Engine_Types, Engines, Engine_Thermo_Params, Parameter_Provenance, Aircraft, Engine_Aircraft_Map, Performance_Data, Mission_Profiles, References and Simulation_Log. Project_Metadata is an additional table. Three views expose catalogue, reference rows and joined provenance; three indexes support condition/provenance/log lookups. Foreign keys are explicitly enabled on every connection. CHECK constraints guard architecture parameters, efficiencies, enum values, flight conditions and mission weight sum. Triggers make Reference rows immutable and Simulation_Log append-only.

Recreate in a new filename with setup_project(newDatabaseFile), or use db_initialize followed by db_seed. Both seed scripts are idempotent. Existing database files are never automatically deleted; an incompatible schema raises an error. No destructive migration utility is included.

The distributed SQL scripts use -- @statement delimiters so complete trigger bodies remain intact when MATLAB executes statements. Runtime values travel through typed sqlwrite tables; they are not concatenated into SQL. MATLAB NaN numeric data maps to SQLite NULL. The host SQL quoting test uses Node prepared bindings and therefore does not substitute for executing MATLAB sqlwrite tests.

The delivered databases have 11 engines, 192 numerical provenance records, 13 unverified secondary reference rows, 7 mission presets, 11 aircraft records and 11 engine-aircraft mappings. There are no supplied simulated or ML-predicted performance rows. Aircraft mappings are secondary catalogue associations, not flight suitability evidence.

See ../docs/DATA_PROVENANCE.md before interpreting a reference or changing an engine input.

Machine-learning workflow

Gate and data generation

The surrogate learns the implemented physics, not measured engine behaviour. ml_generate_dataset requires an accepted current validation certificate and uses the architecture grid through the authoritative coordinator. Invalid points, nonfinite targets and inappropriate fuel-consumption fields are excluded. The full validity mask is retained to reject prediction through invalid holes. The dataset is labelled Physics-generated synthetic data.

All default secondary targets are unverified, so no engine is automatically approved for ML. First execute validate_physics, inspect internal test results and Track A, review independent reference inputs as documented in DATA_PROVENANCE.md, resolve genuine model errors, then re-run validation. An explicitly reviewed estimated target only establishes agreement with an academic target. There is no bypass flag and no fabricated acceptance certificate in the distribution.

Split and preprocessing

Features are Mach, geopotential altitude and throttle. Random seed is 42; prior RNG state is restored. The split is jointly stratified in Mach/altitude bins with approximate 70/15/15 allocation. Integer rounding within bins means exact global percentages can differ and are recorded. Duplicate input rows are rejected before splitting. Input and response scales are estimated exclusively from training rows. Validation and test rows are transformed using those training statistics.

Candidate families

Family: Polynomial | Candidates: Degree 3 or 5 with ridge regularization | Notes: Shared explicit polynomial-feature generator

Family: RF | Candidates: 100 or 200 regression trees | Notes: Separate ensemble per target

Family: GPR | Candidates: Two kernel settings | Notes: Separate Gaussian process per target

Family: ANN | Candidates: Two fitrnet layer configurations | Notes: Separate network per target, compatible with R2023b

Hyperparameter choice within a family and family selection use validation relative RMSE only. Family selection is frozen before test metrics are inspected. The selected family is not retroactively replaced by a better test score. Test results are reported for each family, while deployment acceptance additionally checks actual accuracy and single-point timing.

Metrics and acceptance

RMSE and MAE retain the target's SI unit. R2 is undefined (NaN) for constant targets. MAPE excludes near-zero true targets and reports its included sample count. Relative errors normalize by mean absolute reference output. Criteria are R2 >=0.95, relative RMSE <2%, relative MAE <1.5%, MAPE <5%, plus measured single-point latency below 10 ms for operational use. These are thresholds, not delivered achievements. No R2 or latency is reported as achieved until training runs.

The test set evaluates approximation of the implemented physics only. It does not establish agreement with real engine data, uncertainty calibration or safety. Dense grid splits can be easier than extrapolation; the operational predictor disallows extrapolation.

Artifacts and prediction

After execution, ml_models stores actual engine_ID_FAMILY.mat and engine_ID_best.mat bundles with trained objects, architecture, targets/features, fingerprint, validation score, grid/mask, test metrics, actual split, timing acceptance and timestamp. Reports include selection diagnostics, residuals and predicted-versus-physics plots. ml_learning_curve is an additional training-size study; it also retains the gate and is not precomputed.

ml_predict verifies architecture, current fingerprint, operational acceptance, input bounds and the validity of surrounding grid corners. Missing, stale, rejected or out-of-domain bundles cause an explicit physics fallback through DT_Coordinator L5. The interface does not silently turn a physics fallback into an ML prediction. No trained MAT bundle is supplied because MATLAB training was unavailable.

```
setup_project;
validation = validate_physics;
% Review Track A and source inputs before continuing.
dt = DT_Coordinator;
data = ml_generate_dataset(dt,4);
metrics = ml_train_surrogates(dt,4,data);
delete(dt);
```

The last two commands intentionally raise a validation-required error for the unreviewed default seed.

User manual

Installation and first use

Extract the archive into a writable folder. In MATLAB R2023b or newer, set Current Folder to Aircraft_Engine_Digital_Twin. Install Database Toolbox. Run setup_project, verify_project and run_all_tests. The test runner writes actual Passed/Failed/Incomplete columns to reports/matlab_test_results.csv. Optional toolbox or desktop limitations may produce incomplete tests, which are not passes. Run run_project('gui') to open the application.

The first setup call preserves the provided seeded database and adds only the authoritative project source folders. You can create a separate database with setup_project(fullfile(tempdir,'my_abacus.db')). To use it in subsequent workflows, pass its filename explicitly to DT_Coordinator or EngineAnalyser_v1; calling run_project uses the standard project database.

Eight application tabs

1. Engine Database Browser: select architecture/engine, inspect catalogue and per-parameter provenance. Source confidence appears with the selected engine.

2. Single-Point Analysis: enter geometric altitude in metres, Mach and throttle. Run physics at L4 or request L5. Inspect SI/display metrics, source label, warnings and station total conditions. No output exists before calculation.

3. Performance Envelope: generate altitude/Mach/throttle sweeps and architecture maps. Invalid cells remain flagged. Plot force, power or fuel metrics only where applicable.

4. Digital Twin Comparison: compare engines at a common flight condition and with one comparable metric. Jet TSFC and propeller BSFC are not interchangeable.

5. ML Surrogate Manager: generate gated data, train candidates and inspect actual metrics. Missing validation or unavailable models produces an actionable diagnostic.

6. Mission-Based Engine Selection: choose a preset or edit constraints and normalized weights, apply filtering, then inspect survivors, eliminations and score contributions.

7. Validation Report: execute physics tests and Track A comparison. Unverified secondary ratings remain WARNING. No table is preloaded with invented passes.

8. Settings and Export: change a compatible database, choose single-point SI/Imperial display, export CSV/MAT/PNG/PDF and create an HTML session report. GUI altitude entry and CSV remain SI.

The GUI calls backend functions; it contains no second implementation of the thermodynamic equations. It is provided as .m AppBase source and cannot be opened as a .mlapp in App Designer Design View. Runtime screen layout and callbacks still require desktop MATLAB acceptance.

Command-line example

```
setup_project;
dt = DT_Coordinator;
f = flight_condition(0,0,1);
```

```
r = dt.run(4,f,'L4',true);
disp(format_result_metrics(r,'SI'));
comparison = compare_engines(dt,[4,5],f,'Thrust_N');
mission = EngineSelector(dt,1,[],true);
reportFile = generate_report(r,comparison,mission,table());
delete(dt);
```

Check r.Valid and r.Warnings before interpreting a calculated metric. L1 can evaluate without persistence; L4 stores results and metadata. L5 attempts ML and labels any physics fallback. A separate engine-off convention applies at throttle zero.

Mission presets

Mission_ID: 1 | Mission_Name: Subsonic Commercial Transport | Architectures: Turbofan_HighBPR | Altitude_m: 11000 | Mach: 0.82 | Ceiling_m: 11000

Mission_ID: 2 | Mission_Name: Air Superiority Fighter | Architectures: Turbojet,Turbofan_LowBPR | Altitude_m: 12000 | Mach: 1.2 | Ceiling_m: 12000

Mission_ID: 3 | Mission_Name: Ground Attack / CAS | Architectures: Turbojet,Turbofan_LowBPR | Altitude_m: 5000 | Mach: 0.85 | Ceiling_m: 5000

Mission_ID: 4 | Mission_Name: Maritime Patrol / ISR | Architectures: Turboprop | Altitude_m: 3000 | Mach: 0.45 | Ceiling_m: 3000

Mission_ID: 5 | Mission_Name: STOL Transport | Architectures: Turboprop | Altitude_m: 2000 | Mach: 0.35 | Ceiling_m: 2000

Mission_ID: 6 | Mission_Name: High-Altitude Reconnaissance | Architectures: Turbojet,Turbofan_LowBPR | Altitude_m: 20000 | Mach: 0.8 | Ceiling_m: 20000

Mission_ID: 7 | Mission_Name: Academic Trainer | Architectures: Turboprop | Altitude_m: 1500 | Mach: 0.25 | Ceiling_m: 1500

The six PDF presets plus Trainer are academic screening examples. First apply architecture, flight-envelope, thrust/power, fuel-specific-consumption, weight and ceiling constraints. Only survivors are normalized and scored. An engine violating a hard condition cannot win. If no engine survives, the application says so; if exactly one survives, it reports feasibility without pretending a relative ranking. Identical criterion values score 0.5 and cannot distinguish candidates. Explanations show five weighted contributions. No aircraft drag, range or full mission fuel integration is included.

Optional piston workflow

```
setup_project;
db_seed_piston_extension;
app = run_project('gui');
```

The opt-in seed adds two Assumed/Low catalogue records. This is an extension to the PDF's 11-engine core. Use shaft power, BSFC, fuel flow and equivalent propulsive thrust for this architecture.

Simulink workflow

```
setup_project;
models = build_simulink_models;
out = run_simulink_model('ABACUS_Turbojet',[1;0;0;1]);
```

Input order is engine ID, geometric altitude m, Mach, throttle. The builder makes three connected .slx models, each with a constant operating point, Level-2 MATLAB S-function and workspace sink. The S-function calls the shared physics backend. Delivered .mdl files can be inspected before building. Stop time zero represents one equilibrium evaluation; running a longer Simulink time does not add engine dynamics. Close an existing model before rebuilding; the builder refuses to overwrite an open model's unsaved edits.

Reporting and output locations

run_project('demo') creates a session report from real calculations once MATLAB runs. plots holds exported figures, tables contains CSV grids/analysis, results contains saved sessions and ml_models contains actual trained bundles. reports holds verification evidence and reports. The supplied technical report documents the implementation and includes source listings; it contains no claimed engine validation or experimental findings.

Troubleshooting

If sqlite is unavailable, install/enable Database Toolbox. A release error requires R2023b or later. A schema error requires backing up the old database and deliberately migrating or choosing a new filename; do not delete a user's database blindly. A provenance error means a numerical parameter and its provenance are missing or inconsistent. A validation-required error is expected for ML with the default unverified references. A missing model warning during L5 results in physics fallback. Invalid operating points must be corrected or left invalid, not clipped into a different condition. If a Simulink wrapper fails, inspect the original backend diagnostic and run its corresponding MATLAB case first.

Testing and verification

Executed evidence

The host audit uses Node.js built-in SQLite and static source inspection. Its recorded runtime is Node.js v24.19.0; timestamp 2026-09-16T01:46:16.330Z. It passed 28 checks and failed 0. This does not run MATLAB code. The actual evidence is reports/HOST_AUDIT_RESULTS.json.

Check: Fresh schema and seed execute | Status: PASS

Check: Eleven core engines | Status: PASS

Check: Required tables, metadata, views and indexes | Status: PASS

Check: Reproducible idempotent seed | Status: PASS

Check: Foreign keys explicitly enabled | Status: PASS

Check: Foreign-key violations rejected | Status: PASS

Check: Database integrity and FK checks | Status: PASS

Check: Every numeric parameter has matching provenance and unit | Status: PASS

Check: No unverified manufacturer/type-certificate claims | Status: PASS

Check: Kaveri provenance remains Low/Estimated or Assumed | Status: PASS

Check: Missing TSFC not fabricated | Status: PASS

Check: All four data types accepted, invalid enum rejected | Status: PASS

Check: Reference records immutable | Status: PASS

Check: Simulation log append-only | Status: PASS

Check: Transaction rollback | Status: PASS

Check: Quoted text survives prepared-value roundtrip | Status: PASS

Check: Efficiency, OPR and mission weight constraints | Status: PASS

Check: Six PDF missions plus requested Trainer | Status: PASS

Check: Optional piston seed is separate and reproducible | Status: PASS

Check: Delivered engine_analysis.db integrity | Status: PASS

Check: Delivered engine_test.db integrity | Status: PASS

Check: Unique public MATLAB filenames | Status: PASS

Check: No forbidden unfinished implementation markers | Status: PASS

Check: No hardcoded author workspace paths in MATLAB | Status: PASS

Check: MATLAB source entry names match filenames | Status: PASS

Check: GUI dispatched action registry has handlers | Status: PASS

Check: Simulink wrapper and model references exist | Status: PASS

Check: Regression and ML gates are implemented | Status: PASS

MATLAB tests included, not executed here

There are 64 test functions across eight suites. Fixtures create separate temporary databases. Optional GUI, Simulink or toolbox prerequisites are treated as incomplete/skipped tests, not passes. No matlab_test_results.csv is shipped because there was no MATLAB test run.

TestAtmosphere.m

- testSeaLevel
- testGeopotentialCheckpoints
- testLayerContinuity
- testGeometricConversion
- testMachStagnation
- testNegativeAltitude
- testUpperBound
- testInvalidMachAndThrottle

TestDatabase.m

- testAllTablesAndEngines
- testSeedIdempotent
- testProvenanceAudit
- testForeignKeyFailure
- testLiteralTextRoundTrip
- testAtomicRollback
- testImmutableReference
- testMissingProvenanceRejected
- testInvalidEngineID
- testSchemaConstraints

TestEngines.m

- testIdealCompressionCheckpoint
- testNozzleChoked
- testNozzleUnchoked
- testNozzleBoundaryContinuity
- testNoExpansionRejected
- testAllCoreEnginesSLS
- testTurbofanMassAndFanDemand
- testTurbopropPowerAndStaticCase
- testPropellerContinuousAndFinite
- testKaveriWarningAndType
- testAltitudeAndMachEnvelope
- testAfterburnerRejected
- testBadEfficiencyAndTemperature
- testPistonPowerTorqueAndAltitude
- testZeroThrottleState

- testChangedParametersInvalidateFingerprint

TestGUI.m

- testEightTabsAndKaveriWarning

TestIntegration.m

- testPersistenceAndLog
- testFailedPointLogsWithoutPerformance
- testLogImmutable
- testArchitectureGridCounts
- testGridValidityMaskAndTemperatureFields
- testCompatibleComparison
- testIncompatibleComparisonRejected
- testAllMissionPresets
- testUnverifiedReferencesCannotPass
- testReportingContainsProvenance
- testOutputUnitConversion
- testHTMLTextEscaped

TestML.m

- testSplitDeterministicAndDisjoint
- testDuplicateInputsRejected
- testMetricsKnownErrors
- testConstantAndZeroTargets
- testPolynomialTrainPredict
- testFourCandidateAPIs
- testMLCellRejectsHolesAndExtrapolation
- testMissingModelAndGate

TestMission.m

- testNormalisationDirections
- testTieAndSingleMetric
- testAllEliminated
- testOnlyOneSurvives
- testHighScoreCannotOverrideConstraint
- testArchitectureAndCeiling
- testWeightsRejected
- testMixedSFCRejected

TestSimulink.m

- testBuiltModelsMatchBackend

Run in MATLAB

```
setup_project;
installation = verify_project;
results = run_all_tests;
phaseResults = run_phase_tests(7);
dependencies = audit_dependencies;
validation = validate_physics;
```

Review every Failed and Incomplete result. A source parser cannot verify toolbox licensing, native SQLite types, GUI layout or S-function execution. The phase runner reuses meaningful suites instead of creating duplicate implementation-mirroring tests. For complete acceptance use run_all_tests, not only a single phase suite.

Reference and regression checks

Track A compares exact reference conditions and reports real discrepancies without fitting. Unverified sources cannot receive PASS. Reviewed estimated targets remain explicitly academic. Track B reports real held-out surrogate error after training. regression_baseline('capture') requires passing internal tests and refuses to replace an existing baseline; regression_baseline('compare') uses a 0.01% relative-change threshold. A baseline is an executed software-regression artifact, not an independent engine measurement. parameter_sensitivity varies explicit model inputs and reports changes without claiming experimental uncertainty.

Optional host reproduction

```
node scripts/audit_host.mjs .
```

Requires Node.js 24+. Its prepared-value roundtrip checks SQL/storage semantics through Node; the separate MATLAB typed-value roundtrip test must still execute with Database Toolbox.

Limitations and unresolved acceptance work

1. MATLAB R2023b, Database Toolbox, Statistics and Machine Learning Toolbox, graphical app execution and Simulink were unavailable on the delivery host. Source parsing and native host SQLite checks cannot establish that those APIs execute correctly. All 64 MATLAB tests remain unexecuted here.

2. Engine data is Low-confidence secondary/assumed input. There are no independently verified manufacturer measurements in the supplied seed. Track A reference acceptance and all runtime output accuracy remain unresolved.

3. ML training is blocked until actual internal tests and reviewed in-tolerance reference comparisons produce a current certificate. No trained model, R2, RMSE, MAPE, timing or predicted performance is delivered as an experimental result.

4. The GUI is programmatic AppBase source, not a .mlapp Design View project. Its eight-tab layout and backend wiring need runtime desktop review. Source inspection is not a GUI walkthrough.

5. Three text .mdl wrappers and the .slx builder are delivered. .slx generation and model loading/running have not occurred. The wrappers provide steady-state shared-backend evaluations, no physical transient dynamics or code generation support.

6. Constant gas properties, assumed efficiencies/TIT/flow, algebraic throttle laws and fixed corrected-flow scaling exclude detailed component-map matching. Afterburners, variable geometry, cooling and bleed, surge/stall, installation losses, nozzle area schedules, emissions and combustion chemistry are absent.

7. Propeller static blending and residual-jet pressure allocation are explicit academic assumptions. They do not model a complete propeller map. Turboprop rating agreement must be checked; secondary mass flow and TIT can yield substantial excess predicted power.

8. Piston modelling is an optional, naturally aspirated extension with assumed parameters. It is not a manufacturer dynamometer map.

9. Mission selection is relative to feasible database entries and selected weights. Envelope ceiling is a model input; it is not an aircraft service ceiling. No drag, takeoff distance, payload/range, flight-path integration or airframe installation compatibility is solved.

10. Confidence flags are categorical provenance labels, not quantified uncertainty. Fingerprints detect changes for reproducibility, not malicious modification. The database is a local academic store, not a multi-user access-control service.

11. Dynamic HTML reports and static PDF documentation are included. A comprehensive thesis with real experimental validation, benchmark plots or trained-model results cannot be completed without those experiments. The technical PDF's source appendix is explicitly labelled.

12. Source and documentation are ready to place in a Git repository, but no remote repository, fabricated commit history or publication was created.

Do not treat host audit PASS as acceptance of all implementation phases. FINAL_VERIFICATION_REPORT.md lists each phase separately and identifies what was actually executed.

Changelog

Implementation 1.0 - 2026-09-16

The 77-page ABACUS v1.1 PDF remains the primary specification. The two uploaded PDF copies are byte-identical; one is retained in docs. The implementation adds the following explicit clarifications/corrections.

ID: C01 | Issue: The blueprint illustrates JDBC parameter markers; the requested stack also allows native SQLite. | Implemented decision and consequence: Use Database Toolbox native sqlite, allowlisted table identifiers, typed sqlread/sqlwrite, and MATLAB-side filtering. No user value is interpolated into SQL. This is not a claim to implement JDBC prepared-query APIs.

ID: C02 | Issue: Earlier schema lists three data classes; later corrections require four. | Implemented decision and consequence: Reference, Simulated, ML_Predicted and Simulated_Estimated are all constrained values.

ID: C03 | Issue: The PDF contains geometric/geopotential atmosphere values and pressure checkpoints that do not consistently follow its equations. | Implemented decision and consequence: Internal geopotential altitude, explicit conversion at the GUI, R=287.058 J/(kg K), and continuous layer equations. At 15 km geopotential the pressure is about 12045 Pa, not 12111; at 25 km about 2511 Pa, not 2549. The geometric/geopotential difference at 32 km is about 0.5%, not 0.16%.

ID: C04 | Issue: Stated cp, gamma and R values are not fully thermodynamically consistent. | Implemented decision and consequence: Derive cp=gamma*R/(gamma-1) for each constant-property gas. Results can therefore differ from arithmetic using rounded independent cp.

ID: C05 | Issue: Turbofan work expression multiplies fan specific work by cp again. | Implemented decision and consequence: Fan Work_Jkg already contains cp. Demand per kg core air equals compressor work plus (1+BPR)*fan work.

ID: C06 | Issue: Non-ideal nozzle and choking equations need a consistent sonic boundary. | Implemented decision and consequence: Derive the efficiency-adjusted sonic pressure ratio; retain the classical ratio at eta=1. Compute exit total pressure from actual exit static state and Mach.

ID: C07 | Issue: Turboprop energy/momentum expressions omit fuel mass in places. | Implemented decision and consequence: Use gas mass flow = core air*(1+fuel-air ratio) for turbine power and nozzle momentum.

ID: C08 | Issue: Setting power-turbine exit total pressure equal to ambient removes the pressure head for a residual nozzle. | Implemented decision and consequence: An explicit assumed back-pressure ratio of 1.05 permits residual jet expansion. This is a modelling choice, not a manufacturer parameter.

ID: C09 | Issue: Propeller eta*P/V is singular at zero flight speed. | Implemented decision and consequence: Use an actuator-disc static limit with a smooth blend to the cruise expression by Mach 0.05. Retain the propeller geometry assumption in provenance.

ID: C10 | Issue: The printed ESHP expression places propeller efficiency inconsistently. | Implemented decision and consequence: ESHP=(propeller-shaft power + residual thrust*speed/eta_propeller)/745.6998716. Static residual thrust contributes zero equivalent power.

ID: C11 | Issue: Throttle and off-design compressor behaviour are underspecified. | Implemented decision and consequence: Document explicit pressure-ratio, temperature-rise and corrected-flow laws; these are academic assumptions, not compressor maps. Zero throttle uses an engine-off convention.

ID: C12 | Issue: Catalogue variants/ratings conflict within the PDF. | Implemented decision and consequence: Use CF6-80C2B1F label; PT6A-67D 1119 kW from later Table 23 rather than 895; AE2100D3 3458 kW rather than 3459. All remain unverified secondary values. Kaveri uses the 53400 N estimated dry target, not the 81000 N afterburning target.

ID: C13 | Issue: User requests piston engines, but the PDF final core is 11 gas turbines. | Implemented decision and consequence: Provide a separate opt-in two-engine piston seed and model. Default catalogue remains the PDF's 11 engines.

ID: C14	Issue: User requests a Trainer mission in addition to the PDF presets. | Implemented decision and consequence: Preserve six PDF presets and add an explicitly academic Trainer preset.
ID: C15	Issue: Desktop binary .mlapp cannot be generated/validated here. | Implemented decision and consequence: Deliver a complete programmatic AppBase .m application with eight tabs and shared backend callbacks. App Designer Design View editing is not delivered.
ID: C16	Issue: MATLAB/Simulink is absent on the execution host. | Implemented decision and consequence: Deliver three text .mdl models plus a real .slx builder and execution test. No generated .slx, claimed execution or fake binary is supplied.
ID: C17	Issue: Primary engine sources were not independently established. | Implemented decision and consequence: All seed engine parameters stay Estimated/Assumed and Low, all secondary reference rows stay unverified. Training requires actual internal tests plus a reviewed in-tolerance reference/explicit estimated target.
ID: C18	Issue: A reviewed estimate must not be called measured validation. | Implemented decision and consequence: Reference_Kind separates Measured, PublishedRating and EstimatedTarget from UnverifiedSecondary. Kaveri can only use EstimatedTarget. Acceptance against an estimate remains academic.
ID: C19	Issue: A repository history cannot be recovered/generated honestly from files. | Implemented decision and consequence: Deliver GitHub-ready source, license and ignore rules without fabricated commits or publication.

No reference rating is used to tune the engine equations. No numerical validation result, accuracy, timing, GUI screenshot or trained model is fabricated.

Phase traceability

Statuses distinguish implementation from execution. No entire physics, ML or GUI phase is marked PASS based on source presence.

Phase: 1 | Area: Database and project infrastructure | Status: PASS for executed host SQL subchecks; WARNING for phase acceptance
Phase: 2 | Area: ISA atmosphere | Status: NOT EXECUTED
Phase: 3 | Area: Piston physics | Status: NOT EXECUTED
Phase: 4 | Area: Turbojet physics | Status: NOT EXECUTED
Phase: 5 | Area: Turbofan physics | Status: NOT EXECUTED
Phase: 6 | Area: Turboprop physics | Status: NOT EXECUTED
Phase: 7 | Area: Physics validation and automated testing | Status: WARNING
Phase: 8 | Area: Digital Twin coordinator | Status: NOT EXECUTED
Phase: 9 | Area: Performance grids | Status: NOT EXECUTED
Phase: 10 | Area: Comparison and analysis | Status: NOT EXECUTED
Phase: 11 | Area: Mission-based selection | Status: NOT EXECUTED
Phase: 12 | Area: ML dataset generation | Status: NOT EXECUTED
Phase: 13 | Area: ML surrogate training and evaluation | Status: NOT EXECUTED
Phase: 14 | Area: MATLAB GUI | Status: NOT EXECUTED
Phase: 15 | Area: Full integration and reports | Status: WARNING

PHASE 1 - Database and project infrastructure

Status: PASS for executed host SQL subchecks; WARNING for phase acceptance

Files: setup_project.m, project_paths.m, matlab/setup, database/schema.sql, database/seed_data.sql, database/engine_analysis.db, database/engine_test.db.

Dependencies: MATLAB R2023b, Database Toolbox.

Included test suites: TestDatabase.m.

Actually executed / reviewed: 28 host audit checks include real SQLite creation, seed, constraints, rollback, provenance and delivered-file integrity. MATLAB native connection/write APIs are NOT EXECUTED.

Known limitations and unresolved issues: Native SQLite typed operations replace JDBC-specific examples. Runtime API acceptance remains open.

Test cases in the applicable shared suites:

- TestDatabase.m: testAllTablesAndEngines, testSeedIdempotent, testProvenanceAudit, testForeignKeyFailure, testLiteralTextRoundTrip, testAtomicRollback, testImmutableReference, testMissingProvenanceRejected, testInvalidEngineID, testSchemaConstraints.

PHASE 2 - ISA atmosphere

Status: NOT EXECUTED

Files: matlab/atmosphere/ISA_model.m, matlab/utilities/flight_condition.m.

Dependencies: project_config.

Included test suites: TestAtmosphere.m.

Actually executed / reviewed: Static source review only. No MATLAB atmosphere results were executed.

Known limitations and unresolved issues: Geometric/geopotential convention and inconsistent blueprint checkpoints are explicitly corrected.

Test cases in the applicable shared suites:

- TestAtmosphere.m: testSeaLevel, testGeopotentialCheckpoints, testLayerContinuity, testGeometricConversion, testMachStagnation, testNegativeAltitude, testUpperBound, testInvalidMachAndThrottle.

PHASE 3 - Piston physics

Status: NOT EXECUTED

Files: matlab/engines/piston/Piston_Model.m, database/seed_piston_extension.sql, matlab/setup/db_seed_piston_extension.m.

Dependencies: Phase 1, Phase 2, propeller_model.

Included test suites: TestEngines.m.

Actually executed / reviewed: Host SQL check confirmed optional seed yields 13 engines. MATLAB physics was not executed.

Known limitations and unresolved issues: Opt-in extension outside PDF core; assumed naturally aspirated parameters.

Test cases in the applicable shared suites:

- TestEngines.m: testIdealCompressionCheckpoint, testNozzleChoked, testNozzleUnchoked, testNozzleBoundaryContinuity, testNoExpansionRejected, testAllCoreEnginesSLS, testTurbofanMassAndFanDemand, testTurbopropPowerAndStaticCase, testPropellerContinuousAndFinite, testKaveriWarningAndType, testAltitudeAndMachEnvelope, testAfterburnerRejected, testBadEfficiencyAndTemperature, testPistonPowerTorqueAndAltitude, testZeroThrottleState, testChangedParametersInvalidateFingerprint.

PHASE 4 - Turbojet physics

Status: NOT EXECUTED

Files: matlab/engines/turbojet/TJ_Model.m, matlab/engines/common.

Dependencies: Phase 1, Phase 2.

Included test suites: TestEngines.m.

Actually executed / reviewed: Common cycle/component source reviewed; MATLAB mass/energy and nozzle tests unexecuted.

Known limitations and unresolved issues: Constant properties and algebraic off-design flow; no afterburner.

Test cases in the applicable shared suites:

- TestEngines.m: testIdealCompressionCheckpoint, testNozzleChoked, testNozzleUnchoked, testNozzleBoundaryContinuity, testNoExpansionRejected, testAllCoreEnginesSLS, testTurbofanMassAndFanDemand, testTurbopropPowerAndStaticCase, testPropellerContinuousAndFinite, testKaveriWarningAndType, testAltitudeAndMachEnvelope, testAfterburnerRejected, testBadEfficiencyAndTemperature, testPistonPowerTorqueAndAltitude, testZeroThrottleState, testChangedParametersInvalidateFingerprint.

PHASE 5 - Turbofan physics

Status: NOT EXECUTED

Files: matlab/engines/turbofan/TF_Model.m, matlab/engines/common/gas_turbine_cycle.m.

Dependencies: Phase 4 components.

Included test suites: TestEngines.m.

Actually executed / reviewed: Static station, split-flow and work-demand review; no MATLAB execution.

Known limitations and unresolved issues: Separate exhaust, corrected fan work, no mixer or compressor map.

Test cases in the applicable shared suites:

- TestEngines.m: testIdealCompressionCheckpoint, testNozzleChoked, testNozzleUnchoked, testNozzleBoundaryContinuity, testNoExpansionRejected, testAllCoreEnginesSLS, testTurbofanMassAndFanDemand, testTurbopropPowerAndStaticCase, testPropellerContinuousAndFinite, testKaveriWarningAndType, testAltitudeAndMachEnvelope, testAfterburnerRejected, testBadEfficiencyAndTemperature, testPistonPowerTorqueAndAltitude, testZeroThrottleState, testChangedParametersInvalidateFingerprint.

PHASE 6 - Turboprop physics

Status: NOT EXECUTED

Files: matlab/engines/turboprop/TP_Model.m, matlab/engines/common/propeller_model.m.

Dependencies: Phase 4 components.

Included test suites: TestEngines.m.

Actually executed / reviewed: Static gas-generator/free-turbine/propeller review; no MATLAB power/reference result.

Known limitations and unresolved issues: Assumed residual pressure and propeller geometry. Rating agreement unresolved.

Test cases in the applicable shared suites:

- TestEngines.m: testIdealCompressionCheckpoint, testNozzleChoked, testNozzleUnchoked, testNozzleBoundaryContinuity, testNoExpansionRejected, testAllCoreEnginesSLS, testTurbofanMassAndFanDemand, testTurbopropPowerAndStaticCase, testPropellerContinuousAndFinite, testKaveriWarningAndType, testAltitudeAndMachEnvelope, testAfterburnerRejected, testBadEfficiencyAndTemperature, testPistonPowerTorqueAndAltitude, testZeroThrottleState, testChangedParametersInvalidateFingerprint.

PHASE 7 - Physics validation and automated testing

Status: WARNING

Files: matlab/validation, run_all_tests.m, tests.

Dependencies: Phases 1-6, matlab.unittest.

Included test suites: TestAtmosphere.m, TestEngines.m, TestDatabase.m.

Actually executed / reviewed: Host audit executed. MATLAB validation, sensitivity and regression baseline have NOT EXECUTED.

Known limitations and unresolved issues: No supplied verified primary engine targets, no accepted validation certificates.

Test cases in the applicable shared suites:

- TestAtmosphere.m: testSeaLevel, testGeopotentialCheckpoints, testLayerContinuity, testGeometricConversion, testMachStagnation, testNegativeAltitude, testUpperBound, testInvalidMachAndThrottle.
- TestEngines.m: testIdealCompressionCheckpoint, testNozzleChoked, testNozzleUnchoked, testNozzleBoundaryContinuity, testNoExpansionRejected, testAllCoreEnginesSLS, testTurbofanMassAndFanDemand, testTurbopropPowerAndStaticCase, testPropellerContinuousAndFinite, testKaveriWarningAndType, testAltitudeAndMachEnvelope, testAfterburnerRejected, testBadEfficiencyAndTemperature, testPistonPowerTorqueAndAltitude, testZeroThrottleState, testChangedParametersInvalidateFingerprint.
- TestDatabase.m: testAllTablesAndEngines, testSeedIdempotent, testProvenanceAudit, testForeignKeyFailure, testLiteralTextRoundTrip, testAtomicRollback, testImmutableReference, testMissingProvenanceRejected, testInvalidEngineID, testSchemaConstraints.

PHASE 8 - Digital Twin coordinator

Status: NOT EXECUTED

Files: matlab/digital_twin/DT_Coordinator.m, matlab/digital_twin/simulate_engine.m, matlab/setup/db_store_results.m.

Dependencies: Phases 1-7, ML predictor for L5.

Included test suites: TestIntegration.m.

Actually executed / reviewed: Static L1-L5 dispatch/persistence/fallback review. MATLAB integration tests unexecuted.

Known limitations and unresolved issues: Academic steady-state coordinator; no live sensor connection.

Test cases in the applicable shared suites:

- TestIntegration.m: testPersistenceAndLog, testFailedPointLogsWithoutPerformance, testLogImmutable, testArchitectureGridCounts, testGridValidityMaskAndTemperatureFields, testCompatibleComparison, testIncompatibleComparisonRejected, testAllMissionPresets, testUnverifiedReferencesCannotPass, testReportingContainsProvenance, testOutputUnitConversion, testHTMLTextEscaped.

PHASE 9 - Performance grids

Status: NOT EXECUTED

Files: matlab/performance/performance_grid.m, matlab/performance/evaluate_grid.m.

Dependencies: Phase 8.

Included test suites: TestIntegration.m.

Actually executed / reviewed: Configured cardinalities/source checked; full engine grids not generated.

Known limitations and unresolved issues: Invalid cells retained and excluded from training; some grid knots exceed an engine envelope by design.

Test cases in the applicable shared suites:

- TestIntegration.m: testPersistenceAndLog, testFailedPointLogsWithoutPerformance, testLogImmutable, testArchitectureGridCounts, testGridValidityMaskAndTemperatureFields, testCompatibleComparison, testIncompatibleComparisonRejected, testAllMissionPresets, testUnverifiedReferencesCannotPass, testReportingContainsProvenance, testOutputUnitConversion, testHTMLTextEscaped.

PHASE 10 - Comparison and analysis

Status: NOT EXECUTED

Files: matlab/comparison, matlab/reporting/plot_performance.m, matlab/reporting/plot_performance_map.m.

Dependencies: Phase 8, Phase 9.

Included test suites: TestIntegration.m.

Actually executed / reviewed: Static metric compatibility and sorting review; numerical tables/plots not executed.

Known limitations and unresolved issues: No unconditional comparison of jet TSFC to propeller BSFC.

Test cases in the applicable shared suites:

- TestIntegration.m: testPersistenceAndLog, testFailedPointLogsWithoutPerformance, testLogImmutable, testArchitectureGridCounts, testGridValidityMaskAndTemperatureFields, testCompatibleComparison, testIncompatibleComparisonRejected, testAllMissionPresets, testUnverifiedReferencesCannotPass, testReportingContainsProvenance, testOutputUnitConversion, testHTMLTextEscaped.

PHASE 11 - Mission-based selection

Status: NOT EXECUTED

Files: matlab/mission.

Dependencies: Phases 1,8,10.

Included test suites: TestMission.m.

Actually executed / reviewed: Host audit verifies seven mission seeds; MATLAB constraint/scoring tests unexecuted.

Known limitations and unresolved issues: Relative engine screening only; no airframe drag/range solver.

Test cases in the applicable shared suites:

- TestMission.m: testNormalisationDirections, testTieAndSingleMetric, testAllEliminated, testOnlyOneSurvives, testHighScoreCannotOverrideConstraint, testArchitectureAndCeiling, testWeightsRejected, testMixedSFCRejected.

PHASE 12 - ML dataset generation

Status: NOT EXECUTED

Files: matlab/ml/ml_generate_dataset.m, matlab/ml/ml_stratified_split.m, matlab/ml/ml_targets.m.

Dependencies: Phase 7 acceptance, Phase 9.

Included test suites: TestML.m.

Actually executed / reviewed: Static gate/split/domain review. No accepted physics dataset generated.

Known limitations and unresolved issues: Default validation gate closed. Synthetic physics data only.

Test cases in the applicable shared suites:

- TestML.m: testSplitDeterministicAndDisjoint, testDuplicateInputsRejected, testMetricsKnownErrors, testConstantAndZeroTargets, testPolynomialTrainPredict, testFourCandidateAPIs, testMLCellRejectsHolesAndExtrapolation, testMissingModelAndGate.

PHASE 13 - ML surrogate training and evaluation

Status: NOT EXECUTED

Files: matlab/ml/ml_train_surrogates.m, matlab/ml/ml_fit_candidate.m, matlab/ml/ml_apply_candidate.m, matlab/ml/ml_predict.m, matlab/ml/ml_metrics.m, matlab/ml/ml_diagnostic_report.m, matlab/ml/ml_learning_curve.m.

Dependencies: Phase 12, Statistics and Machine Learning Toolbox.

Included test suites: TestML.m.

Actually executed / reviewed: Static audit checked gate and validation-selection-before-test ordering. No training or performance benchmark executed.

Known limitations and unresolved issues: No trained MAT files or accuracy values. ANN uses fitrnet per output.

Test cases in the applicable shared suites:

- TestML.m: testSplitDeterministicAndDisjoint, testDuplicateInputsRejected, testMetricsKnownErrors, testConstantAndZeroTargets, testPolynomialTrainPredict, testFourCandidateAPIs, testMLCellRejectsHolesAndExtrapolation, testMissingModelAndGate.

PHASE 14 - MATLAB GUI

Status: NOT EXECUTED

Files: gui/EngineAnalyser_v1.m, gui/gui_build.m, gui/gui_action.m, gui.

Dependencies: Phases 1-13, MATLAB desktop.

Included test suites: TestGUI.m.

Actually executed / reviewed: Host audit checks callback action registry and source references. No desktop walkthrough.

Known limitations and unresolved issues: Programmatic AppBase .m instead of .mlapp Design View artifact.

Test cases in the applicable shared suites:

- TestGUI.m: testEightTabsAndKaveriWarning.

PHASE 15 - Full integration and reports

Status: WARNING

Files: run_project.m, verify_project.m, scripts/audit_dependencies.m, scripts/run_phase_tests.m, simulink, matlab/reporting, docs, reports.

Dependencies: All previous phases, Simulink for model execution.

Included test suites: TestIntegration.m, TestSimulink.m, TestGUI.m.

Actually executed / reviewed: 28 host checks passed. Documentation/PDF and archive checks are recorded separately. Full MATLAB/Simulink integration NOT EXECUTED.

Known limitations and unresolved issues: Text .mdl plus .slx builder; no generated .slx, real benchmark or fabricated publication history.

Test cases in the applicable shared suites:

- TestIntegration.m: testPersistenceAndLog, testFailedPointLogsWithoutPerformance, testLogImmutable, testArchitectureGridCounts, testGridValidityMaskAndTemperatureFields, testCompatibleComparison, testIncompatibleComparisonRejected, testAllMissionPresets, testUnverifiedReferencesCannotPass, testReportingContainsProvenance, testOutputUnitConversion, testHTMLTextEscaped.
- TestSimulink.m: testBuiltModelsMatchBackend.
- TestGUI.m: testEightTabsAndKaveriWarning.

Sources and verification scope

Primary project specification

ABACUS specification / blueprint v1.1, 77 pages, supplied by the user in two byte-identical PDF attachments. A single original copy is retained as ABACUS_specification_v1_1.pdf. It is the authority for architecture, terminology, engine set, database, Digital Twin levels, methodology and academic scope. Engine values transcribed from this secondary document remain unverified.

The user's accompanying text request adds the 15 implementation phases, piston support, packaging and explicit no-fabrication requirements. Conflicts are recorded in CHANGELOG.md.

Official implementation references consulted

- MathWorks native SQLite interface: https://www.mathworks.com/help/database/ug/sqlite.html
- MathWorks sqlread: https://www.mathworks.com/help/database/ug/sqlite.sqlread.html
- MathWorks sqlwrite: https://www.mathworks.com/help/database/ug/sqlite.sqlwrite.html
- MathWorks JDBC prepared statements, used to distinguish that API from the delivered native interface: https://www.mathworks.com/help/database/sql-prepared-statements.html
- MathWorks fitrnet: https://www.mathworks.com/help/stats/fitrnet.html
- MathWorks Level-2 MATLAB S-functions: https://www.mathworks.com/help/simulink/sfg/writing-level-2-matlab-s-functions.html
- NASA Glenn isentropic-flow relations: https://www.grc.nasa.gov/www/k-12/airplane/isentrop.html
- NASA Glenn compressible mass-flow relations: https://www.grc.nasa.gov/www/k-12/airplane/wcora.html

These sources support API/physical formulation choices. They are not evidence that a particular engine's proprietary parameters or the generated implementation have been experimentally validated. No cited engine source was promoted to independently verified manufacturer status.

Appendix - MATLAB source inventory

The PDF counterpart includes the complete source listing for these files. The original .m files in the package remain authoritative.

- gui/EngineAnalyser_v1.m
- gui/gui_action.m
- gui/gui_analysis_action.m
- gui/gui_build.m
- gui/gui_build_analysis_tabs.m
- gui/gui_build_mission_tab.m
- gui/gui_export_action.m
- gui/gui_mission_action.m
- gui/gui_mission_preset.m
- gui/gui_ml_action.m
- gui/gui_refresh_engine.m
- matlab/atmosphere/ISA_model.m
- matlab/comparison/compare_engines.m
- matlab/comparison/comparison_sweep.m
- matlab/digital_twin/DT_Coordinator.m
- matlab/digital_twin/simulate_engine.m
- matlab/engines/common/burner_stage.m
- matlab/engines/common/compress_stage.m
- matlab/engines/common/gas_turbine_cycle.m
- matlab/engines/common/nozzle_model.m
- matlab/engines/common/propeller_model.m
- matlab/engines/common/result_template.m
- matlab/engines/common/turbine_stage.m
- matlab/engines/piston/Piston_Model.m
- matlab/engines/turbofan/TF_Model.m
- matlab/engines/turbojet/TJ_Model.m
- matlab/engines/turboprop/TP_Model.m
- matlab/mission/EngineSelector.m
- matlab/mission/apply_hard_constraints.m
- matlab/mission/load_mission.m
- matlab/mission/normalise_metric.m
- matlab/mission/validate_mission.m
- matlab/ml/ml_apply_candidate.m
- matlab/ml/ml_diagnostic_report.m
- matlab/ml/ml_domain_valid.m

- matlab/ml/ml_fit_candidate.m
- matlab/ml/ml_generate_dataset.m
- matlab/ml/ml_learning_curve.m
- matlab/ml/ml_metrics.m
- matlab/ml/ml_polynomial_features.m
- matlab/ml/ml_predict.m
- matlab/ml/ml_stratified_split.m
- matlab/ml/ml_targets.m
- matlab/ml/ml_train_surrogates.m
- matlab/performance/evaluate_grid.m
- matlab/performance/performance_grid.m
- matlab/reporting/format_result_metrics.m
- matlab/reporting/generate_report.m
- matlab/reporting/html_escape.m
- matlab/reporting/plot_performance.m
- matlab/reporting/plot_performance_map.m
- matlab/reporting/result_row.m
- matlab/reporting/table_to_html.m
- matlab/setup/DB_Interface.m
- matlab/setup/db_add_reference.m
- matlab/setup/db_execute_script.m
- matlab/setup/db_get_engine.m
- matlab/setup/db_initialize.m
- matlab/setup/db_log_failure.m
- matlab/setup/db_seed.m
- matlab/setup/db_seed_piston_extension.m
- matlab/setup/db_store_results.m
- matlab/setup/project_config.m
- matlab/utilities/flight_condition.m
- matlab/utilities/model_fingerprint.m
- matlab/utilities/utc_timestamp.m
- matlab/utilities/write_text.m
- matlab/validation/parameter_sensitivity.m
- matlab/validation/regression_baseline.m
- matlab/validation/require_validation.m
- matlab/validation/validate_inputs.m
- matlab/validation/validate_outputs.m
- matlab/validation/validate_physics.m
- matlab/validation/validate_references.m

- project_paths.m
- run_all_tests.m
- run_project.m
- scripts/audit_dependencies.m
- scripts/run_phase_tests.m
- setup_project.m
- simulink/abacus_sfun.m
- simulink/build_simulink_models.m
- simulink/run_simulink_model.m
- tests/TestAtmosphere.m
- tests/TestDatabase.m
- tests/TestEngines.m
- tests/TestGUI.m
- tests/TestIntegration.m
- tests/TestML.m
- tests/TestMission.m
- tests/TestSimulink.m
- tests/new_test_database.m
- verify_project.m

Appendix: authoritative MATLAB source

The following listings reproduce all 93 delivered MATLAB source files. This appendix documents implementation; it is not executed validation evidence. Long lines wrap visually. Use the original .m files for execution.

gui/EngineAnalyser_v1.m

```
classdef EngineAnalyser_v1 < matlab.apps.AppBase
%ENGINEANALYSER_V1 Eight-tab App Designer-framework academic propulsion application.
% Input: optional DB path. Output: interactive app object. SI backend, display conversions.
% Programmatic AppBase source; not a binary .mlapp Design View file (see CHANGELOG).
    properties
        UIFigure
        DT
        Controls=struct()
        LastResults=table()
        LastResult=struct()
        LastSelection=struct()
        LastML=table()
    end
    methods
        function app=EngineAnalyser_v1(databaseFile)
            % Build the UI only after the backend connection succeeds.
            if nargin<1, p=project_paths(); databaseFile=p.database_file; end
            app.DT=DT_Coordinator(databaseFile);
            app.UIFigure=uifigure('Name','ABACUS | Aircraft Engine Analysis','Position',[80,60
,1240,850],...
                'Color',[.95,.97,.99],'Visible','off');
            app.UIFigure.CloseRequestFcn=@(~,~) delete(app);
            gui_build(app); registerApp(app,app.UIFigure); gui_refresh_engine(app);
            app.UIFigure.Visible='on';
        end
        function delete(app)
            % Close this app's DB and figure; other coordinators remain independent.
            if ~isempty(app.DT), delete(app.DT); app.DT=[]; end
            if ~isempty(app.UIFigure) && isvalid(app.UIFigure), delete(app.UIFigure); end
        end
    end
end
```

gui/gui_action.m

```
function gui_action(app,action)
%GUI_ACTION Catch user-facing callback errors and dispatch to functional backend modules.
% Input: app and closed action string. Output: updated UI/files; no duplicated equations.
try
    switch action
        case 'filter'
            t=app.DT.Database.read('v_Engine_Catalogue'); kind=app.Controls.Architecture.Value
;
            if ~strcmp(kind,'All'), t=t(string(t.Type_Name)==kind,:); end
            if isempty(t), error('ABACUS:NoEngines','No engines in that architecture. Run the
optional piston seed if needed.'); end
            app.Controls.Catalogue.Data=t; app.Controls.Engine.ItemsData=[];
            app.Controls.Engine.Items=cellstr(string(t.Designation));
            app.Controls.Engine.ItemsData=t.Engine_ID'; app.Controls.Engine.Value=t.Engine_ID(
1); gui_refresh_engine(app);
        case 'engine', gui_refresh_engine(app);
        case 'sources'
            p=project_paths(); open(fullfile(p.docs,'ABACUS_specification_v1_1.pdf'));
```

```
        case {'physics','predict','sweep','map','compare','overlay'}, gui_analysis_action(app,action);
        case {'dataset','train','ml_metrics'}, gui_ml_action(app,action);
        case 'mission_preset', gui_mission_preset(app);
        case 'mission', gui_mission_action(app);
        case 'validate'
            app.Controls.Banner.Text='Executing physics tests and reference comparisons...'; drawnow;
            v=validate_physics(app.DT.DatabaseFile); app.Controls.Validation.Data=v.TrackA;
            app.Controls.Banner.Text='Validation run finished. Inspect actual PASS / FAIL / WARNING rows in tab 7.';
        case {'database','units','csv','mat','png','pdf','report','verify'}, gui_export_action(app,action);
        otherwise, error('ABACUS:GUIAction','Unknown action %s.',action);
    end
catch exception
    e=[];
    try, e=db_get_engine(app.DT.Database,app.Controls.Engine.Value); catch, end
    message=string(exception.message);
    if ~isempty(e) && e.IsKaveri, c=project_config(); message=message+" | "+c.KaveriWarning; end
    app.Controls.Banner.Text=char(message); app.Controls.Banner.FontColor=[.70,.08,.08];
    uialert(app.UIFigure,char(message),'ABACUS diagnostic','Icon','warning');
end
end
```

gui/gui_analysis_action.m

```
function gui_analysis_action(app,action)
%GUI_ANALYSIS_ACTION Run current point/sweep/map/comparison through DT_Coordinator.
% GUI uses geometric metre input; grid axes use explicitly labelled geopotential metres.
id=app.Controls.Engine.Value;
f=flight_condition(app.Controls.Altitude.Value,app.Controls.Mach.Value,app.Controls.Throttle.Value);
e=db_get_engine(app.DT.Database,id);
switch action
    case {'physics','predict'}
        if strcmp(action,'predict'), level='L5'; else, level='L4'; end
        r=app.DT.run(id,f,level,true); app.LastResult=r; app.LastResults=result_row(r);
        app.Controls.Metrics.Data=format_result_metrics(r,app.Controls.Units.Value);
        app.Controls.Stations.Data=r.Stations;
        app.Controls.ResultSource.Text=sprintf('%s | %s | %s confidence',r.Status,r.Prediction_Source,r.Confidence);
        app.Controls.Banner.Text=char(strjoin(string(r.Warnings),' | '));
    case 'sweep'
        values=linspace(app.Controls.RangeMin.Value,app.Controls.RangeMax.Value,41);
        axis=app.Controls.Axis.Value; h=f.GeopotentialAltitude_m; m=f.Mach; theta=f.Throttle;
        switch axis
            case 'GeopotentialAltitude_m', h=values;
            case 'Mach', m=values;
            case 'Throttle', theta=values;
        end
        t=evaluate_grid(e,h,m,theta,app.DT.Database,false,'L2'); app.LastResults=t;
        app.Controls.EnvelopeTable.Data=t; plot_performance(t,axis,app.Controls.Metric.Value,app.Controls.EnvelopeAxes);
    case 'map'
        app.Controls.Banner.Text='Evaluating the architecture grid and retaining rejected-point diagnostics...'; drawnow;
        t=app.DT.map(id,true); app.LastResults=t; app.Controls.EnvelopeTable.Data=t;
        plot_performance_map(t,app.Controls.Metric.Value,1,app.Controls.EnvelopeAxes);
        app.Controls.Banner.Text=sprintf('%d valid / %d candidate points; estimated physics data.',nnz(t.Valid),height(t));
    case 'compare'
```

```
        t=compare_engines(app.DT,app.Controls.CompareIDs.Value,f,app.Controls.CompareMetric.Value);
        app.LastResults=t; app.Controls.CompareTable.Data=t; ax=app.Controls.CompareAxes; cla(ax);
        valid=logical(t.Valid); bar(ax,categorical(t.Engine(valid)),t.(app.Controls.CompareMetric.Value)(valid));
        ylabel(ax,strrep(app.Controls.CompareMetric.Value,'_',' ')); title(ax,'Same-condition academic comparison');
    case 'overlay'
        ref=app.DT.Database.read('v_Reference_Data'); ref=ref(ref.Engine_ID==id & ref.Afterburner==0,:);
        metric='Thrust_N'; if strcmp(e.Type,'Turboprop') || strcmp(e.Type,'Piston'), metric='ShaftPower_kW'; end
        ax=app.Controls.CompareAxes; cla(ax); hold(ax,'on');
        if ~isempty(ref)
            scatter(ax,ref.GeopotentialAltitude_m,ref.(metric),60,'DisplayName','Reference (source verification shown in table)');
            for i=1:height(ref)
                condition=flight_condition(ref.GeopotentialAltitude_m(i),ref.Mach(i),ref.Throttle(i),'geopotential');
                physics=app.DT.run(id,condition,'L1',false); model=app.DT.run(id,condition,'L5',false);
                scatter(ax,condition.GeopotentialAltitude_m,physics.(metric),45,'DisplayName','Physics, same reference condition');
                if strcmp(model.Prediction_Source,'ML'), scatter(ax,condition.GeopotentialAltitude_m,model.(metric),45,'DisplayName','ML, same reference condition'); end
            end
        end
        hold(ax,'off'); xlabel(ax,'Geopotential altitude (m)'); ylabel(ax,strrep(metric,'_',' ')); legend(ax,'Location','best');
        app.Controls.CompareTable.Data=ref;
end
if e.IsKaveri, c=project_config(); app.Controls.Banner.Text=[app.Controls.Banner.Text,' | ',c.KaveriWarning]; end
end
```

gui/gui_build.m

```
function gui_build(app)
%GUI_BUILD Assemble eight tabs and common controls; all callbacks dispatch to backend.
% Input: AppBase app. Output: app.Controls/UI components. Units labelled at each control.
root=uigridlayout(app.UIFigure,[4,1]); root.RowHeight={58,42,'1x',38};
header=uigridlayout(root,[1,5]); header.ColumnWidth={260,90,200,60,'1x'};
uilabel(header,'Text','ABACUS ENGINE ANALYSER','FontWeight','bold','FontSize',19,'FontColor',[.05,.18,.30]);
uilabel(header,'Text','Architecture');
app.Controls.Architecture=uidropdown(header,'Items',{'All','Turbojet','Turbofan_HighBPR','Turbofan_LowBPR','Turboprop','Piston'},...
    'ValueChangedFcn',@(~,~) gui_action(app,'filter'));
uilabel(header,'Text','Engine');
catalogue=app.DT.Database.read('v_Engine_Catalogue');
app.Controls.Engine=uidropdown(header,'Items',cellstr(string(catalogue.Designation)),...
    'ItemsData',catalogue.Engine_ID','Value',4,'ValueChangedFcn',@(~,~) gui_action(app,'engine'));
app.Controls.Banner=uilabel(root,'Text','Academic estimates; validation status is shown in tab 7.','WordWrap','on',...
    'FontColor',[.42,.23,.03],'BackgroundColor',[1,.94,.80]);
group=uitabgroup(root);
app.Controls.TabGroup=group;
names={'Engine Database Browser','Single-Point Analysis','Performance Envelope (L2/L3)',...
    'Digital Twin Comparison','ML Surrogate Manager','Mission-Based Engine Selection','Validation Report','Settings and Export'};
tabs=cell(1,8);
```

```
for i=1:8, tabs{i}=uitab(group,'Title',names{i}); end
% Tab 1: catalogue and per-value provenance.
g=uigridlayout(tabs{1},[3,1]); g.RowHeight={'1x','2x',32};
app.Controls.Catalogue=uitable(g,'Data',catalogue,'ColumnName',catalogue.Properties.VariableNames);
app.Controls.Provenance=uitable(g); uibutton(g,'Text','Open the project specification and source notes','ButtonPushedFcn',@(~,~) gui_action(app,'sources'));
% Tab 2: single-point controls, metrics and station totals.
g=uigridlayout(tabs{2},[3,2]); g.RowHeight={96,'1x',32};
form=uigridlayout(g,[2,6]); form.Layout.Column=[1,2];
labels={'Geometric altitude (m)','Mach (-)','Throttle (0 to 1)'};
for i=1:3, uilabel(form,'Text',labels{i}); end
uibutton(form,'Text','Run physics (L4)','ButtonPushedFcn',@(~,~) gui_action(app,'physics'));
uibutton(form,'Text','Predict (L5)','ButtonPushedFcn',@(~,~) gui_action(app,'predict'));
uilabel(form,'Text','Model / source shown below');
app.Controls.Altitude=uieditfield(form,'numeric','Limits',[0,32000],'Value',0);
app.Controls.Mach=uieditfield(form,'numeric','Limits',[0,3],'Value',0);
app.Controls.Throttle=uieditfield(form,'numeric','Limits',[0,1],'Value',1);
app.Controls.ResultSource=uilabel(form,'Text','No calculation yet'); app.Controls.ResultSource.Layout.Column=[4,6];
app.Controls.Metrics=uitable(g); app.Controls.Stations=uitable(g);
label=uilabel(g,'Text','Steady-state model only - transient dynamics are not represented. Station table uses TOTAL K and Pa.'); label.Layout.Column=[1,2];
gui_build_analysis_tabs(app,tabs);
gui_build_mission_tab(app,tabs{6});
% Tab 7: real execution results, with no preloaded pass claims.
g=uigridlayout(tabs{7},[3,1]); g.RowHeight={38,35,'1x'};
uibutton(g,'Text','Run physics tests and Track A reference comparison','ButtonPushedFcn',@(~,~) gui_action(app,'validate'));
uilabel(g,'Text','Unverified reference points cannot receive PASS or open the default ML training gate.','WordWrap','on');
app.Controls.Validation=uitable(g);
% Tab 8: selected database and export controls.
g=uigridlayout(tabs{8},[7,2]); g.RowHeight={38,38,38,38,38,38,'1x'};
uilabel(g,'Text','Database path'); app.Controls.DatabasePath=uieditfield(g,'text','Value',app.DT.DatabaseFile);
uibutton(g,'Text','Connect to database','ButtonPushedFcn',@(~,~) gui_action(app,'database'));
app.Controls.Units=uidropdown(g,'Items',{'SI','Imperial'},'ValueChangedFcn',@(~,~) gui_action(app,'units'));
uibutton(g,'Text','Export current table (CSV)','ButtonPushedFcn',@(~,~) gui_action(app,'csv'));
uibutton(g,'Text','Export current session (MAT)','ButtonPushedFcn',@(~,~) gui_action(app,'mat'));
uibutton(g,'Text','Export analysis plot (PNG)','ButtonPushedFcn',@(~,~) gui_action(app,'png'));
uibutton(g,'Text','Export analysis plot (PDF)','ButtonPushedFcn',@(~,~) gui_action(app,'pdf'));
uibutton(g,'Text','Create session report (HTML)','ButtonPushedFcn',@(~,~) gui_action(app,'report'));
uibutton(g,'Text','Verify project installation','ButtonPushedFcn',@(~,~) gui_action(app,'verify'));
c=project_config(); label=uilabel(g,'Text',['Model: ',c.Version]); label.Layout.Column=[1,2];
label=uilabel(g,'Text','SI values are used internally and for CSV export. The display selector converts the single-point metrics. GUI altitude inputs remain geometric metres. No afterburner, transient, hardware or certification model.','WordWrap','on'); label.Layout.Column=[1,2];
uilabel(root,'Text','Academic steady-state software Digital Twin | Reference, physics and ML remain distinct | Source provenance is available in tab 1',...
    'FontColor',[.15,.25,.35],'WordWrap','on');
end
```

gui/gui_build_analysis_tabs.m

```
function gui_build_analysis_tabs(app,tabs)
%GUI_BUILD_ANALYSIS_TABS Construct envelope, comparison and surrogate controls.
% Input: app and tabs; output: UI fields. Each button routes to an existing function.
g=uigridlayout(tabs{3},[3,1]); g.RowHeight={80,'1x',180};
bar=uigridlayout(g,[2,4]);
app.Controls.Axis=uidropdown(bar,'Items',{'GeopotentialAltitude_m','Mach','Throttle'});
app.Controls.Metric=uidropdown(bar,'Items',{'EquivalentThrust_N','Thrust_N','TSFC_kgNs','FuelFlow_kgs','ShaftPower_kW','BSFC_kgkWh','Eta_thermal'});
uibutton(bar,'Text','Run selected-axis sweep','ButtonPushedFcn',@(~,~) gui_action(app,'sweep'));
uibutton(bar,'Text','Generate full map','ButtonPushedFcn',@(~,~) gui_action(app,'map'));
uilabel(bar,'Text','Range minimum'); app.Controls.RangeMin=uieditfield(bar,'numeric','Value',0,'Limits',[0,32000]);
uilabel(bar,'Text','Range maximum'); app.Controls.RangeMax=uieditfield(bar,'numeric','Value',13000,'Limits',[0,32000]);
app.Controls.EnvelopeAxes=uiaxes(g); app.Controls.EnvelopeTable=uitable(g);
g=uigridlayout(tabs{4},[3,2]); g.RowHeight={105,'1x',180};
cat=app.DT.Database.read('v_Engine_Catalogue');
app.Controls.CompareIDs=uilistbox(g,'Items',cellstr(string(cat.Designation)),'ItemsData',cat.Engine_ID',...
    'Multiselect','on','Value',[4,5]);
bar=uigridlayout(g,[3,1]);
app.Controls.CompareMetric=uidropdown(bar,'Items',{'EquivalentThrust_N','Thrust_N','TSFC_kgNs','FuelFlow_kgs','ShaftPower_kW','BSFC_kgkWh'});
uibutton(bar,'Text','Compare selected engines at current inputs','ButtonPushedFcn',@(~,~) gui_action(app,'compare'));
uibutton(bar,'Text','Reference / physics / ML overlay for current engine','ButtonPushedFcn',@(~,~) gui_action(app,'overlay'));
app.Controls.CompareAxes=uiaxes(g); app.Controls.CompareAxes.Layout.Column=[1,2];
app.Controls.CompareTable=uitable(g); app.Controls.CompareTable.Layout.Column=[1,2];
g=uigridlayout(tabs{5},[4,1]); g.RowHeight={42,42,'1x',60};
bar=uigridlayout(g,[1,3]);
app.Controls.MLFamily=uidropdown(bar,'Items',{'Poly','RF','GPR','ANN'},'ValueChangedFcn',@(~,~) gui_action(app,'ml_metrics'));
uibutton(bar,'Text','Generate gated synthetic dataset','ButtonPushedFcn',@(~,~) gui_action(app,'dataset'));
uibutton(bar,'Text','Train and compare four families','ButtonPushedFcn',@(~,~) gui_action(app,'train'));
uibutton(g,'Text','Query selected operational surrogate (L5)','ButtonPushedFcn',@(~,~) gui_action(app,'predict'));
app.Controls.MLMetrics=uitable(g);
app.Controls.MLStatus=uilabel(g,'Text','No training has run in this session. Missing or unqualified models use physics fallback.', 'WordWrap','on');
end
```

gui/gui_build_mission_tab.m

```
function gui_build_mission_tab(app,tab)
%GUI_BUILD_MISSION_TAB Presets, custom constraints, weights, ranking and eliminations.
% All altitude inputs are geometric m. Zero maximum limits mean unconstrained.
g=uigridlayout(tab,[5,1]); g.RowHeight={35,125,78,45,'1x'};
t=app.DT.Database.read('Mission_Profiles');
app.Controls.MissionPreset=uidropdown(g,'Items',[cellstr(string(t.Mission_Name));{'Custom'}],...
    'ItemsData',[t.Mission_ID',0],'Value',1,'ValueChangedFcn',@(~,~) gui_action(app,'mission_preset'));
form=uigridlayout(g,[3,6]);
labels={'Altitude (m geometric)','Mach','Ceiling (m geometric)','Min thrust (kN)','Max dry mass (kg; 0 = none)','Min shaft power (kW)'};
for i=1:6, uilabel(form,'Text',labels{i},'WordWrap','on'); end
keys={'MissionAltitude','MissionMach','MissionCeiling','MissionThrust','MissionWeight','Missio
```

```
    nPower'};
12  for i=1:6, app.Controls.(keys{i})=uieditfield(form,'numeric','Value',0,'Limits',[0,Inf]); end
13  uilabel(form,'Text','Max TSFC (kg/kN/h; 0 = none)','WordWrap','on');
14  app.Controls.MissionTSFC=uieditfield(form,'numeric','Value',0,'Limits',[0,Inf]);
15  uilabel(form,'Text','Max BSFC (kg/kW/h; 0 = none)','WordWrap','on');
16  app.Controls.MissionBSFC=uieditfield(form,'numeric','Value',0,'Limits',[0,Inf]);
17  app.Controls.MissionArchitecture=uidropdown(form,'Items',{'Turbofan_HighBPR','Turbojet,Turbofa
    n_LowBPR','Turboprop','Piston'});
18  uilabel(form,'Text','Constraints apply before scoring.','WordWrap','on');
19  weights=uigridlayout(g,[2,5]); names={'Thrust','Efficiency','Fuel SFC','Altitude','Engine TWR'
    };
20  for i=1:5, uilabel(weights,'Text',[names{i},' weight']); end
21  app.Controls.Weights=gobjects(1,5);
22  for i=1:5, app.Controls.Weights(i)=uieditfield(weights,'numeric','Limits',[0,1],'Value',.2); e
    nd
23  bar=uigridlayout(g,[1,2]); uibutton(bar,'Text','Select engines','ButtonPushedFcn',@(~,~) gui_a
    ction(app,'mission'));
24  app.Controls.MissionMessage=uilabel(bar,'Text','Five weights must sum to 1.0.','WordWrap','on'
    );
25  app.Controls.MissionResults=uitable(g);
26  end
```

gui/gui_export_action.m

```
 1  function gui_export_action(app,action)
 2  %GUI_EXPORT_ACTION Save actual session artifacts and switch verified database connections.
 3  % Inputs: app, action. Output: CSV/MAT/PNG/PDF/HTML files. Cancellation is non-destructive.
 4  p=project_paths();
 5  switch action
 6      case 'database'
 7          file=app.Controls.DatabasePath.Value; db_initialize(file); db_seed(file);
 8          replacement=DT_Coordinator(file); previous=app.DT; app.DT=replacement; delete(previous
    );
 9          t=app.DT.Database.read('v_Engine_Catalogue'); app.Controls.Catalogue.Data=t;
10          app.Controls.Engine.ItemsData=[]; app.Controls.Engine.Items=cellstr(string(t.Designati
    on));
11          app.Controls.Engine.ItemsData=t.Engine_ID'; app.Controls.Engine.Value=t.Engine_ID(1);
    gui_refresh_engine(app);
12      case 'units'
13          if isfield(app.LastResult,'Condition'), app.Controls.Metrics.Data=format_result_metric
    s(app.LastResult,app.Controls.Units.Value); end
14      case 'csv'
15          if isempty(app.LastResults), error('ABACUS:NoExportData','Run an analysis before expor
    ting.'); end
16          [name,folder]=uiputfile('*.csv','Save results',fullfile(p.tables,'results.csv'));
17          if isequal(name,0), return; end
18          writetable(app.LastResults,fullfile(folder,name));
19      case 'mat'
20          [name,folder]=uiputfile('*.mat','Save session',fullfile(p.results,'session.mat'));
21          if isequal(name,0), return; end
22          result=app.LastResult; results=app.LastResults; selection=app.LastSelection; ml=app.La
    stML; %#ok<NASGU>
23          save(fullfile(folder,name),'result','results','selection','ml');
24      case {'png','pdf'}
25          [name,folder]=uiputfile(['*.',action],'Save plot',fullfile(p.plots,['performance.',act
    ion]));
26          if isequal(name,0), return; end
27          exportgraphics(app.Controls.EnvelopeAxes,fullfile(folder,name),'Resolution',200);
28      case 'report'
29          file=generate_report(app.LastResult,app.LastResults,app.LastSelection,app.LastML);
30          app.Controls.Banner.Text=['Report created: ',file];
31      case 'verify'
32          t=verify_project(app.DT.DatabaseFile); app.Controls.Validation.Data=t;
```

```
end
end
```

gui/gui_mission_action.m

```
function gui_mission_action(app)
%GUI_MISSION_ACTION Read custom constraints and show survivors plus eliminated engines.
% Inputs: app controls in labelled display units. Output: rankings without bypassing filters.
f=flight_condition(app.Controls.MissionAltitude.Value,app.Controls.MissionMach.Value,1);
ceiling=flight_condition(app.Controls.MissionCeiling.Value,0,1);
m=struct('Mission_ID',0,'Mission_Name','GUI custom / edited preset','Category','Custom',...
    'Architectures',app.Controls.MissionArchitecture.Value,'Altitude_m',f.GeopotentialAltitude_m,...
    'Ceiling_m',ceiling.GeopotentialAltitude_m,'Mach',f.Mach,'Min_Thrust_N',1000*app.Controls.MissionThrust.Value,...
    'Min_Power_kW',app.Controls.MissionPower.Value,'Max_TSFC_kgNs',app.Controls.MissionTSFC.Value/3.6e6,...
    'Max_BSFC_kgkWh',app.Controls.MissionBSFC.Value,'Max_DryWeight_kg',app.Controls.MissionWeight.Value);
for name=["Min_Power_kW","Max_TSFC_kgNs","Max_BSFC_kgkWh","Max_DryWeight_kg"]
    if m.(name)==0, m.(name)=NaN; end
end
names={'W_Thrust','W_Efficiency','W_Fuel','W_Altitude','W_TWR'};
for i=1:5, m.(names{i})=app.Controls.Weights(i).Value; end
s=EngineSelector(app.DT,m,[],true); app.LastSelection=s; app.Controls.MissionMessage.Text=char(s.Message);
if isempty(s.Ranking)
    t=s.Candidates; t.Score=nan(height(t),1);
else
    t=s.Candidates; t.Score=nan(height(t),1);
    for i=1:height(s.Ranking), t.Score(t.Engine_ID==s.Ranking.Engine_ID(i))=s.Ranking.Score(i); end
end
app.LastResults=t; app.Controls.MissionResults.Data=t;
removeStyle(app.Controls.MissionResults); rows=find(strlength(t.EliminationReason)>0);
if ~isempty(rows), addStyle(app.Controls.MissionResults,uistyle('BackgroundColor',[.90,.90,.90]),'row',rows); end
if any(contains(string(t.Engine),'Kaveri'))
    c=project_config(); app.Controls.Banner.Text=c.KaveriWarning;
end
end
```

gui/gui_mission_preset.m

```
function gui_mission_preset(app)
%GUI_MISSION_PRESET Copy a selected preset into editable controls with unit conversion.
% Input/output: app UI. This never runs selection or changes stored presets.
id=app.Controls.MissionPreset.Value; if id==0, return; end
m=load_mission(app.DT.Database,id); c=project_config();
app.Controls.MissionAltitude.Value=c.EarthRadius_m*m.Altitude_m/(c.EarthRadius_m-m.Altitude_m);
app.Controls.MissionMach.Value=m.Mach;
app.Controls.MissionCeiling.Value=c.EarthRadius_m*m.Ceiling_m/(c.EarthRadius_m-m.Ceiling_m);
app.Controls.MissionThrust.Value=m.Min_Thrust_N/1000;
app.Controls.MissionArchitecture.Value=char(string(m.Architectures));
names={'W_Thrust','W_Efficiency','W_Fuel','W_Altitude','W_TWR'};
for i=1:5, app.Controls.Weights(i).Value=m.(names{i}); end
app.Controls.MissionWeight.Value=0; app.Controls.MissionTSFC.Value=0;
app.Controls.MissionBSFC.Value=0; app.Controls.MissionPower.Value=0;
end
```

gui/gui_ml_action.m

```
function gui_ml_action(app,action)
%GUI_ML_ACTION Execute gated dataset/training actions and show actual saved metrics.
% Input/output: app UI. Runs serially; missing validation is surfaced by gui_action.
id=app.Controls.Engine.Value; p=project_paths();
switch action
    case 'dataset'
        app.Controls.MLStatus.Text='Checking physics validation, then generating synthetic data...'; drawnow;
        data=ml_generate_dataset(app.DT,id);
        app.Controls.MLStatus.Text=sprintf('%d valid synthetic points. Split fractions: %.3f / %.3f / %.3f.',size(data.X,1),data.Split.ActualFractions);
    case 'train'
        app.Controls.MLStatus.Text='Checking validation and training four families; this can take several minutes.'; drawnow;
        t=ml_train_surrogates(app.DT,id); app.Controls.MLMetrics.Data=t; app.LastML=t;
        app.Controls.MLStatus.Text='Training/evaluation finished. Actual test metrics are shown; low-accuracy models stay unavailable to L5.';
    case 'ml_metrics'
        filename=fullfile(p.ml_models,sprintf('engine_%d_%s.mat',id,app.Controls.MLFamily.Value));
        if ~isfile(filename), error('ABACUS:MissingML','That surrogate has not been trained.'); end
        loaded=load(filename,'bundle'); app.Controls.MLMetrics.Data=loaded.bundle.TestMetrics;
end
end
```

gui/gui_refresh_engine.m

```
function gui_refresh_engine(app)
%GUI_REFRESH_ENGINE Load provenance and show permanent estimate/envelope warnings.
% Input: app. Output: controls. Units: backend SI; limits shown as geometric altitude.
e=db_get_engine(app.DT.Database,app.Controls.Engine.Value); p=e.Params; c=project_config();
app.Controls.Provenance.Data=e.Provenance; app.Controls.Provenance.ColumnName=e.Provenance.Properties.VariableNames;
removeStyle(app.Controls.Provenance);
low=find(string(e.Provenance.Confidence_Level)=="Low");
if ~isempty(low), addStyle(app.Controls.Provenance,uistyle('BackgroundColor',[1,.94,.80]),'row',low); end
maxGeometric=min(32000,c.EarthRadius_m*p.MaxAltitude_m/(c.EarthRadius_m-p.MaxAltitude_m));
app.Controls.Altitude.Value=min(app.Controls.Altitude.Value,maxGeometric); app.Controls.Altitude.Limits=[0,maxGeometric];
app.Controls.Mach.Value=min(app.Controls.Mach.Value,p.MaxMach); app.Controls.Mach.Limits=[0,p.MaxMach];
app.Controls.RangeMax.Value=p.MaxAltitude_m;
message=sprintf('%s | %s confidence | H envelope 0-%.0f m; Mach 0-%.2f | Estimated academic parameters',e.Name,e.Confidence,p.MaxAltitude_m,p.MaxMach);
if e.IsKaveri, message=[message,' | ',c.KaveriWarning]; end
app.Controls.Banner.Text=message;
app.Controls.Banner.FontColor=[.42,.23,.03];
app.Controls.ResultSource.Text='Select Run to evaluate this engine.';
app.LastResult=struct(); app.Controls.Metrics.Data=table(); app.Controls.Stations.Data=table();
gui_mission_preset(app);
end
```

matlab/atmosphere/ISA_model.m

```
function a = ISA_model(altitude_m, mach, convention)
%ISA_MODEL Three-layer ISA, with geometric/geopotential conversion and total state.
% Inputs: altitude [m], Mach [-], convention (default geometric). Output: SI struct.
% Ref: ABACUS 16; hydrostatic perfect gas. No extrapolation above H=32000 m.
if nargin < 2, mach = 0; end
if nargin < 3, convention = 'geometric'; end
f = flight_condition(altitude_m,mach,1,convention);
c = project_config(); H = f.GeopotentialAltitude_m;
P11 = c.P0_Pa*(216.65/c.T0_K)^(c.g0/(.0065*c.R_air));
P20 = P11*exp(-c.g0*9000/(c.R_air*216.65));
if H <= 11000
    T = c.T0_K-.0065*H;
    P = c.P0_Pa*(T/c.T0_K)^(c.g0/(.0065*c.R_air));
elseif H <= 20000
    T = 216.65; P = P11*exp(-c.g0*(H-11000)/(c.R_air*T));
else
    T = 216.65+.001*(H-20000);
    P = P20*(T/216.65)^(-c.g0/(.001*c.R_air));
end
rho = P/(c.R_air*T); sound = sqrt(c.gamma_air*c.R_air*T);
mu = 1.716e-5*(T/273.15)^1.5*(273.15+110.4)/(T+110.4);
tau = 1+(c.gamma_air-1)/2*mach^2;
a = struct('Altitude_m',f.Altitude_m,'GeopotentialAltitude_m',H,...
    'T_K',T,'P_Pa',P,'Density_kgm3',rho,'SoundSpeed_ms',sound,...
    'DynamicViscosity_Pas',mu,'KinematicViscosity_m2s',mu/rho,...
    'Gravity_ms2',c.g0*(c.EarthRadius_m/(c.EarthRadius_m+f.Altitude_m))^2,...
    'Mach',mach,'Velocity_ms',mach*sound,'Tt_K',T*tau,...
    'Pt_Pa',P*tau^(c.gamma_air/(c.gamma_air-1)),...
    'DynamicPressure_Pa',.5*rho*(mach*sound)^2);
end
```

matlab/comparison/compare_engines.m

```
function t = compare_engines(dt,engineIDs,f,metric)
%COMPARE_ENGINES Same-condition performance table with explicit compatibility checks.
% Inputs: coordinator, engine IDs, flight condition, metric field. Output: ranked SI table.
% Ref: ABACUS 21-22. TSFC/BSFC and jet/shaft power cannot be ranked interchangeably.
if nargin<4, metric='EquivalentThrust_N'; end
allowed={'Thrust_N','EquivalentThrust_N','TSFC_kgNs','FuelFlow_kgs','ShaftPower_kW','BSFC_kgkW
h'};
validatestring(metric,allowed); rows=cell(numel(engineIDs),1);
for i=1:numel(engineIDs), rows{i}=result_row(dt.run(engineIDs(i),f,'L1',false)); end
t=vertcat(rows{:});
if any(isnan(t.(metric)(logical(t.Valid))))
    error('ABACUS:IncompatibleMetric','%s is not defined for every selected architecture. Use
equivalent thrust or select compatible engines.',metric);
end
ascending=ismember(metric,{'TSFC_kgNs','FuelFlow_kgs','BSFC_kgkWh'});
if ascending, direction='ascend'; else, direction='descend'; end
t=sortrows(t,metric,direction,'MissingPlacement','last');
t.ComparisonNote=repmat("Same operating condition; uninstalled academic outputs.",height(t),1)
;
end
```

matlab/comparison/comparison_sweep.m

```
function t = comparison_sweep(dt,engineIDs,axisName,values,f,metric)
%COMPARISON_SWEEP Compare altitude, Mach or throttle trends through the same backend.
% Inputs: DT, IDs, axis name, vector, baseline f, metric. Output: tidy comparison table.
% Units: H in geopotential m, Mach/throttle dimensionless; errors propagate.
validatestring(axisName,{'Altitude','Mach','Throttle'}); parts=cell(numel(values),1);
for i=1:numel(values)
    h=f.GeopotentialAltitude_m; m=f.Mach; theta=f.Throttle;
    switch axisName
        case 'Altitude', h=values(i);
        case 'Mach', m=values(i);
        case 'Throttle', theta=values(i);
    end
    parts{i}=compare_engines(dt,engineIDs,flight_condition(h,m,theta,'geopotential'),metric);
end
t=vertcat(parts{:});
end
```

matlab/digital_twin/DT_Coordinator.m

```
classdef DT_Coordinator < handle
%DT_COORDINATOR Academic steady-state software Digital Twin, levels L1-L5.
% Input: optional DB path. Outputs: run result, sweep/map tables, persisted metadata.
% Model equations live only in engine modules; L5 falls back explicitly to physics.
    properties (SetAccess=private)
        Database
        DatabaseFile
    end
    methods
        function obj=DT_Coordinator(databaseFile)
            % Open a database for this coordinator, with independent connection lifetime.
            if nargin<1, p=project_paths(); databaseFile=p.database_file; end
            obj.DatabaseFile=char(databaseFile); obj.Database=DB_Interface(databaseFile);
        end
        function r=run(obj,engineID,f,level,storeResult)
            % L1 single point; L4 logs by default; L5 checks surrogate freshness/domain.
            if nargin<4, level='L4'; end
            validatestring(level,{'L1','L4','L5'});
            if nargin<5, storeResult=ismember(level,{'L4','L5'}); end
            try
                e=db_get_engine(obj.Database,engineID);
            catch exception
                if storeResult, db_log_failure(obj.Database,engineID,f,level,exception); end
                rethrow(exception);
            end
            if strcmp(level,'L5')
                [r,available,message]=ml_predict(e,f);
                if ~available
                    r=simulate_engine(e,f); r.Warnings(end+1)=string(message);
                    if r.Valid, r.Status='WARNING'; end
                end
            else
                r=simulate_engine(e,f);
            end
            if storeResult, db_store_results(obj.Database,{r},level); end
        end
        function t=sweep(obj,engineID,altitudes,mach,throttle,storeResult)
            % L2 sweep accepts geopotential altitude vector [m]. Invalid points stay flagged.
            if nargin<6, storeResult=false; end
            e=db_get_engine(obj.Database,engineID);
            t=evaluate_grid(e,altitudes,mach,throttle,obj.Database,storeResult,'L2');
        end
```

```
43          function t=map(obj,engineID,storeResult)
44              % L3 map uses the exact architecture grid; L4 persistence is optional.
45              if nargin<3, storeResult=false; end
46              e=db_get_engine(obj.Database,engineID); grid=performance_grid(e);
47              t=evaluate_grid(e,grid.Altitude_m,grid.Mach,grid.Throttle,obj.Database,storeResult
    ,'L3');
48          end
49          function delete(obj)
50              % Release the coordinator's connection without closing other objects' DBs.
51              if ~isempty(obj.Database), delete(obj.Database); obj.Database=[]; end
52          end
53      end
54  end
```

matlab/digital_twin/simulate_engine.m

```
 1  function r = simulate_engine(e,f)
 2  %SIMULATE_ENGINE Route the architecture and convert cycle errors to visible results.
 3  % Inputs: loaded engine and flight structs. Output: standard result r, including FAIL.
 4  % Units: SI. Missing DB IDs fail before this function; no numeric result on failure.
 5  r = result_template(e,f); runTimer = tic;
 6  try
 7      switch e.Type
 8          case 'Turbojet', r = TJ_Model(e,f);
 9          case {'Turbofan_HighBPR','Turbofan_LowBPR'}, r = TF_Model(e,f);
10          case 'Turboprop', r = TP_Model(e,f);
11          case 'Piston', r = Piston_Model(e,f);
12          otherwise, error('ABACUS:MissingModel','No model exists for architecture %s.',e.Type);
13      end
14  catch exception
15      r.Status='FAIL'; r.Valid=false; r.Error_Code=exception.identifier;
16      r.Warnings(end+1)=string(exception.message);
17      if e.IsKaveri, c=project_config(); r.Warnings(end+1)=string(c.KaveriWarning); end
18  end
19  r.Elapsed_ms = toc(runTimer)*1000;
20  end
```

matlab/engines/common/burner_stage.m

```
 1  function s = burner_stage(T3,P3,T4,eta,loss,cpAir,cpGas,LHV)
 2  %BURNER_STAGE Fuel-air ratio from total-enthalpy conservation, including fuel mass.
 3  % Inputs: K, Pa, K, efficiency, fractional pressure loss, cp J/kg/K, LHV J/kg.
 4  % Output: f [-], total state, energy residual [J/kg air]. Ref: ABACUS 17.2.3.
 5  if T4<=T3, error('ABACUS:ERR_COMB_01','Turbine inlet temperature must exceed compressor exit t
    emperature.'); end
 6  denominator = eta*LHV-cpGas*T4;
 7  if denominator<=0, error('ABACUS:ERR_FAR_01','Combustor fuel energy denominator is not positiv
    e.'); end
 8  f = (cpGas*T4-cpAir*T3)/denominator;
 9  if f<=0 || f>=.1, error('ABACUS:ERR_FAR_02','Fuel-air ratio is outside (0,0.1).'); end
10  s = struct('Tt_K',T4,'Pt_Pa',P3*(1-loss),'FuelAirRatio',f,...
11      'EnergyResidual_Jkg',cpAir*T3+f*eta*LHV-(1+f)*cpGas*T4);
12  end
```

matlab/engines/common/compress_stage.m

```
 1  function s = compress_stage(Tin,Pin,ratio,eta,cp,gamma)
 2  %COMPRESS_STAGE Isentropic-efficiency compression in total properties.
 3  % Inputs: Tin K, Pin Pa, pressure ratio, eta, cp J/kg/K, gamma. Output: stage struct.
 4  % Ref: ABACUS 17.2.2; throws on nonphysical conditions. Work is J/kg of stage flow.
 5  if min([Tin,Pin,cp])<=0 || ratio<1 || eta<=0 || eta>1 || gamma<=1
 6      error('ABACUS:Compressor','Invalid compressor inputs.');
```

```
end
Tout = Tin*(1+(ratio^((gamma-1)/gamma)-1)/eta);
s = struct('Tt_K',Tout,'Pt_Pa',Pin*ratio,'Work_Jkg',cp*(Tout-Tin));
end
```

matlab/engines/common/gas_turbine_cycle.m

```
function r = gas_turbine_cycle(e,f)
%GAS_TURBINE_CYCLE Authoritative shared TJ/TF/TP steady-state thermodynamic kernel.
% Inputs: e from db_get_engine; f from flight_condition. Output: standard SI result.
% Ref: ABACUS 17; departures and off-design assumptions are in CHANGELOG/ASSUMPTIONS.
% Rated thrust/power never scales predictions; efficiencies never fit reference data.
r = result_template(e,f); r.Warnings = validate_inputs(e,f);
p = e.Params; c = project_config(); a = ISA_model(f.GeopotentialAltitude_m,f.Mach,'geopotentia
l');
if f.Throttle==0
    r.Valid = true; r.Status = 'WARNING'; r.FuelFlow_kgs = 0;
    r.EquivalentThrust_N = 0; r.Thrust_N = 0; r.ShaftPower_kW = 0;
    r.Warnings(end+1) = "Engine-off convention: zero net output; windmilling drag is outside s
cope.";
    return
end
isFan = contains(e.Type,'Turbofan'); isProp = strcmp(e.Type,'Turboprop');
ram = 1;
if f.Mach>1, ram = 1-.075*(f.Mach-1)^1.35; end
T2 = a.Tt_K; P2 = a.Pt_Pa*p.Inlet_PR_recovery*ram;
% Fixed corrected-flow approximation, independently documented, not a compressor map.
flowFactor = .35+.65*f.Throttle;
mdot = p.MassFlow_kgs*(P2/(c.P0_Pa*p.Inlet_PR_recovery))/sqrt(T2/c.T0_K)*flowFactor;
opr = 1+(p.OPR-1)*f.Throttle; alpha = 0;
fan = struct('Tt_K',T2,'Pt_Pa',P2,'Work_Jkg',0);
if isFan
    alpha = p.BPR; fanPR = 1+(p.Fan_PR-1)*f.Throttle;
    fan = compress_stage(T2,P2,fanPR,p.Eta_fan,c.cp_air,c.gamma_air);
    corePR = opr/fanPR;
else
    corePR = opr;
end
mc = mdot/(1+alpha); mb = mdot-mc;
comp = compress_stage(fan.Tt_K,fan.Pt_Pa,corePR,p.Eta_compressor,c.cp_air,c.gamma_air);
T4 = comp.Tt_K+f.Throttle*(p.T_t4_K-comp.Tt_K);
burn = burner_stage(comp.Tt_K,comp.Pt_Pa,T4,p.Eta_combustor,p.Burner_PR_loss,c.cp_air,c.cp_gas
,c.FuelLHV_Jkg);
far = burn.FuelAirRatio; mg = mc*(1+far); fuel = mc*far;
% Fan Work_Jkg already contains cp: do not multiply it by cp a second time.
demand = comp.Work_Jkg+(1+alpha)*fan.Work_Jkg;
turb = turbine_stage(T4,burn.Pt_Pa,demand,far,p.Eta_mechanical,p.Eta_turbine,c.cp_gas,c.gamma_
gas);
nozzleT = turb.Tt_K; nozzleP = turb.Pt_Pa;
prop = struct('Thrust_N',0); shaft = 0; p7 = NaN; t7 = NaN;
if isProp
    p7 = a.P_Pa*p.PowerTurbine_BackPressure_Ratio;
    if p7>=turb.Pt_Pa, error('ABACUS:PowerTurbine','No pressure head remains for the power tur
bine.'); end
    t7 = turb.Tt_K*(1-p.Eta_power_turbine*(1-(p7/turb.Pt_Pa)^((c.gamma_gas-1)/c.gamma_gas)));
    shaft = mg*c.cp_gas*(turb.Tt_K-t7)*p.Eta_mechanical;
    propPower = shaft*p.Eta_gearbox;
    prop = propeller_model(propPower,p.Eta_propeller,a.Density_kgm3,p.Prop_Diameter_m,a.Veloci
ty_ms,a.SoundSpeed_ms,p.Prop_RPM);
    nozzleT = t7; nozzleP = p7;
    r.ShaftPower_kW = propPower/1000; r.PropShaftPower_kW = propPower/1000;
    r.BSFC_kgkWh = fuel*3600/r.ShaftPower_kW;
end
```

```
core = nozzle_model(nozzleT,nozzleP,a.P_Pa,mg,c.gamma_gas,c.R_gas,p.Eta_nozzle_core);
coreForce = core.GrossThrust_N-mc*a.Velocity_ms;
bypassForce = 0; bypassEnergy = 0;
if isFan && mb>0
    bypass = nozzle_model(fan.Tt_K,fan.Pt_Pa,a.P_Pa,mb,c.gamma_air,c.R_air,p.Eta_nozzle_bypass);
    bypassForce = bypass.GrossThrust_N-mb*a.Velocity_ms;
    bypassEnergy = .5*mb*(bypass.ExpandedVelocity_ms^2-a.Velocity_ms^2);
    r.Bypass_Nozzle_State = bypass.State;
end
net = coreForce+bypassForce+prop.Thrust_N;
if net<=0 || ~isfinite(net), error('ABACUS:NonpositiveNetThrust','No positive propulsive output at this operating point.'); end
jetEnergy = .5*(mg*core.ExpandedVelocity_ms^2-mc*a.Velocity_ms^2)+bypassEnergy;
if isProp
    convertedPower = jetEnergy+propPower;
    r.Thrust_N = NaN;
    r.ESHP = (propPower+coreForce*a.Velocity_ms/p.Eta_propeller)/745.6998715822702;
else
    convertedPower = jetEnergy;
    r.Thrust_N = net; r.TSFC_kgNs = fuel/net;
end
r.EquivalentThrust_N = net; r.FuelFlow_kgs = fuel;
r.Eta_thermal = convertedPower/(fuel*c.FuelLHV_Jkg);
r.Eta_propulsive = net*a.Velocity_ms/convertedPower;
r.Eta_overall = net*a.Velocity_ms/(fuel*c.FuelLHV_Jkg);
r.T_t4_K = T4; r.P_t3_Pa = comp.Pt_Pa; r.Core_Nozzle_State = core.State;
ids = [0;2;2.5;3;4;5]; temps = [a.Tt_K;T2;fan.Tt_K;comp.Tt_K;T4;turb.Tt_K];
pressures = [a.Pt_Pa;P2;fan.Pt_Pa;comp.Pt_Pa;burn.Pt_Pa;turb.Pt_Pa];
flows = [mdot;mdot;mdot;mc;mg;mg];
if ~isFan, ids(3)=[]; temps(3)=[]; pressures(3)=[]; flows(3)=[]; end
if isProp, ids(end+1)=7; temps(end+1)=t7; pressures(end+1)=p7; flows(end+1)=mg; end
r.Stations = table(ids,temps,pressures,flows,'VariableNames',{'Station','Tt_K','Pt_Pa','MassFlow_kgs'});
r.Stations = [r.Stations;table(8,core.Tt_K,core.Pt_Pa,mg,'VariableNames',r.Stations.Properties.VariableNames)];
if isFan && mb>0
    r.Stations = [r.Stations;table(18,bypass.Tt_K,bypass.Pt_Pa,mb,'VariableNames',r.Stations.Properties.VariableNames)];
end
r.Diagnostics = struct('CoreNozzle',core,'CompressorWork_Jkg',comp.Work_Jkg,...
    'FanWork_Jkg',fan.Work_Jkg,'CoreFlow_kgs',mc,'BypassFlow_kgs',mb,...
    'GasFlow_kgs',mg,'AirFlow_kgs',mdot,'FuelAirRatio',far,'Propeller',prop,...
    'MassResidual_kgs',mg+mb-mdot-fuel,'BurnerResidual_W',mc*burn.EnergyResidual_Jkg,...
    'ShaftResidual_W',mc*turb.WorkResidual_Jkg,'GasGeneratorPower_W',shaft,...
    'CoreThrust_N',coreForce,'BypassThrust_N',bypassForce,'JetEnergyRate_W',jetEnergy,...
    'Model_OPR',opr,'InletRecovery',ram*p.Inlet_PR_recovery);
if isFan && mb>0, r.Diagnostics.BypassNozzle = bypass; end
if far>.07, r.Warnings(end+1) = "Fuel-air ratio exceeds 0.07; near the academic model limit."; end
r = validate_outputs(r);
end
```

matlab/engines/common/nozzle_model.m

```
function n = nozzle_model(Tt,Pt,Pa,mdot,gamma,R,eta)
%NOZZLE_MODEL Adiabatic converging nozzle with sonic cap and pressure thrust.
% Inputs: total T [K], total/ambient P [Pa], gas flow [kg/s], gamma, R [J/kg/K], eta.
% Output: exit state, area [m2], momentum/pressure forces [N], critical NPR.
% eta is kinetic-energy efficiency; choking threshold follows the same energy law.
% Ref: ABACUS 17.2.5, NASA isentropic equations, CHANGELOG C05. No fixed geometry.
validateattributes([Tt,Pt,Pa,mdot,gamma,R,eta],{'numeric'},{'real','finite','vector'});
if min([Tt,Pt,Pa,mdot,R])<=0 || gamma<=1 || eta<=0 || eta>1
```

```
 9      error('ABACUS:ERR_NOZ_01','Nozzle needs positive state, flow, R, gamma>1 and 0<eta<=1.');
10  end
11  if Pt<=Pa, error('ABACUS:ERR_NOZ_02','Nozzle total pressure must exceed ambient pressure.'); e
    nd
12  cp = gamma*R/(gamma-1); exponent = (gamma-1)/gamma;
13  criticalBase = 1-(gamma-1)/((gamma+1)*eta);
14  if criticalBase<=0, error('ABACUS:NozzleEfficiency','Efficiency too small for the sonic nozzle
     formulation.'); end
15  critical = criticalBase^(-gamma/(gamma-1));
16  choked = Pt/Pa >= critical;
17  if choked, Pe = Pt/critical; else, Pe = Pa; end
18  Te = Tt*(1-eta*(1-(Pe/Pt)^exponent));
19  Ve = sqrt(2*cp*(Tt-Te)); Me = Ve/sqrt(gamma*R*Te);
20  rho = Pe/(R*Te); Ae = mdot/(rho*Ve);
21  pressureForce = (Pe-Pa)*Ae;
22  % Remaining ideal expansion energy provides an explicit pressure-energy convention.
23  expandedVelocity = sqrt(Ve^2+2*cp*Te*(1-(Pa/Pe)^exponent));
24  if choked, state = 'CHOKED'; else, state = 'UNCHOKED'; end
25  exitTotalPressure = Pe*(1+(gamma-1)/2*Me^2)^(gamma/(gamma-1));
26  n = struct('Tt_K',Tt,'Pt_Pa',exitTotalPressure,'Inlet_Pt_Pa',Pt,'T_K',Te,'P_Pa',Pe,'Velocity_m
    s',Ve,...
27      'Mach',Me,'MassFlow_kgs',mdot,'Area_m2',Ae,'PressureThrust_N',pressureForce,...
28      'GrossThrust_N',mdot*Ve+pressureForce,'ExpandedVelocity_ms',expandedVelocity,...
29      'Choked',choked,'State',state,'NPR',Pt/Pa,'NPR_critical',critical,...
30      'EnergyResidual_W',mdot*(cp*Tt-cp*Te-.5*Ve^2));
31  end
```

matlab/engines/common/propeller_model.m

```
 1  function prop = propeller_model(power_W,eta,rho,diameter,V,a,rpm)
 2  %PROPELLER_MODEL Static actuator-disc thrust with C1 blending into cruise thrust.
 3  % Inputs: shaft W, efficiency [-], density kg/m3, diameter m, V/a m/s, rpm rev/min.
 4  % Outputs: thrust N, shaft torque N m, advance ratio, useful power W.
 5  % Ref: ABACUS 17.4.3. eta at static is an effective disc-power factor, not FV/P.
 6  if power_W<0 || eta<=0 || eta>1 || rho<=0 || diameter<=0 || V<0 || a<=0 || rpm<=0
 7      error('ABACUS:Propeller','Invalid propeller input.');
 8  end
 9  A = pi*diameter^2/4; useful = eta*power_W;
10  static = (2*rho*A*useful^2)^(1/3); transition = .05*a;
11  if V >= transition
12      thrust = useful/V;
13  else
14      x = V/transition; weight = x^2*(3-2*x);
15      % Algebraically evaluate weight/V without division at V=0.
16      thrust = (1-weight)*static+useful*(3*x-2*x^2)/transition;
17  end
18  prop = struct('Thrust_N',thrust,'StaticThrust_N',static,...
19      'ShaftTorque_Nm',power_W/(rpm*2*pi/60),'AdvanceRatio',V/(rpm/60*diameter),...
20      'UsefulPower_W',thrust*V,'DiscArea_m2',A);
21  end
```

matlab/engines/common/result_template.m

```
 1  function r = result_template(e,f)
 2  %RESULT_TEMPLATE Define one output contract for physics, ML and error paths.
 3  % Inputs: engine and flight structures. Output: r with SI fields and explicit status.
 4  % NaN denotes unavailable/inapplicable quantities, never a valid predicted value.
 5  c = project_config();
 6  r = struct('Engine_ID',e.ID,'Engine',e.Name,'Architecture',e.Type,'Condition',f,...
 7      'Valid',false,'Status','FAIL','Error_Code','','Warnings',strings(0,1),...
 8      'Data_Type','Simulated','Prediction_Source','Physics','Confidence',e.Confidence,...
 9      'Model_Version',c.Version,'Model_Fingerprint',e.Fingerprint,'Timestamp_UTC',utc_timestamp(
```

```
    ),...
10      'Thrust_N',NaN,'EquivalentThrust_N',NaN,'ShaftPower_kW',NaN,'PropShaftPower_kW',NaN,...
11      'FuelFlow_kgs',NaN,'TSFC_kgNs',NaN,'BSFC_kgkWh',NaN,'Eta_thermal',NaN,...
12      'Eta_propulsive',NaN,'Eta_overall',NaN,'ESHP',NaN,'T_t4_K',NaN,'P_t3_Pa',NaN,...
13      'Core_Nozzle_State','N/A','Bypass_Nozzle_State','N/A','Stations',table(),...
14      'Diagnostics',struct(),'Provenance',e.Provenance,'Elapsed_ms',NaN,'ML_Model_Type','');
15  if e.Estimated, r.Data_Type = 'Simulated_Estimated'; end
16  end
```

matlab/engines/common/turbine_stage.m

```
 1  function s = turbine_stage(Tin,Pin,work,f,etaM,etaT,cp,gamma)
 2  %TURBINE_STAGE Balance compressor/fan shaft demand including fuel-added gas mass.
 3  % Inputs: total K/Pa, demand J/kg core air, f, efficiencies, cp J/kg/K, gamma.
 4  % Output: state and residual J/kg air. Ref: ABACUS 17.2.4; invalid expansion aborts.
 5  Tout = Tin-work/(etaM*(1+f)*cp);
 6  base = 1-(1-Tout/Tin)/etaT;
 7  if Tout<=0 || Tout>=Tin || base<=0 || base>=1
 8      error('ABACUS:ERR_TURB_02','Turbine work demand cannot be supplied by the specified cycle.
    ');
 9  end
10  Pout = Pin*base^(gamma/(gamma-1));
11  s = struct('Tt_K',Tout,'Pt_Pa',Pout,...
12      'WorkResidual_Jkg',etaM*(1+f)*cp*(Tin-Tout)-work);
13  end
```

matlab/engines/piston/Piston_Model.m

```
 1  function r = Piston_Model(e,f)
 2  %PISTON_MODEL Optional naturally aspirated four/two-stroke brake-power model.
 3  % Inputs: provenance-bearing engine e, standard flight f. Output: SI performance.
 4  % Airflow = rho*displacement*RPM/(30*strokes)*volumetric_efficiency*throttle.
 5  % Brake efficiency is explicit; Otto efficiency is an upper-bound check, not a fit.
 6  % Ref: user phase 3; this architecture is outside ABACUS's 11-engine core.
 7  if ~strcmp(e.Type,'Piston'), error('ABACUS:Architecture','Piston_Model requires a piston engin
    e.'); end
 8  r = result_template(e,f); r.Warnings = validate_inputs(e,f); p = e.Params;
 9  a = ISA_model(f.GeopotentialAltitude_m,f.Mach,'geopotential');
10  if f.Throttle==0
11      r.Valid=true; r.Status='WARNING'; r.FuelFlow_kgs=0; r.ShaftPower_kW=0; r.EquivalentThrust_
    N=0;
12      r.Warnings(end+1)="Engine-off convention; windmilling is excluded."; return
13  end
14  c = project_config(); idealOtto = 1-p.Compression_Ratio^(1-c.gamma_air);
15  if p.Brake_Thermal_Efficiency>=idealOtto
16      error('ABACUS:PistonEfficiency','Brake efficiency must be below the ideal Otto cycle limit
    .');
17  end
18  rpm = p.Rated_RPM; air = a.Density_kgm3*p.Displacement_m3*rpm/(30*p.Strokes)*p.Volumetric_Effi
    ciency*f.Throttle;
19  fuel = air/p.AFR; power = fuel*p.Fuel_LHV_Jkg*p.Brake_Thermal_Efficiency;
20  prop = propeller_model(power,p.Eta_propeller,a.Density_kgm3,p.Prop_Diameter_m,a.Velocity_ms,a.
    SoundSpeed_ms,rpm);
21  r.ShaftPower_kW=power/1000; r.PropShaftPower_kW=power/1000; r.FuelFlow_kgs=fuel;
22  r.BSFC_kgkWh=fuel*3600/r.ShaftPower_kW; r.EquivalentThrust_N=prop.Thrust_N;
23  r.Eta_thermal=p.Brake_Thermal_Efficiency; r.Eta_propulsive=prop.UsefulPower_W/power;
24  r.Eta_overall=prop.UsefulPower_W/(fuel*p.Fuel_LHV_Jkg);
25  r.Diagnostics=struct('RPM',rpm,'Torque_Nm',power/(rpm*2*pi/60),'AirFlow_kgs',air,...
26      'IdealOttoEfficiency',idealOtto,'Propeller',prop,'MassResidual_kgs',0,...
27      'BurnerResidual_W',0,'ShaftResidual_W',0);
28  r.Warnings(end+1)="Optional piston extension: manifold pressure, detailed heat transfer and su
    percharging are not modelled.";
```

```
29  r = validate_outputs(r);
30  end
```

matlab/engines/turbofan/TF_Model.m

```
1  function r = TF_Model(e,f)
2  %TF_MODEL Turbofan architecture entry point to the shared cycle.
3  % Inputs: engine struct e and flight struct f. Output: SI result r.
4  % Ref: ABACUS 17. Model errors propagate; incorrect architecture is rejected.
5  if ~contains(e.Type,'Turbofan'), error('ABACUS:Architecture','TF_Model needs a Turbofan engine
   .'); end
6  r = gas_turbine_cycle(e,f);
7  end
```

matlab/engines/turbojet/TJ_Model.m

```
1  function r = TJ_Model(e,f)
2  %TJ_MODEL Turbojet architecture entry point to the shared cycle.
3  % Inputs: engine struct e and flight struct f. Output: SI result r.
4  % Ref: ABACUS 17. Model errors propagate; incorrect architecture is rejected.
5  if ~contains(e.Type,'Turbojet'), error('ABACUS:Architecture','TJ_Model needs a Turbojet engine
   .'); end
6  r = gas_turbine_cycle(e,f);
7  end
```

matlab/engines/turboprop/TP_Model.m

```
1  function r = TP_Model(e,f)
2  %TP_MODEL Turboprop architecture entry point to the shared cycle.
3  % Inputs: engine struct e and flight struct f. Output: SI result r.
4  % Ref: ABACUS 17. Model errors propagate; incorrect architecture is rejected.
5  if ~contains(e.Type,'Turboprop'), error('ABACUS:Architecture','TP_Model needs a Turboprop engi
   ne.'); end
6  r = gas_turbine_cycle(e,f);
7  end
```

matlab/mission/EngineSelector.m

```
1  function selection = EngineSelector(dt,mission,engineIDs,persist)
2  %ENGINESELECTOR Apply hard filters, normalise survivors, then rank by weighted score.
3  % Inputs: DT handle, mission struct/ID, optional engine vector, persistence flag.
4  % Output: selection struct with survivors, elimination reasons and metric contributions.
5  % Ref: ABACUS 21. Engine-level TWR only; TP uses explicitly simulated static thrust.
6  if ~isstruct(mission), mission=load_mission(dt.Database,mission); end
7  weights=validate_mission(mission);
8  if nargin<3 || isempty(engineIDs), t=dt.Database.read('Engines'); engineIDs=t.Engine_ID; end
9  if nargin<4, persist=true; end
10 f=flight_condition(mission.Altitude_m,mission.Mach,1,'geopotential');
11 rows=cell(numel(engineIDs),1); constants=project_config();
12 for i=1:numel(engineIDs)
13     e=db_get_engine(dt.Database,engineIDs(i)); r=simulate_engine(e,f); t=result_row(r);
14     ceiling=simulate_engine(e,flight_condition(mission.Ceiling_m,mission.Mach,1,'geopotential'
   ));
15     t.Ceiling_OK=ceiling.Valid && ceiling.EquivalentThrust_N>0;
16     t.DryWeight_kg=e.Params.DryWeight_kg; t.ModelCeiling_m=e.Params.MaxAltitude_m;
17     if ismember(e.Type,{'Turboprop','Piston'})
18         static=simulate_engine(e,flight_condition(0,0,1)); rated=static.EquivalentThrust_N;
19         t.TWR_Basis="Simulated SLS equivalent thrust / dry engine weight";
20     else
21         rated=e.Params.Rated_Thrust_N; t.TWR_Basis="Catalogue dry rated thrust / dry engine we
   ight";
```

```
    end
    t.Engine_TWR=rated/(e.Params.DryWeight_kg*constants.g0); rows{i}=t;
end
candidates=vertcat(rows{:}); [mask,reasons]=apply_hard_constraints(candidates,mission);
% Missing scoring inputs are also disqualifying, rather than silently imputed.
for i=1:height(candidates)
    if ~isfinite(candidates.Engine_TWR(i))
        mask(i)=false; reasons(i)=strtrim(reasons(i)+"; Missing TWR basis");
    end
end
candidates.EliminationReason=reasons;
valid=candidates(mask,:); excluded=candidates(~mask,:); ranking=table(); notes=strings(0,1);
if isempty(valid)
    message="No engine in the database meets all hard constraints for this mission. Relax one or more constraints to proceed.";
elseif height(valid)==1
    message="Only one engine meets all hard constraints; no comparative ranking is produced.";
else
    if all(ismember(string(valid.Architecture),["Turboprop","Piston"]))
        fuelMetric=valid.BSFC_kgkWh; fuelName='BSFC';
    else
        fuelMetric=valid.TSFC_kgNs; fuelName='TSFC';
    end
    metrics=[valid.EquivalentThrust_N,valid.Eta_thermal,fuelMetric,valid.ModelCeiling_m,valid.Engine_TWR];
    scores=zeros(size(metrics)); labels={'Thrust','Thermal efficiency',fuelName,'Model envelope ceiling','Engine TWR'};
    for j=1:5
        [scores(:,j),tied]=normalise_metric(metrics(:,j),j~=3);
        if tied, notes(end+1)="All feasible engines have identical "+labels{j}+"; this criterion does not differentiate candidates."; end
    end
    contributions=scores.*weights; ranking=valid;
    ranking.Score=sum(contributions,2);
    names={'Thrust_Contribution','Efficiency_Contribution','Fuel_Contribution','Altitude_Contribution','TWR_Contribution'};
    for j=1:5, ranking.(names{j})=contributions(:,j); end
    ranking=sortrows(ranking,'Score','descend'); ranking.Rank=(1:height(ranking))';
    message=ranking.Engine(1)+" has the highest score after all hard constraints; see the five contribution columns.";
end
selection=struct('Mission',mission,'Candidates',candidates,'Feasible',valid,'Eliminated',excluded,...
    'Ranking',ranking,'Message',message,'Notes',notes,'Weights',weights,...
    'Scope','Academic engine comparison; no aircraft drag, range or flight-planning model.');
if persist
    log=struct('Engine_ID',NaN,'DT_Level',"L4",'Model_Type',"Physics",...
        'Inputs_JSON',string(jsonencode(mission)),'Outputs_JSON',string(jsonencode(selection)),...
        'Model_Version',string(constants.Version),'Event_Type',"MissionSelection",'Timestamp_UTC',string(utc_timestamp()));
    dt.Database.append('Simulation_Log',struct2table(log));
end
end
```

matlab/mission/apply_hard_constraints.m

```
function [feasible,reasons] = apply_hard_constraints(candidates,mission)
%APPLY_HARD_CONSTRAINTS Eliminate each engine that fails any mandatory requirement.
% Inputs: evaluated candidate table, mission struct. Outputs: logical mask and reasons.
% Units: N, kg/(N s), kg/(kW h), kW, kg, m. Ref: ABACUS 21.1; no scoring here.
validate_mission(mission); n=height(candidates); feasible=true(n,1); reasons=strings(n,1);
architectures=strtrim(split(string(mission.Architectures),','));
```

```
for i=1:n
    problems=strings(0,1);
    if ~candidates.Valid(i), problems(end+1)="Invalid model operating point"; end
    if ~ismember(string(candidates.Architecture(i)),architectures), problems(end+1)="Architect
ure mismatch"; end
    if ~candidates.Ceiling_OK(i), problems(end+1)="Required altitude ceiling unavailable"; end
    rules={'Min_Thrust_N','EquivalentThrust_N','min';'Max_TSFC_kgNs','TSFC_kgNs','max';...
        'Min_Power_kW','ShaftPower_kW','min';'Max_BSFC_kgkWh','BSFC_kgkWh','max';...
        'Max_DryWeight_kg','DryWeight_kg','max'};
    for j=1:size(rules,1)
        field=rules{j,1};
        if ~isfield(mission,field) || isnan(mission.(field)), continue; end
        value=candidates.(rules{j,2})(i); limit=mission.(field);
        failed=~isfinite(value);
        if strcmp(rules{j,3},'min'), failed=failed || value<limit; else, failed=failed || valu
e>limit; end
        if failed, problems(end+1)=string(field); end
    end
    feasible(i)=isempty(problems); reasons(i)=strjoin(problems,'; ');
end
end
```

matlab/mission/load_mission.m

```
function mission = load_mission(db,id)
%LOAD_MISSION Retrieve an exact named or numbered preset from Mission_Profiles.
% Inputs: DB handle, ID/name. Output: mission struct in SI and geopotential m.
% Ref: ABACUS Table 21.2. Missing/ambiguous presets are rejected.
t=db.read('Mission_Profiles');
if isnumeric(id), match=t.Mission_ID==id; else, match=string(t.Mission_Name)==string(id); end
if nnz(match)~=1, error('ABACUS:MissionID','Mission preset not found.'); end
mission=table2struct(t(match,:));
end
```

matlab/mission/normalise_metric.m

```
function [score,tied] = normalise_metric(value,higherBetter)
%NORMALISE_METRIC Min-max normalisation of FEASIBLE candidates only.
% Input: finite numeric vector, direction flag. Outputs: scores [-], tie flag.
% Ref: ABACUS 21.2. Equal values score 0.5; empty input returns an empty vector.
if nargin<2, higherBetter=true; end
if isempty(value), score=value; tied=false; return; end
validateattributes(value,{'numeric'},{'real','finite','vector'});
lo=min(value); hi=max(value); tied=(hi==lo);
if tied, score=.5*ones(size(value)); return; end
score=(value-lo)/(hi-lo);
if ~higherBetter, score=1-score; end
end
```

matlab/mission/validate_mission.m

```
function w = validate_mission(mission)
%VALIDATE_MISSION Enforce finite conditions, valid constraints and five weights.
% Input: mission struct. Output: 1x5 weights [-]. Ref: ABACUS 21; errors are explicit.
names={'W_Thrust','W_Efficiency','W_Fuel','W_Altitude','W_TWR'};
w=zeros(1,5);
for i=1:5, w(i)=mission.(names{i}); end
if any(~isfinite(w)) || any(w<0 | w>1) || abs(sum(w)-1)>1e-6
    error('ABACUS:MissionWeights','Five weights must be finite, nonnegative and sum to one.');
end
flight_condition(mission.Altitude_m,mission.Mach,1,'geopotential');
validateattributes(mission.Ceiling_m,{'numeric'},{'scalar','real','finite','>=',0,'<=',32000})
```

```
    ;
12  allowed=["Turbojet","Turbofan_HighBPR","Turbofan_LowBPR","Turboprop","Piston"];
13  architectures=strtrim(split(string(mission.Architectures),','));
14  if any(~ismember(architectures,allowed)), error('ABACUS:MissionArchitecture','Unknown mission
    architecture.'); end
15  if any(ismember(architectures,["Turboprop","Piston"])) && any(ismember(architectures,allowed(1
    :3)))
16      error('ABACUS:MissionMetric','A mission ranking cannot mix shaft BSFC and jet TSFC.');
17  end
18  for name=["Min_Thrust_N","Max_TSFC_kgNs","Min_Power_kW","Max_BSFC_kgkWh","Max_DryWeight_kg"]
19      if isfield(mission,name) && ~isnan(mission.(name))
20          validateattributes(mission.(name),{'numeric'},{'real','finite','scalar','nonnegative'}
    );
21      end
22  end
23  end
```

matlab/ml/ml_apply_candidate.m

```
 1  function Y = ml_apply_candidate(model,X)
 2  %ML_APPLY_CANDIDATE Predict with stored training scales; never refit on query data.
 3  % Inputs: fitted candidate, n-by-3 SI features. Output: n-by-target SI predictions.
 4  % Prediction errors propagate to the coordinator's physics fallback.
 5  Z=(X-model.Mu)./model.Sigma; Y=zeros(size(X,1),numel(model.Objects));
 6  for j=1:numel(model.Objects)
 7      object=model.Objects{j};
 8      if strcmp(model.Family,'Poly')
 9          A=ml_polynomial_features(Z,object.Degree,object.Powers); pred=A*object.Beta;
10      else
11          pred=predict(object,Z);
12      end
13      Y(:,j)=double(pred)*model.YSigma(j)+model.YMu(j);
14  end
15  end
```

matlab/ml/ml_diagnostic_report.m

```
 1  function ml_diagnostic_report(data,predicted,metrics,engineID)
 2  %ML_DIAGNOSTIC_REPORT Save test residuals and prediction/reference plots with SI labels.
 3  % Inputs: dataset, held-out predictions, metrics, engine ID. Outputs: CSV and PNG artifacts.
 4  % Ref: ABACUS Track B; references here are physics data only. Never used for model selection.
 5  p=project_paths(); test=data.Split.Test; folder=fullfile(p.reports,sprintf('engine_%d_ml',engi
    neID));
 6  if ~isfolder(folder), mkdir(folder); end
 7  t=array2table(data.X(test,:),'VariableNames',data.FeatureNames);
 8  for j=1:numel(data.TargetNames)
 9      name=data.TargetNames{j}; ref=data.Y(test,j); pred=predicted(:,j);
10      t.([name,'_Physics'])=ref; t.([name,'_ML'])=pred; t.([name,'_Residual'])=pred-ref;
11      fig=figure('Visible','off','Color','w'); cleanup=onCleanup(@() close(fig));
12      tiledlayout(fig,1,2); ax=nexttile; scatter(ax,ref,pred,8,'filled'); hold(ax,'on');
13      limits=[min(ref),max(ref)]; plot(ax,limits,limits,'k--'); grid(ax,'on');
14      xlabel(ax,'Held-out physics'); ylabel(ax,'ML prediction'); title(ax,strrep(name,'_',' '));
15      ax=nexttile; scatter(ax,data.X(test,1),data.X(test,2),15,pred-ref,'filled'); colorbar(ax);
16      xlabel(ax,'Mach'); ylabel(ax,'Geopotential altitude (m)'); title(ax,'Held-out residual map
    ');
17      exportgraphics(fig,fullfile(folder,[name,'.png']),'Resolution',150);
18      clear cleanup
19  end
20  writetable(t,fullfile(folder,'held_out_residuals.csv')); writetable(metrics,fullfile(folder,'m
    etrics.csv'));
21  end
```

matlab/ml/ml_domain_valid.m

```
function valid = ml_domain_valid(bundle,f)
%ML_DOMAIN_VALID Require a query to lie inside a cell with valid physics at every corner.
% Inputs: bundle grid/mask and standard flight f. Output: logical, no extrapolation.
% Units: H m, Mach, throttle. Bounding-box-only interpolation across rejected holes is prohibited.
axesValues={bundle.Grid.Altitude_m,bundle.Grid.Mach,bundle.Grid.Throttle};
values=[f.GeopotentialAltitude_m,f.Mach,f.Throttle]; indices=cell(1,3); valid=false;
for d=1:3
    axis=axesValues{d}; x=values(d);
    if ~isfinite(x) || x<min(axis) || x>max(axis), return; end
    lower=find(axis<=x,1,'last'); upper=find(axis>=x,1,'first'); indices{d}=unique([lower,upper]);
end
mask=bundle.ValidityMask(indices{1},indices{2},indices{3}); valid=all(mask(:));
end
```

matlab/ml/ml_fit_candidate.m

```
function model = ml_fit_candidate(X,Y,family,setting)
%ML_FIT_CANDIDATE Fit one family to training data only, with train-only scaling.
% Inputs: X n-by-3, Y n-by-target, family, hyperparameter setting. Output: model struct.
% Ref: ABACUS 20.3. ANN uses fitrnet (Statistics Toolbox, R2021a+), one target at a time.
if nargin<4, setting=1; end
validateattributes(X,{'numeric'},{'2d','finite','real','ncols',3});
validateattributes(Y,{'numeric'},{'2d','finite','real','nrows',size(X,1)});
family=char(validatestring(family,{'Poly','RF','GPR','ANN'}));
mu=mean(X,1); sigma=std(X,0,1); sigma(sigma==0)=1; Z=(X-mu)./sigma;
yMu=mean(Y,1); ySigma=std(Y,0,1); ySigma(ySigma==0)=1; target=(Y-yMu)./ySigma;
model=struct('Family',family,'Setting',setting,'Mu',mu,'Sigma',sigma,...
    'YMu',yMu,'YSigma',ySigma,'Objects',{cell(1,size(Y,2))});
for j=1:size(Y,2)
    y=target(:,j);
    switch family
        case 'Poly'
            degree=3+2*(setting==2); [A,powers]=ml_polynomial_features(Z,degree);
            lambda=1e-6; penalty=eye(size(A,2)); penalty(1,1)=0;
            beta=[A;sqrt(lambda)*penalty]\[y;zeros(size(A,2),1)];
            model.Objects{j}=struct('Beta',beta,'Powers',powers,'Degree',degree);
        case 'RF'
            trees=100+100*(setting==2);
            model.Objects{j}=TreeBagger(trees,Z,y,'Method','regression','MinLeafSize',2,...
                'NumPredictorsToSample',2,'OOBPrediction','off');
        case 'GPR'
            kernels={'ardsquaredexponential','ardmatern52'};
            model.Objects{j}=fitrgp(Z,y,'KernelFunction',kernels{setting},'Standardize',false,...
                'FitMethod','exact','PredictMethod','exact');
        case 'ANN'
            sizes={[64,32,16],[32,16]};
            model.Objects{j}=fitrnet(Z,y,'LayerSizes',sizes{setting},'Activations','relu',...
                'Lambda',1e-4,'Standardize',false,'IterationLimit',500);
    end
end
end
```

matlab/ml/ml_generate_dataset.m

```
function data = ml_generate_dataset(dt,engineID)
%ML_GENERATE_DATASET Generate synthetic data only after current physics acceptance.
% Inputs: DT coordinator and engine ID. Output: dataset struct and MAT/CSV artifacts.
% Ref: ABACUS 20. Invalid grid rows retained separately; only valid rows enter training.
e=db_get_engine(dt.Database,engineID); certificate=require_validation(e);
grid=performance_grid(e); allRows=dt.map(engineID,false); targets=ml_targets(e.Type); c=project_config();
valid=logical(allRows.Valid) & all(isfinite(allRows{:,targets}),2);
t=allRows(valid,:); X=t{:,c.MLFeatures}; Y=t{:,targets};
split=ml_stratified_split(X,c.RandomSeed);
data=struct('Label','Physics-generated synthetic data','Engine_ID',e.ID,'Engine',e.Name,...
    'Architecture',e.Type,'Confidence',e.Confidence,'Fingerprint',e.Fingerprint,...
    'ModelVersion',c.Version,'FeatureNames',{c.MLFeatures},'TargetNames',{targets},...
    'X',X,'Y',Y,'Split',split,'Grid',grid,'ValidityMask',reshape(valid,[numel(grid.Altitude_m),numel(grid.Mach),numel(grid.Throttle)]),...
    'ValidationCertificate',certificate,'Timestamp_UTC',utc_timestamp());
p=project_paths(); folder=fullfile(p.results,'ml_datasets'); if ~isfolder(folder), mkdir(folder); end
save(fullfile(folder,sprintf('engine_%d_dataset.mat',e.ID)),'data','-v7.3');
writetable(t,fullfile(folder,sprintf('engine_%d_synthetic.csv',e.ID)));
writetable(allRows(~valid,:),fullfile(folder,sprintf('engine_%d_rejected.csv',e.ID)));
end
```

matlab/ml/ml_learning_curve.m

```
function curve = ml_learning_curve(data,family,targetIndex)
%ML_LEARNING_CURVE Assess sample-size sensitivity on validation data only.
% Inputs: current synthetic dataset, family, target index. Output: size/error table.
% Training subsets are deterministic. Test set remains untouched. Ref: ABACUS 23.3.
if nargin<3, targetIndex=1; end
train=find(data.Split.Train); val=data.Split.Validation; old=rng; clean=onCleanup(@() rng(old)); rng(42,'twister');
train=train(randperm(numel(train))); fractions=[.25,.5,.75,1]; pieces=cell(4,1);
for i=1:4
    count=max(10,round(fractions(i)*numel(train))); count=min(count,numel(train)); subset=train(1:count);
    model=ml_fit_candidate(data.X(subset,:),data.Y(subset,targetIndex),family,1);
    prediction=ml_apply_candidate(model,data.X(val,:)); metric=ml_metrics(data.Y(val,targetIndex),prediction,data.TargetNames(targetIndex));
    metric.TrainingCount=count; pieces{i}=metric;
end
curve=vertcat(pieces{:});
end
```

matlab/ml/ml_metrics.m

```
function t = ml_metrics(reference,predicted,names)
%ML_METRICS Track B metrics against held-out physics values, not engine measurements.
% Inputs: matching numeric matrices, output names. Output: table, target-specific SI errors.
% MAPE excludes true zeros and reports count. R2 is NaN for a constant target.
if ~isequal(size(reference),size(predicted)), error('ABACUS:MetricShape','Prediction/reference dimensions differ.'); end
parts=cell(size(reference,2),1);
for j=1:size(reference,2)
    y=reference(:,j); p=predicted(:,j);
    if any(~isfinite(y)|~isfinite(p)), error('ABACUS:MetricFinite','Metrics require finite values.'); end
    errorValue=p-y; rmse=sqrt(mean(errorValue.^2)); mae=mean(abs(errorValue)); total=sum((y-mean(y)).^2);
    if total<=eps(max(1,sum(y.^2))), r2=NaN; else, r2=1-sum(errorValue.^2)/total; end
```

```
    nonzero=abs(y)>1e-12*max(1,max(abs(y)));
    if any(nonzero), mape=100*mean(abs(errorValue(nonzero)./y(nonzero))); else, mape=NaN; end
    scale=mean(abs(y)); nrmse=rmse/max(scale,eps); nmae=mae/max(scale,eps);
    accepted=isfinite(r2)&&r2>=.95&&nrmse<.02&&nmae<.015&&mape<5;
    parts{j}=table(string(names{j}),r2,rmse,mae,mape,nrmse,nmae,nnz(nonzero),accepted,...
        'VariableNames',{'Target','R2','RMSE','MAE','MAPE_Percent','RelativeRMSE','RelativeMAE','MAPE_Count','MeetsAccuracyCriteria'});
end
t=vertcat(parts{:});
end
```

matlab/ml/ml_polynomial_features.m

```
function [features,powers] = ml_polynomial_features(X,degree,powers)
%ML_POLYNOMIAL_FEATURES Complete monomial basis up to a total degree, including bias.
% Inputs: standardised predictors, degree, optional fitted powers. Output: matrix/basis.
% Units: dimensionless. Deterministic; no external polyfitn dependency.
if nargin<3
    validateattributes(degree,{'numeric'},{'scalar','integer','>=',1,'<=',5});
    if size(X,2)~=3, error('ABACUS:MLFeatures','Exactly three predictors are required.'); end
    powers=zeros(0,3);
    for a=0:degree
        for b=0:degree-a
            for c=0:degree-a-b, powers(end+1,:)=[a,b,c]; end %#ok<AGROW>
        end
    end
end
features=ones(size(X,1),size(powers,1));
for j=1:size(powers,1), features(:,j)=prod(X.^powers(j,:),2); end
end
```

matlab/ml/ml_predict.m

```
function [r,available,message] = ml_predict(e,f)
%ML_PREDICT Load the selected current surrogate and enforce domain/accuracy bounds.
% Inputs: loaded engine and flight f. Outputs: result, availability, fallback reason.
% Ref: ABACUS L5. Missing toolbox/model, stale model and invalid cells fall back safely.
r=result_template(e,f); available=false; message="ML surrogate not available; physics model used.";
p=project_paths(); filename=fullfile(p.ml_models,sprintf('engine_%d_best.mat',e.ID));
if ~isfile(filename), return; end
predictionTimer=tic;
try
    warnings=validate_inputs(e,f); loaded=load(filename,'bundle'); b=loaded.bundle;
    if ~strcmp(b.Fingerprint,e.Fingerprint), message="ML model is stale after model/data changes; physics model used."; return; end
    if ~b.Operational, message="ML model did not meet measured acceptance criteria; physics model used."; return; end
    if ~ml_domain_valid(b,f), message="Query outside validated ML cells; physics model used."; return; end
    prediction=ml_apply_candidate(b.Model,[f.Mach,f.GeopotentialAltitude_m,f.Throttle]);
    for i=1:numel(b.TargetNames), r.(b.TargetNames{i})=prediction(i); end
    a=ISA_model(f.GeopotentialAltitude_m,f.Mach,'geopotential'); c=project_config();
    if ismember(e.Type,{'Turboprop','Piston'})
        r.PropShaftPower_kW=r.ShaftPower_kW; r.FuelFlow_kgs=r.BSFC_kgkWh*r.ShaftPower_kW/3600;
    else
        r.EquivalentThrust_N=r.Thrust_N; r.FuelFlow_kgs=r.TSFC_kgNs*r.Thrust_N;
    end
    LHV=c.FuelLHV_Jkg; if strcmp(e.Type,'Piston'), LHV=e.Params.Fuel_LHV_Jkg; end
    r.Eta_overall=r.EquivalentThrust_N*a.Velocity_ms/(r.FuelFlow_kgs*LHV);
    if ~ismember('Eta_propulsive',b.TargetNames), r.Eta_propulsive=r.Eta_overall/r.Eta_thermal; end
```

```
    if abs(r.Eta_overall-r.Eta_thermal*r.Eta_propulsive)>.03
        message="ML efficiency identities disagree beyond tolerance; physics model used."; return
    end
    r.Prediction_Source='ML'; r.Data_Type='ML_Predicted'; r.ML_Model_Type=b.Model.Family;
    r.Warnings=[warnings;"ML approximates synthetic physics data; it does not validate real engine performance."];
    r.Diagnostics=struct('TestMetrics',b.TestMetrics,'RawPrediction',prediction,'Label',b.Label);
    r=validate_outputs(r); r.Elapsed_ms=toc(predictionTimer)*1000; available=true; message="ML prediction available.";
catch exception
    message="ML unavailable ("+string(exception.message)+"); physics model used.";
end
end
```

matlab/ml/ml_stratified_split.m

```
function split = ml_stratified_split(X,seed)
%ML_STRATIFIED_SPLIT Approximately 70/15/15 within joint altitude/Mach bins.
% Input: unique [Mach,H_m,throttle] rows, RNG seed. Output: disjoint index masks.
% Ref: ABACUS 20.4. Small bins are reported; no target data influence the split.
if nargin<2, seed=42; end
validateattributes(X,{'numeric'},{'2d','real','finite','ncols',3});
if size(X,1)<40, error('ABACUS:MLSampleCount','At least 40 unique operating points are required.'); end
if size(unique(X,'rows'),1)~=size(X,1), error('ABACUS:MLDuplicate','Duplicate operating points would leak across splits.'); end
old=rng; cleanup=onCleanup(@() rng(old)); rng(seed,'twister');
bins=zeros(size(X,1),2);
for j=1:2
    values=X(:,j); lo=min(values); hi=max(values);
    if lo==hi, bins(:,j)=1; else, bins(:,j)=discretize(values,linspace(lo,hi+eps(max(1,abs(hi)))*4,5)); end
end
group=bins(:,1)+4*(bins(:,2)-1); train=false(size(group)); val=train; test=train; sparse=[];
for id=unique(group)'
    indices=find(group==id); indices=indices(randperm(numel(indices))); n=numel(indices);
    if n<3
        train(indices)=true; sparse(end+1)=id; %#ok<AGROW>
    else
        nTrain=min(n-2,max(1,round(.7*n))); nVal=min(n-nTrain-1,max(1,round(.15*n)));
        train(indices(1:nTrain))=true; val(indices(nTrain+1:nTrain+nVal))=true;
        test(indices(nTrain+nVal+1:end))=true;
    end
end
split=struct('Train',train,'Validation',val,'Test',test,'Group',group,'Seed',seed,...
    'ActualFractions',[mean(train),mean(val),mean(test)],'SparseGroups',sparse);
if ~any(val) || ~any(test), error('ABACUS:MLSplit','Not enough points to construct stratified validation/test sets.'); end
end
```

matlab/ml/ml_targets.m

```
function names = ml_targets(engineType)
%ML_TARGETS Architecture-specific output schema; no cross-architecture regressor.
% Input: architecture string. Output: cell array of SI result fields. Ref: ABACUS 20.2.
switch char(engineType)
    case 'Turbojet', names={'Thrust_N','TSFC_kgNs','Eta_thermal','Eta_propulsive'};
    case {'Turbofan_HighBPR','Turbofan_LowBPR'}, names={'Thrust_N','TSFC_kgNs','Eta_thermal'};
    case 'Turboprop', names={'EquivalentThrust_N','BSFC_kgkWh','ShaftPower_kW','ESHP','Eta_thermal','Eta_propulsive'};
```

```
    case 'Piston', names={'EquivalentThrust_N','BSFC_kgkWh','ShaftPower_kW','Eta_thermal','Eta_propulsive'};
    otherwise, error('ABACUS:MLArchitecture','No ML output contract for this architecture.');
end
end
```

matlab/ml/ml_train_surrogates.m

```
function summary = ml_train_surrogates(dt,engineID,data)
%ML_TRAIN_SURROGATES Fit four families, select using validation, then unlock test set.
% Inputs: DT, engine ID, optional validated dataset. Output: actual metrics and MAT artifacts.
% Ref: ABACUS 20; rng(42). No fabricated pretrained models or target accuracy claims.
e=db_get_engine(dt.Database,engineID); require_validation(e);
if nargin<3, data=ml_generate_dataset(dt,engineID); end
if ~strcmp(data.Fingerprint,e.Fingerprint), error('ABACUS:StaleDataset','Dataset does not match the current physics/provenance.'); end
if ~license('test','Statistics_Toolbox'), error('ABACUS:MLToolbox','Statistics and Machine Learning Toolbox is required.'); end
p=project_paths(); old=rng; clean=onCleanup(@() rng(old)); rng(42,'twister');
train=data.Split.Train; val=data.Split.Validation; test=data.Split.Test;
families={'Poly','RF','GPR','ANN'}; selected=cell(4,1); valScores=inf(4,1); diagnostics=cell(0,1);
for i=1:4
    for setting=1:2
        model=ml_fit_candidate(data.X(train,:),data.Y(train,:),families{i},setting);
        prediction=ml_apply_candidate(model,data.X(val,:));
        metrics=ml_metrics(data.Y(val,:),prediction,data.TargetNames);
        score=mean(metrics.RelativeRMSE);
        item=metrics; item.Family=repmat(string(families{i}),height(item),1); item.Setting=repmat(setting,height(item),1);
        diagnostics{end+1,1}=item; %#ok<AGROW>
        if score<valScores(i), valScores(i)=score; selected{i}=model; end
    end
end
% Family choice is frozen using validation loss BEFORE test metrics are inspected.
[~,winner]=min(valScores); evaluations=cell(4,1);
for i=1:4
    model=selected{i}; predictions=ml_apply_candidate(model,data.X(test,:));
    metrics=ml_metrics(data.Y(test,:),predictions,data.TargetNames);
    start=tic;
    for query=1:1000, ml_apply_candidate(model,data.X(find(test,1),:)); end
    latency=toc(start)*1000/1000; metrics.Prediction_ms=repmat(latency,height(metrics),1);
    metrics.Family=repmat(string(families{i}),height(metrics),1); metrics.Selected=repmat(i==winner,height(metrics),1);
    bundle=struct('Model',model,'Engine_ID',e.ID,'Engine',e.Name,'Architecture',e.Type,...
        'Fingerprint',e.Fingerprint,'TargetNames',{data.TargetNames},'FeatureNames',{data.FeatureNames},...
        'Grid',data.Grid,'ValidityMask',data.ValidityMask,'TestMetrics',metrics,...
        'ValidationScore',valScores(i),'ActualSplit',data.Split.ActualFractions,...
        'Operational',all(metrics.MeetsAccuracyCriteria)&&latency<10,...
        'Label','Surrogate of physics-generated synthetic data; not measured engine data.',...
        'Timestamp_UTC',utc_timestamp());
    save(fullfile(p.ml_models,sprintf('engine_%d_%s.mat',e.ID,families{i})),'bundle','-v7.3');
    if i==winner
        save(fullfile(p.ml_models,sprintf('engine_%d_best.mat',e.ID)),'bundle','-v7.3');
        ml_diagnostic_report(data,predictions,metrics,e.ID);
    end
    evaluations{i}=metrics;
end
summary=vertcat(evaluations{:}); writetable(summary,fullfile(p.reports,sprintf('engine_%d_track_b.csv',e.ID)));
writetable(vertcat(diagnostics{:}),fullfile(p.reports,sprintf('engine_%d_validation_selection.csv',e.ID)));
```

```
end
```

matlab/performance/evaluate_grid.m

```
function t = evaluate_grid(e,altitudes,machs,throttles,db,persist,level)
%EVALUATE_GRID Evaluate validated points and retain diagnostics for rejected points.
% Inputs: engine; H [m], M, throttle vectors; optional DB/persistence; DT level.
% Output: flat table in SI. Invalid conditions are rejected before cycle evaluation.
if nargin<5, db=[]; end
if nargin<6, persist=false; end
if nargin<7, level='L3'; end
validateattributes(altitudes,{'numeric'},{'vector','real','finite','>=',0,'<=',32000});
validateattributes(machs,{'numeric'},{'vector','real','finite','>=',0,'<=',3});
validateattributes(throttles,{'numeric'},{'vector','real','finite','>=',0,'<=',1});
[H,M,theta]=ndgrid(altitudes,machs,throttles);
results=cell(numel(H),1); rows=cell(numel(H),1);
for i=1:numel(H)
    f=flight_condition(H(i),M(i),theta(i),'geopotential');
    results{i}=simulate_engine(e,f); rows{i}=result_row(results{i});
end
t=vertcat(rows{:});
if persist
    if isempty(db), error('ABACUS:Persistence','A DB connection is required for persistence.')
; end
    db_store_results(db,results,level);
end
end
```

matlab/performance/performance_grid.m

```
function g = performance_grid(e)
%PERFORMANCE_GRID ABACUS Table 20.1 candidate grid, before per-engine validity filter.
% Input: loaded engine. Output: H [m geopotential], Mach, throttle arrays and count.
% HBP/TP use 21x8x6; TJ/LBP use 21x9x6. No invalid point becomes training data.
switch e.Type
    case 'Turbojet'
        h=0:1000:20000; m=[0,.3,.6,.9,1.2,1.5,1.8,2.1,2.5];
    case 'Turbofan_HighBPR'
        h=linspace(0,13000,21); h(17)=10668; h(18)=11000; h=sort(h);
        m=[0,.15,.3,.5,.65,.78,.85,.92];
    case 'Turbofan_LowBPR'
        h=linspace(0,17000,21); m=[0,.3,.6,.9,1.2,1.5,1.8,2,2.5];
    case 'Turboprop'
        h=linspace(0,9000,21); m=0:.1:.7;
    case 'Piston'
        h=linspace(0,e.Params.MaxAltitude_m,21); m=linspace(0,e.Params.MaxMach,8);
    otherwise
        error('ABACUS:GridArchitecture','No grid for %s.',e.Type);
end
g=struct('Altitude_m',h,'Altitude_Convention','geopotential','Mach',m,...
    'Throttle',.5:.1:1,'CandidateCount',numel(h)*numel(m)*6);
end
```

matlab/reporting/format_result_metrics.m

```
function t = format_result_metrics(r,units)
%FORMAT_RESULT_METRICS Convert display values only; saved/backend data remain SI.
% Input: standard result and SI/Imperial. Output: Quantity/Value/Unit table.
% Ref: ABACUS 22.3; exact international pound/foot conversions where relevant.
if nargin<2, units='SI'; end
keys={'Thrust_N','EquivalentThrust_N','ShaftPower_kW','FuelFlow_kgs','TSFC_kgNs','BSFC_kgkWh',
'Eta_thermal','Eta_propulsive','Eta_overall'};
```

```
si={'kN','kN','kW','kg/s','kg/(kN h)','kg/(kW h)','1','1','1'};
factor=[.001,.001,1,1,3.6e6,1,1,1,1];
if strcmp(units,'Imperial')
    si={'lbf','lbf','hp','lb/s','lb/(lbf h)','lb/(hp h)','1','1','1'};
    factor=[1/4.4482216152605,1/4.4482216152605,1000/745.6998715822702,...
        1/.45359237,3600*4.4482216152605/.45359237,...
        .7456998715822702/.45359237,1,1,1];
end
values=zeros(numel(keys),1);
for i=1:numel(keys), values(i)=r.(keys{i})*factor(i); end
keep=isfinite(values); t=table(string(keys(keep))',values(keep),string(si(keep))',...
    'VariableNames',{'Quantity','Value','Unit'});
end
```

matlab/reporting/generate_report.m

```
function filename = generate_report(result,comparison,selection,mlMetrics,folder)
%GENERATE_REPORT Export an auditable HTML session report from actual backend outputs.
% Inputs: result struct, comparison table, selection struct, ML table, optional folder.
% Outputs: self-contained HTML plus JSON. Ref: ABACUS 27; absent results are explicit.
p=project_paths(); if nargin<5, folder=fullfile(p.reports,'session'); end
if ~isfolder(folder), mkdir(folder); end
c=project_config();
html=['<!doctype html><html><head><meta charset="utf-8"><title>ABACUS session report</title>',...
    '<style>body{font:14px Arial;max-width:1200px;margin:36px auto;color:#183047}h1,h2{color:#143e60}',...
    'table{border-collapse:collapse;max-width:100%;font-size:11px}td,th{border:1px solid #ccd6df;padding:6px;word-break:break-word}',...
    'th{background:#edf3f8}.warning{background:#fff1d0;padding:16px}section{margin:28px 0;overflow:auto}</style></head><body>',...
    '<h1>Aircraft Engine Performance Analysis</h1><p>',html_escape(c.Scope),' | ',html_escape(utc_timestamp()),'</p>',...
    '<p class="warning">Academic estimates and steady-state simulations. No flight-planning, hardware or certification capability. ',...
    'Synthetic-data surrogate accuracy is separate from real-engine validation.</p>'];
if isfield(result,'Condition')
    html=[html,'<section><h2>Inputs and selected engine</h2><p>',html_escape(result.Engine),' | ',...
        html_escape(result.Prediction_Source),' | ',html_escape(result.Confidence),' confidence</p>',...
        table_to_html(struct2table(result.Condition)),'<h2>Calculated outputs</h2>',...
        table_to_html(format_result_metrics(result,'SI')),'<p class="warning">',...
        html_escape(strjoin(result.Warnings,' | ')),'</p><h2>Station total properties</h2>',...
        table_to_html(result.Stations),'<h2>Parameter provenance</h2>',table_to_html(result.Provenance),'</section>'];
end
html=[html,'<section><h2>Comparison / current data</h2>',table_to_html(comparison),'</section>'];
if isfield(selection,'Message')
    html=[html,'<section><h2>Mission selection</h2><p>',html_escape(selection.Message),'</p>',...
        table_to_html(selection.Ranking),'<h3>Eliminated candidates and reasons</h3>',table_to_html(selection.Eliminated),'</section>'];
end
html=[html,'<section><h2>ML held-out physics metrics (Track B)</h2>',table_to_html(mlMetrics),...
    '</section><h2>Model assumptions</h2><p>Constant component efficiencies, constant specific heats, simplified corrected airflow and throttle schedules, ',...
    'separate exhaust, converging nozzles and simplified propeller treatment. Read docs/ASSUMPTIONS.md and CHANGELOG.md for all resolutions.</p></body></html>'];
filename=fullfile(folder,'ABACUS_session_report.html'); write_text(filename,html);
```

```
session=struct('Result',result,'Comparison',comparison,'MissionSelection',selection,'MLMetrics',mlMetrics);
write_text(fullfile(folder,'ABACUS_session_data.json'),jsonencode(session,'PrettyPrint',true));
end
```

matlab/reporting/html_escape.m

```
function text = html_escape(value)
%HTML_ESCAPE Encode arbitrary source/user text before embedding it into HTML.
% Input: text-like value. Output: safe HTML text. Units: none; no markup accepted.
text=char(string(value)); text=strrep(text,'&','&amp;'); text=strrep(text,'<','&lt;');
text=strrep(text,'>','&gt;'); text=strrep(text,'"','&quot;'); text=strrep(text,'''','&#39;');
end
```

matlab/reporting/plot_performance.m

```
function ax = plot_performance(t,xName,yName,ax)
%PLOT_PERFORMANCE Plot valid points grouped by engine, preserving source labels.
% Inputs: result table, numeric field names, optional axes. Output: axes handle.
% Units come from SI column names. Errors on missing fields or no valid points.
if nargin<4 || isempty(ax), figure('Color','w'); ax=axes; end
if ~ismember(xName,t.Properties.VariableNames) || ~ismember(yName,t.Properties.VariableNames)
    error('ABACUS:PlotField','Requested field is not in the result table.');
end
valid=logical(t.Valid)&isfinite(t.(xName))&isfinite(t.(yName));
if ~any(valid), error('ABACUS:NoPlotData','No valid results are available for this plot.'); end
cla(ax); hold(ax,'on'); engines=unique(string(t.Engine(valid)),'stable');
for i=1:numel(engines)
    part=t(valid & string(t.Engine)==engines(i),:); part=sortrows(part,xName);
    plot(ax,part.(xName),part.(yName),'-o','DisplayName',char(engines(i)),'MarkerSize',3);
end
xlabel(ax,strrep(xName,'_',' ')); ylabel(ax,strrep(yName,'_',' '));
title(ax,'Academic steady-state model'); grid(ax,'on'); legend(ax,'Location','best'); hold(ax,'off');
end
```

matlab/reporting/plot_performance_map.m

```
function ax = plot_performance_map(t,metric,throttle,ax)
%PLOT_PERFORMANCE_MAP Display one throttle slice without filling rejected grid points.
% Inputs: performance table, metric field, throttle, optional axes. Output: axes.
% Coordinates: geopotential m and Mach; colour: selected SI metric. NaN holes retained.
if nargin<4 || isempty(ax), figure('Color','w'); ax=axes; end
part=t(abs(t.Throttle-throttle)<1e-8,:); h=unique(part.GeopotentialAltitude_m); m=unique(part.Mach);
z=nan(numel(h),numel(m));
for i=1:height(part)
    if part.Valid(i), z(h==part.GeopotentialAltitude_m(i),m==part.Mach(i))=part.(metric)(i); end
end
surf(ax,m,h,z,'EdgeColor','none'); view(ax,2); colorbar(ax);
xlabel(ax,'Mach'); ylabel(ax,'Geopotential altitude (m)'); title(ax,strrep(metric,'_',' '));
end
```

matlab/reporting/result_row.m

```
function t = result_row(r)
%RESULT_ROW Flatten one result for grids/CSV without losing validity or provenance.
% Input: standard result. Output: one-row table, SI units indicated in column names.
% NaNs are retained for rejected/inapplicable outputs; no imputation is performed.
f=r.Condition;
s=struct('Engine_ID',r.Engine_ID,'Engine',string(r.Engine),'Architecture',string(r.Architecture),...
    'Altitude_m',f.Altitude_m,'GeopotentialAltitude_m',f.GeopotentialAltitude_m,...
    'Mach',f.Mach,'Throttle',f.Throttle,'Afterburner',double(f.Afterburner));
fields={'Thrust_N','EquivalentThrust_N','ShaftPower_kW','FuelFlow_kgs','TSFC_kgNs',...
    'BSFC_kgkWh','Eta_thermal','Eta_propulsive','Eta_overall','T_t4_K','P_t3_Pa','ESHP'};
for i=1:numel(fields), s.(fields{i})=r.(fields{i}); end
s.Valid=double(r.Valid); s.Status=string(r.Status); s.Error_Code=string(r.Error_Code);
s.Data_Type=string(r.Data_Type); s.Prediction_Source=string(r.Prediction_Source);
s.Confidence=string(r.Confidence); s.Warnings=strjoin(string(r.Warnings),' | ');
s.Core_Nozzle_State=string(r.Core_Nozzle_State); s.Bypass_Nozzle_State=string(r.Bypass_Nozzle_State);
s.Model_Version=string(r.Model_Version); s.Model_Fingerprint=string(r.Model_Fingerprint);
s.ML_Model_Type=string(r.ML_Model_Type); s.Timestamp_UTC=string(r.Timestamp_UTC);
t=struct2table(s);
end
```

matlab/reporting/table_to_html.m

```
function html = table_to_html(t)
%TABLE_TO_HTML Render a MATLAB table with escaped values and explicit unavailable cells.
% Input: table. Output: HTML string. Units remain in headers. No JavaScript is emitted.
if isempty(t), html='<p>No data generated for this section.</p>'; return; end
html='<table><thead><tr>';
for name=string(t.Properties.VariableNames), html=[html,'<th>',html_escape(name),'</th>']; end %#ok<AGROW>
html=[html,'</tr></thead><tbody>'];
for i=1:height(t)
    html=[html,'<tr>']; %#ok<AGROW>
    for j=1:width(t)
        value=t{i,j}; if iscell(value), value=value{1}; end
        if isnumeric(value), value=mat2str(value,7); end
        html=[html,'<td>',html_escape(value),'</td>']; %#ok<AGROW>
    end
    html=[html,'</tr>']; %#ok<AGROW>
end
html=[html,'</tbody></table>'];
end
```

matlab/setup/DB_Interface.m

```
classdef DB_Interface < handle
%DB_INTERFACE Typed SQLite access with no user values interpolated into SQL.
% Input: SQLite path. Output: handle with read/append/atomic methods.
% Ref: MathWorks sqlite, sqlread, sqlwrite (R2022a+); CHANGELOG C01.
% Schema identifiers are allowlisted. NULL numeric values use MATLAB NaN.
    properties (SetAccess=private)
        Connection
        File
    end
    methods
        function obj = DB_Interface(filename)
            % Open existing SQLite DB; establish FK and busy timeout on EVERY connection.
            if nargin < 1, p = project_paths(); filename = p.database_file; end
            if ~isfile(filename), error('ABACUS:MissingDatabase','Run db_initialize for %s.',f
```

```
    ilename); end
15              obj.File = char(filename);
16              obj.Connection = sqlite(obj.File,'connect');
17              execute(obj.Connection,'PRAGMA foreign_keys = ON');
18              execute(obj.Connection,'PRAGMA busy_timeout = 5000');
19              fetch(obj.Connection,'PRAGMA journal_mode = WAL');
20              check = fetch(obj.Connection,'PRAGMA foreign_keys');
21              if ~istable(check), check = cell2table(check); end
22              if check{1,1} ~= 1, error('ABACUS:ForeignKeys','SQLite foreign keys were not enabl
    ed.'); end
23          end
24          function t = read(obj,name)
25              % Read allowlisted table/view; callers filter typed columns in MATLAB.
26              obj.checkName(name);
27              t = sqlread(obj.Connection,char(name));
28          end
29          function append(obj,name,t)
30              % Typed value transfer through sqlwrite; no SQL literal construction.
31              obj.checkName(name);
32              if ~istable(t), error('ABACUS:DatabaseInput','Database rows must be a MATLAB table
    .'); end
33              if isempty(t), return; end
34              sqlwrite(obj.Connection,char(name),t);
35          end
36          function atomic(obj,work)
37              % Execute caller's typed writes as one transaction; rollback on any failure.
38              if strcmp(obj.Connection.AutoCommit,'off')
39                  error('ABACUS:NestedTransaction','Nested database transactions are unsupported
    .');
40              end
41              obj.Connection.AutoCommit='off';
42              try
43                  work();
44                  commit(obj.Connection);
45                  obj.Connection.AutoCommit='on';
46              catch exception
47                  rollback(obj.Connection);
48                  obj.Connection.AutoCommit='on';
49                  rethrow(exception);
50              end
51          end
52          function t = pragma(obj,kind)
53              % Read-only integrity/metadata checks; names are closed choices.
54              options = {'foreign_key_check','integrity_check','foreign_keys','user_version'};
55              validatestring(kind,options);
56              t = fetch(obj.Connection,['PRAGMA ',char(kind)]);
57          end
58          function delete(obj)
59              % Deterministic connection release; safe for partially constructed handles.
60              if ~isempty(obj.Connection)
61                  close(obj.Connection);
62                  obj.Connection = [];
63              end
64          end
65      end
66      methods (Access=private)
67          function checkName(~,name)
68              % Allow only known schema identifiers; user text can never become SQL syntax.
69              allowed = {'Manufacturers','Engine_Types','Engines','Engine_Thermo_Params',...
70                  'Parameter_Provenance','Aircraft','Engine_Aircraft_Map','Performance_Data',...
71                  'Mission_Profiles','References','Simulation_Log','Project_Metadata',...
72                  'v_Engine_Catalogue','v_Reference_Data','v_Provenance'};
73              validatestring(char(name),allowed);
74          end
```

```
    end
end
```

matlab/setup/db_add_reference.m

```
function db_add_reference(db,record,verificationNote)
%DB_ADD_REFERENCE Add explicitly reviewed public reference data as an immutable row.
% Inputs: DB handle, scalar struct containing exact SI condition/value/source, review note.
% Output: appended Reference row. This API records the caller's review; it does not verify URLs.
% Estimated targets require their own kind and Low confidence; never call them measurements.
needed={'Engine_ID','GeopotentialAltitude_m','Mach','Throttle','Source_Ref','Reference_Kind'};
for i=1:numel(needed)
    if ~isfield(record,needed{i}), error('ABACUS:ReferenceInput','Missing %s.',needed{i}); end
end
if strlength(string(verificationNote))<20 || ~startsWith(string(record.Source_Ref),["https://","http://"])
    error('ABACUS:ReferenceReview','Provide the exact public source URL and a review note identifying document, page, units and rating.');
end
e=db_get_engine(db,record.Engine_ID);
kind=validatestring(char(record.Reference_Kind),{'Measured','PublishedRating','EstimatedTarget'});
if e.IsKaveri && ~strcmp(kind,'EstimatedTarget')
    error('ABACUS:EstimatedReference','Kaveri references must remain explicitly estimated targets under ABACUS.');
end
f=flight_condition(record.GeopotentialAltitude_m,record.Mach,record.Throttle,'geopotential');
record.Altitude_m=f.Altitude_m; record.Data_Type="Reference"; record.Prediction_Source="Reference";
record.Reference_Verified=1; record.Reference_Kind=string(kind);
if ~isfield(record,'Confidence'), record.Confidence="Low"; end
record.Confidence=string(validatestring(char(record.Confidence),{'High','Medium','Low'}));
if strcmp(kind,'EstimatedTarget'), record.Confidence="Low"; end
record.Warnings=string(verificationNote);
if strcmp(kind,'EstimatedTarget'), record.Warnings=record.Warnings+" | Estimated academic target, not measured engine performance."; end
record.Timestamp_UTC=string(utc_timestamp()); record.Source_Ref=string(record.Source_Ref);
if ~isfield(record,'FuelFlow_kgs'), record.FuelFlow_kgs=NaN; end
if ~isfield(record,'Thrust_N') && ~isfield(record,'ShaftPower_kW')
    error('ABACUS:ReferenceMetric','Supply measured/rated Thrust_N or ShaftPower_kW with the exact condition.');
end
for metric=["Thrust_N","ShaftPower_kW","TSFC_kgNs","BSFC_kgkWh"]
    if isfield(record,metric) && ~isnan(record.(metric))
        validateattributes(record.(metric),{'numeric'},{'scalar','real','finite','positive'});
    end
end
db.append('Performance_Data',struct2table(record));
end
```

matlab/setup/db_execute_script.m

```
function db_execute_script(connection, filename)
%DB_EXECUTE_SCRIPT Execute a trusted repository SQL script, preserving trigger bodies.
% Inputs: SQLite connection, SQL file path. Output: DB side effects. Units: none.
% Only authored schema/seed scripts may be passed. Errors roll back all statements.
commands = regexp(fileread(filename),'(?m)^-- @statement\s*\r?$','split');
connection.AutoCommit='off';
try
    for i = 1:numel(commands)
        command = strtrim(commands{i});
```

```
        % Foreign keys must be enabled before transaction entry, not inside it.
        if isempty(command) || strcmp(command,'PRAGMA foreign_keys = ON;'), continue; end
        execute(connection,command);
    end
    commit(connection);
    connection.AutoCommit='on';
catch exception
    rollback(connection);
    connection.AutoCommit='on';
    rethrow(exception);
end
end
```

matlab/setup/db_get_engine.m

```
function e = db_get_engine(db, identifier)
%DB_GET_ENGINE Load one engine and verify value-level provenance before modelling.
% Inputs: DB_Interface handle; numeric ID or exact designation. Output: engine struct.
% Units: all parameter values SI; rejects missing values/provenance or invalid IDs.
catalogue = db.read('Engines');
if isnumeric(identifier)
    validateattributes(identifier,{'numeric'},{'scalar','finite','integer','positive'});
    mask = catalogue.Engine_ID == identifier;
else
    validateattributes(string(identifier),{'string'},{'scalar'});
    mask = string(catalogue.Designation) == string(identifier);
end
if nnz(mask) ~= 1, error('ABACUS:InvalidEngine','Engine identifier is not in this database.'); end
row = catalogue(mask,:); params = db.read('Engine_Thermo_Params');
params = params(params.Engine_ID == row.Engine_ID,:);
if height(params) ~= 1, error('ABACUS:MissingParameters','Engine has no unique parameter record.'); end
types = db.read('Engine_Types'); kind = types(types.Type_ID == row.Type_ID,:);
provenance = db.read('Parameter_Provenance');
provenance = provenance(provenance.Engine_ID == row.Engine_ID,:);
p = table2struct(params); fields = fieldnames(p);
for i = 1:numel(fields)
    key = fields{i}; value = p.(key);
    if ismember(key,{'Param_ID','Engine_ID','Overall_Confidence'}) || ~isnumeric(value) || isnan(value), continue; end
    match = string(provenance.Parameter_Name) == key;
    if nnz(match) ~= 1 || abs(provenance.Parameter_Value(match)-value) > max(1e-12,abs(value)*1e-12)
        error('ABACUS:MissingProvenance','Missing or mismatched provenance for %s.',key);
    end
end
e = struct('ID',row.Engine_ID,'Name',char(string(row.Designation)),...
    'Type',char(string(kind.Type_Name)),'AfterburnerAvailable',logical(row.Afterburner),...
    'Params',p,'Provenance',provenance,'Confidence',char(string(p.Overall_Confidence)));
e.IsKaveri = contains(e.Name,'Kaveri','IgnoreCase',true);
e.Estimated = any(ismember(string(provenance.Source_Type),["Estimated","Assumed"])) || e.IsKaveri;
e.Fingerprint = model_fingerprint(e);
end
```

matlab/setup/db_initialize.m

```
function db_initialize(filename)
%DB_INITIALIZE Reproduce schema in a new DB; preserve and verify existing files.
% Input: optional SQLite file path. Output: DB on disk. Ref: ABACUS 11 + schema.sql.
% Existing incompatible databases raise an error; no automatic destructive migration.
p = project_paths();
if nargin < 1, filename = p.database_file; end
if isfile(filename)
    db = DB_Interface(filename); clean = onCleanup(@() delete(db));
    t = db.pragma('user_version');
    if istable(t), version = t{1,1}; else, version = t{1}; end
    if version ~= 2, error('ABACUS:Schema','Expected schema version 2; back up and migrate thi
s database.'); end
    return
end
folder = fileparts(filename);
if ~isempty(folder) && ~isfolder(folder), mkdir(folder); end
connection = sqlite(char(filename),'create');
clean = onCleanup(@() close(connection));
execute(connection,'PRAGMA foreign_keys = ON');
db_execute_script(connection,fullfile(p.database,'schema.sql'));
end
```

matlab/setup/db_log_failure.m

```
function db_log_failure(db,requestedID,f,level,exception)
%DB_LOG_FAILURE Record a rejected coordinator request without violating foreign keys.
% Inputs: DB, original engine selector/flight object, level, exception. Output: log row.
% Unknown engine IDs use NULL; the original selector remains in Inputs_JSON.
c=project_config(); input=struct('RequestedEngine',requestedID,'Condition',f);
output=struct('Status','FAIL','Error_Code',exception.identifier,'Message',exception.message);
row=struct('Engine_ID',NaN,'DT_Level',string(level),'Model_Type',"Physics",...
    'Inputs_JSON',string(jsonencode(input)),'Outputs_JSON',string(jsonencode(output)),...
    'Model_Version',string(c.Version),'Event_Type',"RejectedRequest",'Timestamp_UTC',string(ut
c_timestamp()));
db.append('Simulation_Log',struct2table(row));
end
```

matlab/setup/db_seed.m

```
function db_seed(filename)
%DB_SEED Insert stable catalogue IDs/provenance without overwriting existing rows.
% Input: optional DB path. Output: seeded DB. Units: schema-defined SI.
% Ref: database/seed_data.sql. Repeated calls are idempotent; SQL errors propagate.
p = project_paths();
if nargin < 1, filename = p.database_file; end
db = DB_Interface(filename); clean = onCleanup(@() delete(db));
db_execute_script(db.Connection,fullfile(p.database,'seed_data.sql'));
end
```

matlab/setup/db_seed_piston_extension.m

```
function db_seed_piston_extension(databaseFile)
%DB_SEED_PISTON_EXTENSION Add two optional piston catalogue entries with assumed data.
% Input: optional DB path. Output: 13-engine catalogue when explicitly requested.
% Outside ABACUS core scope; every added parameter is Assumed/Low and unverified.
p=project_paths(); if nargin<1, databaseFile=p.database_file; end
db=DB_Interface(databaseFile); cleanup=onCleanup(@() delete(db));
db_execute_script(db.Connection,fullfile(p.database,'seed_piston_extension.sql'));
end
```

matlab/setup/db_store_results.m

```
function db_store_results(db, results, level)
%DB_STORE_RESULTS Atomically persist valid performance and all execution diagnostics.
% Inputs: DB handle, result cell array, DT level. Output: appended database records.
% Ref: ABACUS 19. Invalid results go to Simulation_Log only; references are immutable.
if ~iscell(results), results={results}; end
performance=cell(0,1); logs=cell(numel(results),1);
for i=1:numel(results)
    r=results{i}; t=result_row(r);
    if r.Valid
        t=removevars(t,{'Engine','Architecture','Eta_overall','Status','Error_Code'});
        performance{end+1,1}=t; %#ok<AGROW>
    end
    f=r.Condition;
    log=struct('Engine_ID',r.Engine_ID,'DT_Level',string(level),...
        'Model_Type',string(r.Prediction_Source),'ML_Model_Name',string(r.ML_Model_Type),...
        'Altitude_m',f.Altitude_m,'Mach',f.Mach,'Throttle',f.Throttle,...
        'Inputs_JSON',string(jsonencode(f)),'Outputs_JSON',string(jsonencode(r)),...
        'Exec_Time_ms',r.Elapsed_ms,'Model_Version',string(r.Model_Version),...
        'Model_Fingerprint',string(r.Model_Fingerprint),'Timestamp_UTC',string(r.Timestamp_UTC
));
    logs{i}=struct2table(log);
end
if isempty(performance), p=table(); else, p=vertcat(performance{:}); end
l=vertcat(logs{:});
db.atomic(@() append_both(db,p,l));
end
function append_both(db,p,l)
%APPEND_BOTH Commit helper; typed tables and SI fields; transaction owner is caller.
if ~isempty(p), db.append('Performance_Data',p); end
db.append('Simulation_Log',l);
end
```

matlab/setup/project_config.m

```
function c = project_config()
%PROJECT_CONFIG Central physical constants and reproducibility settings.
% Inputs: none. Output: SI constants c. Ref: ABACUS 16-17 and CHANGELOG C04.
% cp is derived from R and gamma for internal thermodynamic consistency.
c.Version = 'ABACUS-1.1-implementation-1.0';
c.SchemaVersion = 2;
c.RandomSeed = 42;
c.EarthRadius_m = 6356766;
c.g0 = 9.80665;
c.T0_K = 288.15;
c.P0_Pa = 101325;
c.R_air = 287.058;
c.gamma_air = 1.4;
c.R_gas = 287.4;
c.gamma_gas = 1.33;
c.cp_air = c.gamma_air*c.R_air/(c.gamma_air-1);
c.cp_gas = c.gamma_gas*c.R_gas/(c.gamma_gas-1);
c.FuelLHV_Jkg = 43.2e6;
c.KaveriWarning = 'ESTIMATED / ACADEMIC MODEL - NOT MANUFACTURER-CONFIRMED PERFORMANCE. These
results do NOT represent actual Kaveri engine performance.';
c.Scope = 'Academic steady-state software Digital Twin';
c.MinimumML_R2 = 0.95;
c.MLFeatures = {'Mach','GeopotentialAltitude_m','Throttle'};
end
```

matlab/utilities/flight_condition.m

```
function f = flight_condition(altitude_m, mach, throttle, convention, afterburner)
%FLIGHT_CONDITION Standardised flight input, accepting geometric or geopotential m.
% Inputs: altitude [m], Mach [-], throttle [0,1], convention, optional AB flag.
% Output: f with both altitude conventions. Ref: ABACUS 16. Rejects invalid values.
if nargin < 4, convention = 'geometric'; end
if nargin < 5, afterburner = false; end
validateattributes(altitude_m,{'numeric'},{'real','finite','scalar'});
validateattributes(mach,{'numeric'},{'real','finite','scalar'});
validateattributes(throttle,{'numeric'},{'real','finite','scalar'});
if altitude_m<0, error('ABACUS:ERR_ATM_01','Altitude cannot be negative.'); end
if mach<0 || mach>3, error('ABACUS:ERR_MACH_01','Mach must be between zero and three.'); end
if throttle<0 || throttle>1, error('ABACUS:ERR_IN_01','Throttle must be between zero and one.'
); end
validateattributes(afterburner,{'numeric','logical'},{'scalar','binary'});
c = project_config();
switch lower(string(convention))
    case "geometric"
        if altitude_m > 32000, error('ABACUS:ERR_ATM_01','Geometric altitude must be 0 to 3200
0 m.'); end
        h = altitude_m; H = c.EarthRadius_m*h/(c.EarthRadius_m+h);
    case "geopotential"
        if altitude_m > 32000, error('ABACUS:ERR_ATM_01','Geopotential altitude must be 0 to 3
2000 m.'); end
        H = altitude_m; h = c.EarthRadius_m*H/(c.EarthRadius_m-H);
    otherwise
        error('ABACUS:AltitudeConvention','Choose geometric or geopotential altitude.');
end
f = struct('Altitude_m',h,'GeopotentialAltitude_m',H,'Mach',mach,...
    'Throttle',throttle,'Afterburner',logical(afterburner),'Input_Convention',char(convention)
);
end
```

matlab/utilities/model_fingerprint.m

```
function fingerprint = model_fingerprint(e)
%MODEL_FINGERPRINT SHA-256 over model source, parameters and provenance.
% Input: engine struct. Output: hex text. Units: none; requires MATLAB JVM.
% Invalidates old validation/ML artifacts after a source or parameter change.
p = project_paths();
folders = {'engines','atmosphere','validation','utilities','setup'};
files = [];
for i = 1:numel(folders)
    files = [files; dir(fullfile(p.matlab,folders{i},'','*.m'))]; %#ok<AGROW>
end
[~,index] = sort(string({files.name})); files = files(index);
md = java.security.MessageDigest.getInstance('SHA-256');
for i = 1:numel(files)
    content = fileread(fullfile(files(i).folder,files(i).name));
    md.update(unicode2native([files(i).name,content],'UTF-8'));
end
md.update(unicode2native(jsonencode(e.Params),'UTF-8'));
md.update(unicode2native(jsonencode(e.Provenance),'UTF-8'));
fingerprint = lower(reshape(dec2hex(typecast(md.digest(),'uint8'),2).',1,[]));
end
```

matlab/utilities/utc_timestamp.m

```
function t = utc_timestamp()
%UTC_TIMESTAMP Return an ISO 8601 UTC timestamp; no input and no physical units.
% Assumption: system clock is correct. Output is a char vector suitable for SQLite.
t = char(datetime('now','TimeZone','UTC','Format','yyyy-MM-dd''T''HH:mm:ss.SSS''Z'''));
end
```

matlab/utilities/write_text.m

```
function write_text(filename, text)
%WRITE_TEXT Save UTF-8 text, creating its parent folder when necessary.
% Inputs: filename and text. Output: file on disk. Units: text; throws on I/O errors.
folder = fileparts(filename);
if ~isempty(folder) && ~isfolder(folder), mkdir(folder); end
[fid,message] = fopen(filename,'w','n','UTF-8');
if fid < 0, error('ABACUS:FileIO','Cannot write %s: %s',filename,message); end
cleanup = onCleanup(@() fclose(fid));
fprintf(fid,'%s',char(text));
end
```

matlab/validation/parameter_sensitivity.m

```
function t = parameter_sensitivity(dt,id,f,parameter,relativeChange)
%PARAMETER_SENSITIVITY Report local parameter sensitivity without fitting or DB mutation.
% Inputs: DT, engine ID, flight f, numeric parameter name, fractional perturbation.
% Output: three SI result rows. Overrides are explicitly marked Assumed in provenance.
if nargin<5, relativeChange=.02; end
validateattributes(relativeChange,{'numeric'},{'scalar','real','finite','>',0,'<',.5});
e=db_get_engine(dt.Database,id);
if ~isfield(e.Params,parameter) || ~isnumeric(e.Params.(parameter)) || ~isfinite(e.Params.(par
ameter))
    error('ABACUS:SensitivityParameter','Choose a populated numerical parameter.');
end
rows=cell(3,1); factors=[1-relativeChange,1,1+relativeChange];
for i=1:3
    varied=e; varied.Params.(parameter)=e.Params.(parameter)*factors(i);
    mask=string(varied.Provenance.Parameter_Name)==string(parameter);
    varied.Provenance.Parameter_Value(mask)=varied.Params.(parameter);
    varied.Provenance.Source_Type(mask)="Assumed"; varied.Provenance.Confidence_Level(mask)="L
ow";
    varied.Provenance.Source_Reference(mask)="Local sensitivity perturbation; no parameter cal
ibration";
    varied.Estimated=true; varied.Confidence='Low'; varied.Fingerprint=model_fingerprint(varie
d);
    r=simulate_engine(varied,f); rows{i}=result_row(r);
    rows{i}.VariedParameter=string(parameter); rows{i}.ParameterValue=varied.Params.(parameter
);
end
t=vertcat(rows{:});
end
```

matlab/validation/regression_baseline.m

```
function report = regression_baseline(action,databaseFile)
%REGRESSION_BASELINE Capture/compare real executed model outputs with a 0.01% threshold.
% Input: capture or compare; optional DB path. Output: comparison table and MAT evidence.
% Capturing requires passing internal physics tests. A baseline is not a measured reference.
p=project_paths(); if nargin<2, databaseFile=p.database_file; end
validatestring(action,{'capture','compare'}); file=fullfile(p.reports,'regression_baseline.mat
');
```

```
dt=DT_Coordinator(databaseFile); cleanup=onCleanup(@() delete(dt)); cat=dt.Database.read('Engines');
rows=cell(height(cat),1);
for i=1:height(cat), rows{i}=result_row(dt.run(cat.Engine_ID(i),flight_condition(0,0,1),'L1',false)); end
current=vertcat(rows{:});
if strcmp(action,'capture')
    if isfile(file), error('ABACUS:ExistingBaseline','Archive the existing baseline before capturing a replacement.'); end
    checked=validate_physics(databaseFile);
    if ~all([checked.InternalTests.Passed]), error('ABACUS:BaselineGate','Internal physics tests must pass before baseline capture.'); end
    baseline=current; capturedUTC=utc_timestamp(); %#ok<NASGU>
    save(file,'baseline','capturedUTC'); report=current;
else
    if ~isfile(file), error('ABACUS:MissingBaseline','Capture a baseline after running internal physics tests.'); end
    data=load(file,'baseline'); baseline=data.baseline;
    if ~isequal(baseline.Engine_ID,current.Engine_ID), error('ABACUS:BaselineCatalogue','Catalogue IDs changed; review baseline applicability.'); end
    metrics={'EquivalentThrust_N','FuelFlow_kgs','ShaftPower_kW','T_t4_K','P_t3_Pa'};
    report=table(current.Engine_ID,string(current.Engine),'VariableNames',{'Engine_ID','Engine'});
    maximum=zeros(height(current),1);
    for j=1:numel(metrics)
        old=baseline.(metrics{j}); new=current.(metrics{j});
        difference=abs(new-old)./max(abs(old),1e-12); difference(isnan(old)&isnan(new))=0;
        difference(~isfinite(difference))=Inf; maximum=max(maximum,difference);
    end
    report.MaximumRelativeChange=maximum; report.Pass=maximum<=1e-4 & baseline.Valid==current.Valid;
    writetable(report,fullfile(p.reports,'regression_comparison.csv'));
end
end
```

matlab/validation/require_validation.m

```
function certificate = require_validation(e)
%REQUIRE_VALIDATION Reject missing/stale/unaccepted physics sign-off before ML data.
% Input: loaded engine. Output: certificate struct. Units: none; no bypass flag.
% Ref: ABACUS CR-16; a generated source file never counts as executed validation.
p=project_paths(); file=fullfile(p.reports,sprintf('validation_engine_%d.json',e.ID));
if ~isfile(file), error('ABACUS:ValidationRequired','Run validate_physics and review Track A before training engine %d.',e.ID); end
certificate=jsondecode(fileread(file));
if ~strcmp(certificate.Fingerprint,e.Fingerprint)
    error('ABACUS:StaleValidation','Model/parameter provenance changed; rerun physics validation.');
end
if ~certificate.InternalTestsPassed || ~certificate.AcceptedForML
    error('ABACUS:ValidationRequired','Track A is not accepted for this engine. Review verified references and the actual model error; ML training is blocked.');
end
end
```

matlab/validation/validate_inputs.m

```
function warnings = validate_inputs(e,f)
%VALIDATE_INPUTS Enforce engine envelopes and component parameter bounds.
% Inputs: engine and standard flight structs. Output: warning string column.
% Units: SI. Ref: ABACUS 18, with exact engine limits from Table 18.5.
% Throws ABACUS error identifiers on hard failures; low-throttle use is warned.
```

```
 6  p = e.Params; warnings = strings(0,1);
 7  required = {'MaxAltitude_m','MaxMach','DryWeight_kg'};
 8  if strcmp(e.Type,'Piston')
 9      required = [required,{'Displacement_m3','Compression_Ratio','Rated_RPM','Strokes',...
10          'Volumetric_Efficiency','Brake_Thermal_Efficiency','Fuel_LHV_Jkg','AFR'}];
11  else
12      required = [required,{'OPR','T_t4_K','MassFlow_kgs','Eta_compressor','Eta_combustor',...
13          'Eta_turbine','Eta_mechanical','Eta_nozzle_core','Inlet_PR_recovery','Burner_PR_loss'}
    ];
14  end
15  if contains(e.Type,'Turbofan'), required = [required,{'BPR','Fan_PR','Eta_fan','Eta_nozzle_byp
    ass'}]; end
16  if strcmp(e.Type,'Turboprop')
17      required = [required,{'Eta_gearbox','Eta_propeller','Eta_power_turbine','Prop_Diameter_m',
    ...
18          'Prop_RPM','PowerTurbine_BackPressure_Ratio'}];
19  end
20  for i = 1:numel(required)
21      name = required{i};
22      if ~isfield(p,name) || ~isnumeric(p.(name)) || ~isscalar(p.(name)) || ~isfinite(p.(name))
    || ~isreal(p.(name))
23          error('ABACUS:MissingParameter','Missing or nonfinite parameter %s for %s.',name,e.Nam
    e);
24      end
25  end
26  if f.GeopotentialAltitude_m<0 || f.GeopotentialAltitude_m>min(32000,p.MaxAltitude_m)+1e-8
27      error('ABACUS:ERR_ATM_01','H=%.1f m exceeds the engine envelope 0 to %.1f m.',f.Geopotenti
    alAltitude_m,p.MaxAltitude_m);
28  end
29  if ~isfinite(f.Mach) || f.Mach<0 || f.Mach>p.MaxMach
30      error('ABACUS:ERR_MACH_01','Mach %.3g exceeds the %s range 0 to %.3g.',f.Mach,e.Name,p.Max
    Mach);
31  end
32  if ~isfinite(f.Throttle) || f.Throttle<0 || f.Throttle>1
33      error('ABACUS:ERR_IN_01','Throttle must be between zero and one.');
34  end
35  if f.Afterburner, error('ABACUS:AfterburnerUnsupported','Afterburner-on calculations are outsi
    de ABACUS Phase 1.'); end
36  if strcmp(e.Type,'Piston')
37      if p.Displacement_m3<=0 || p.Rated_RPM<=0 || p.AFR<=0 || p.Compression_Ratio<=1 || ~ismemb
    er(p.Strokes,[2,4])
38          error('ABACUS:PistonParameter','Invalid displacement, RPM, AFR, compression ratio or s
    troke count.');
39      end
40  else
41      if p.OPR<=1 || p.OPR>50, error('ABACUS:ERR_IN_03','Overall pressure ratio must satisfy 1 <
     OPR <= 50.'); end
42      if p.T_t4_K<400 || p.T_t4_K>2500, error('ABACUS:ERR_IN_02','Design TIT must be 400 to 2500
     K.'); end
43      if p.T_t4_K<800 || p.T_t4_K>1900, warnings(end+1) = "Design TIT is outside the usual acade
    mic range."; end
44      if p.MassFlow_kgs<=0, error('ABACUS:ERR_FLOW_01','Corrected design flow must be positive.'
    ); end
45      if p.Burner_PR_loss<0 || p.Burner_PR_loss>.2 || p.Inlet_PR_recovery<=0 || p.Inlet_PR_recov
    ery>1
46          error('ABACUS:PressureLoss','Invalid burner loss or inlet recovery.');
47      end
48  end
49  fields = fieldnames(p);
50  for i = 1:numel(fields)
51      name = fields{i}; value = p.(name);
52      if (startsWith(name,'Eta_') || endsWith(name,'Efficiency')) && isnumeric(value) && ~isnan(
    value)
53          if ~isfinite(value) || value<=0 || value>1, error('ABACUS:Efficiency','%s must be in (
```

```
    0,1].',name); end
54      end
55  end
56  bounds = {'Eta_compressor',.70,.92;'Eta_turbine',.85,.95;'Eta_fan',.80,.92;...
57      'Eta_combustor',.95,1;'Eta_mechanical',.97,1;'Eta_gearbox',.96,1;'Eta_propeller',eps,.92};
58  for i = 1:size(bounds,1)
59      key = bounds{i,1};
60      if isfield(p,key) && ~isnan(p.(key)) && (p.(key)<bounds{i,2} || p.(key)>bounds{i,3})
61          error('ABACUS:EfficiencyRange','%s is outside the ABACUS component range.',key);
62      end
63  end
64  if contains(e.Type,'Turbofan') && (p.Fan_PR<=1 || p.Fan_PR>=p.OPR || p.BPR<0 || p.BPR>15)
65      error('ABACUS:FanParameters','Require 1 < Fan_PR < OPR and 0 <= BPR <= 15.');
66  end
67  if strcmp(e.Type,'Turboprop') && (p.Prop_Diameter_m<=0 || p.Prop_RPM<=0 || p.PowerTurbine_Back
    Pressure_Ratio<=1)
68      error('ABACUS:PropellerParameters','Positive propeller geometry/speed and residual NPR > 1
     are required.');
69  end
70  if f.Throttle>0 && f.Throttle<.5, warnings(end+1) = "Throttle below 0.5: uncalibrated academic
     off-design extrapolation."; end
71  if e.Estimated, warnings(end+1) = "Estimated/assumed cycle parameters; real engine accuracy ha
    s not been established."; end
72  if e.IsKaveri, config = project_config(); warnings(end+1) = string(config.KaveriWarning); end
73  end
```

matlab/validation/validate_outputs.m

```
 1  function r = validate_outputs(r)
 2  %VALIDATE_OUTPUTS Check active outputs, thermodynamics, mass and shaft conservation.
 3  % Input/output: standard SI result struct. Ref: ABACUS 18. Inapplicable NaNs allowed.
 4  % Hard conservation or nonfinite failures raise errors; confidence warnings remain.
 5  active = [r.EquivalentThrust_N,r.FuelFlow_kgs,r.Eta_thermal,r.Eta_propulsive,r.Eta_overall];
 6  if ismember(r.Architecture,{'Turboprop','Piston'})
 7      active = [active,r.ShaftPower_kW,r.BSFC_kgkWh];
 8  else
 9      active = [active,r.Thrust_N,r.TSFC_kgNs];
10  end
11  if ~isreal(active) || any(~isfinite(active)) || any(active<0)
12      error('ABACUS:ERR_NAN_01','An active output is negative, complex or nonfinite.');
13  end
14  if any([r.Eta_thermal,r.Eta_propulsive,r.Eta_overall]>1+1e-9)
15      error('ABACUS:EfficiencyOutput','Computed efficiency exceeds unity; the operating point is
     rejected.');
16  end
17  if ~isempty(r.Stations)
18      if any(r.Stations.Tt_K<=0 | r.Stations.Pt_Pa<=0 | r.Stations.MassFlow_kgs<=0)
19          error('ABACUS:StationState','Station temperature, pressure and flow must be positive.'
    );
20      end
21  end
22  d = r.Diagnostics;
23  if isfield(d,'MassResidual_kgs') && abs(d.MassResidual_kgs)>1e-8*max(1,d.AirFlow_kgs)
24      error('ABACUS:MassBalance','Mass conservation failed.');
25  end
26  for key = ["BurnerResidual_W","ShaftResidual_W"]
27      if isfield(d,key) && abs(d.(key))>1e-6*max(1,r.FuelFlow_kgs*43.2e6)
28          error('ABACUS:EnergyBalance','Energy balance failed at %s.',key);
29      end
30  end
31  r.Valid = true;
32  if isempty(r.Warnings), r.Status = 'PASS'; else, r.Status = 'WARNING'; end
33  end
```

matlab/validation/validate_physics.m

```
function validation = validate_physics(databaseFile)
%VALIDATE_PHYSICS Run internal physics tests, then independent-reference Track A.
% Input: optional DB file. Outputs: actual test results and fingerprinted certificates.
% No training certificate is accepted without executed tests and verified Track A PASS.
p=project_paths(); addpath(p.tests); if nargin<1, databaseFile=p.database_file; end
suite=matlab.unittest.TestSuite.fromFile(fullfile(p.tests,'TestAtmosphere.m'));
suite=[suite,matlab.unittest.TestSuite.fromFile(fullfile(p.tests,'TestEngines.m'))];
runner=matlab.unittest.TestRunner.withTextOutput; testResults=runner.run(suite);
internal=all([testResults.Passed]);
db=DB_Interface(databaseFile); cleanup=onCleanup(@() delete(db));
trackA=validate_references(db); writetable(trackA,fullfile(p.reports,'track_a_validation.csv')
);
engines=db.read('Engines'); certificates=cell(height(engines),1);
for i=1:height(engines)
    e=db_get_engine(db,engines.Engine_ID(i)); rows=trackA(trackA.Engine_ID==e.ID & trackA.Sour
ceVerified,:);
    accepted=internal && ~isempty(rows) && all(rows.Status=="PASS");
    cert=struct('Engine_ID',e.ID,'Engine',e.Name,'Fingerprint',e.Fingerprint,...
        'InternalTestsPassed',internal,'ReferenceAccepted',accepted,'AcceptedForML',accepted,.
..
        'Timestamp_UTC',utc_timestamp(),'TrackA',table2struct(trackA(trackA.Engine_ID==e.ID,:)
),...
        'Note','Only source-reviewed, in-tolerance reference comparisons open the default ML g
ate. Estimated-target comparisons retain their academic classification.');
    write_text(fullfile(p.reports,sprintf('validation_engine_%d.json',e.ID)),jsonencode(cert,'
PrettyPrint',true));
    certificates{i}=cert;
end
validation=struct('InternalTests',testResults,'TrackA',trackA,'Certificates',{certificates});
end
```

matlab/validation/validate_references.m

```
function report = validate_references(db,engineIDs)
%VALIDATE_REFERENCES Track A: compare exact reference conditions without fitting.
% Inputs: DB handle, optional IDs. Output: table of actual errors/status/source confidence.
% SI units; ABACUS Table 23.1. Unverified references and AB cases cannot receive PASS.
engines=db.read('Engines');
if nargin<2, engineIDs=engines.Engine_ID; end
refs=db.read('v_Reference_Data'); refs=refs(ismember(refs.Engine_ID,engineIDs),:);
p=project_paths(); policy=readtable(fullfile(p.database,'reference_policy.csv'));
parts=cell(height(refs),1);
for i=1:height(refs)
    ref=refs(i,:); e=db_get_engine(db,ref.Engine_ID);
    rule=policy.Tolerance_Fraction(policy.Engine_ID==e.ID);
    if isempty(rule), rule=.15; end
    if isfinite(ref.Thrust_N), metric='Thrust_N'; value=ref.Thrust_N; else, metric='ShaftPower
_kW'; value=ref.ShaftPower_kW; end
    result=simulate_engine(e,flight_condition(ref.GeopotentialAltitude_m,ref.Mach,ref.Throttle
,'geopotential',ref.Afterburner));
    actual=result.(metric); discrepancy=100*abs(actual-value)/abs(value);
    status="WARNING"; reason="Unverified secondary/estimated source; numerical comparison is n
ot manufacturer validation.";
    if ref.Afterburner
        reason="Afterburner reference retained for traceability but outside implemented scope.
";
    elseif ~result.Valid
        status="FAIL"; reason=strjoin(result.Warnings,' | ');
    elseif ref.Reference_Verified
        if discrepancy<=100*rule, status="PASS"; reason="Within configured tolerance at the st
ated independently verified condition.";
```

```
        else, status="FAIL"; reason="Prediction outside stated reference tolerance; no paramet
    er fitting performed."; end
        if string(ref.Reference_Kind)=="EstimatedTarget"
            reason="Academic estimated-target comparison only, not manufacturer/measurement va
    lidation. "+reason;
        end
    end
    parts{i}=table(e.ID,string(e.Name),ref.Data_ID,string(metric),value,actual,discrepancy,100
    *rule,...
        logical(ref.Reference_Verified),status,string(reason),string(ref.Source_Ref),string(re
    f.Confidence),...
        'VariableNames',{'Engine_ID','Engine','Data_ID','Metric','ReferenceValue','SimulatedVa
    lue',...
        'RelativeErrorPercent','TolerancePercent','SourceVerified','Status','Reason','Source',
    'Confidence'});
end
if isempty(parts), report=table(); else, report=vertcat(parts{:}); end
end
```

project_paths.m

```
function p = project_paths()
%PROJECT_PATHS Resolve portable project locations from this file.
% Inputs: none. Outputs: p structure of absolute paths. Units: filesystem paths.
% Assumption: this function stays in the extracted project root. No I/O mutations.
p.root = fileparts(mfilename('fullpath'));
for name = ["database","matlab","tests","gui","simulink","results","plots","tables","reports",
    "ml_models","docs"]
    p.(name) = fullfile(p.root,name);
end
p.database_file = fullfile(p.database,'engine_analysis.db');
p.test_database_file = fullfile(p.database,'engine_test.db');
end
```

run_all_tests.m

```
function results = run_all_tests()
%RUN_ALL_TESTS Execute unit, integration, ML, GUI and Simulink tests in a safe test DB.
% Inputs: none. Output: actual matlab.unittest results and CSV evidence.
% Production DB is never used by fixtures. Unavailable optional capabilities are incomplete.
p=project_paths(); addpath(p.root,genpath(p.matlab),p.gui,p.simulink,p.tests);
suite=matlab.unittest.TestSuite.fromFolder(p.tests,'IncludingSubfolders',false);
runner=matlab.unittest.TestRunner.withTextOutput; results=runner.run(suite);
summary=table(string({results.Name})',[results.Passed]',[results.Failed]',[results.Incomplete]
    ',[results.Duration]',...
    'VariableNames',{'Test','Passed','Failed','NotExecutedOrIncomplete','Duration_s'});
if ~isfolder(p.reports), mkdir(p.reports); end
writetable(summary,fullfile(p.reports,'matlab_test_results.csv'));
save(fullfile(p.reports,'matlab_test_results.mat'),'results');
fprintf('Passed: %d | Failed: %d | Incomplete: %d\n',nnz(summary.Passed),nnz(summary.Failed),n
    nz(summary.NotExecutedOrIncomplete));
end
```

run_project.m

```
function output = run_project(action)
%RUN_PROJECT Entry point for the ABACUS academic engineering application.
% Input: gui (default), demo, grid, validate, train or simulink. Output: action result.
% Requires MATLAB R2023b/Database Toolbox; ML and Simulink check their own dependencies.
if nargin<1, action='gui'; end
p=setup_project();
if strcmp(action,'gui'), output=EngineAnalyser_v1(p.database_file); return; end
```

```
dt=DT_Coordinator(p.database_file); cleanup=onCleanup(@() delete(dt));
switch lower(char(action))
    case 'demo'
        f=flight_condition(0,0,1); r=dt.run(4,f,'L4',true);
        if ~r.Valid, error('ABACUS:Demo','%s',strjoin(r.Warnings,' | ')); end
        comparison=compare_engines(dt,[4,5],f,'Thrust_N');
        selection=EngineSelector(dt,1,[],true);
        report=generate_report(r,comparison,selection,table());
        disp(format_result_metrics(r,'SI')); fprintf('Report: %s\n',report);
        output=struct('Result',r,'Comparison',comparison,'Selection',selection,'Report',report
);
    case 'grid'
        engines=dt.Database.read('Engines'); parts=cell(height(engines),1);
        for i=1:height(engines)
            parts{i}=dt.map(engines.Engine_ID(i),true);
            writetable(parts{i},fullfile(p.tables,sprintf('engine_%d_grid.csv',engines.Engine_
ID(i))));
        end
        output=vertcat(parts{:});
    case 'validate'
        output=validate_physics(p.database_file);
    case 'train'
        % User-facing training does not bypass current executed Track A validation.
        validate_physics(p.database_file); engines=dt.Database.read('Engines'); rows=cell(heig
ht(engines),1);
        for i=1:height(engines)
            id=engines.Engine_ID(i);
            try
                metrics=ml_train_surrogates(dt,id); status="PASS"; note="Training executed; in
spect actual Track B acceptance metrics.";
                if ~all(metrics.MeetsAccuracyCriteria(metrics.Selected)), status="WARNING"; en
d
            catch exception
                status="WARNING"; note=string(exception.message);
            end
            rows{i}=table(id,status,note,'VariableNames',{'Engine_ID','ExecutionStatus','Diagn
ostic'});
        end
        output=vertcat(rows{:}); disp(output); writetable(output,fullfile(p.reports,'training_
execution_status.csv'));
    case 'simulink'
        output=build_simulink_models(p.database_file);
    otherwise
        error('ABACUS:Action','Choose gui, demo, grid, validate, train or simulink.');
end
end
```

scripts/audit_dependencies.m

```
function report = audit_dependencies()
%AUDIT_DEPENDENCIES Run MATLAB's dependency analyser and Code Analyzer on all sources.
% Input: none. Output: actual required files/products and diagnostics saved in reports.
% Dynamic GUI action strings/S-function names are also exercised by integration tests.
p=project_paths(); files=dir(fullfile(p.root,'','*.m'));
paths=cell(numel(files),1); issues=cell(numel(files),1);
for i=1:numel(files)
    paths{i}=fullfile(files(i).folder,files(i).name); issues{i}=checkcode(paths{i},'-id');
end
[requiredFiles,products]=matlab.codetools.requiredFilesAndProducts(paths);
report=struct('Timestamp_UTC',utc_timestamp(),'Files',{requiredFiles},'Products',products,...
    'AnalysedSources',{paths},'CodeAnalyzer',{issues});
save(fullfile(p.reports,'matlab_dependency_audit.mat'),'report');
write_text(fullfile(p.reports,'matlab_dependency_audit.json'),jsonencode(report,'PrettyPrint',
```

```
    true));
15  end
```

scripts/run_phase_tests.m

```
 1  function results = run_phase_tests(phase)
 2  %RUN_PHASE_TESTS Run the existing relevant tests for one numbered implementation phase.
 3  % Input: phase integer 1-15. Output: actual results; does not assert phase completion.
 4  p=project_paths(); addpath(p.root,genpath(p.matlab),p.gui,p.simulink,p.tests);
 5  validateattributes(phase,{'numeric'},{'scalar','integer','>=',1,'<=',15});
 6  mapping={'TestDatabase.m','TestAtmosphere.m','TestEngines.m','TestEngines.m',...
 7      'TestEngines.m','TestEngines.m','TestEngines.m','TestIntegration.m','TestIntegration.m',..
    .
 8      'TestIntegration.m','TestMission.m','TestML.m','TestML.m','TestGUI.m','TestSimulink.m'};
 9  suite=matlab.unittest.TestSuite.fromFile(fullfile(p.tests,mapping{phase}));
10  runner=matlab.unittest.TestRunner.withTextOutput; results=runner.run(suite);
11  end
```

setup_project.m

```
 1  function p = setup_project(databaseFile)
 2  %SETUP_PROJECT Add authoritative source folders and initialise an absent database.
 3  % Input: optional SQLite path. Output: path structure p. Units: paths.
 4  % Requires MATLAB R2023b and Database Toolbox; throws actionable dependency errors.
 5  p = project_paths();
 6  if nargin < 1, databaseFile = p.database_file; end
 7  if verLessThan('matlab','23.2'), error('ABACUS:Version','MATLAB R2023b or newer is required.')
    ; end
 8  addpath(p.root,genpath(p.matlab),p.gui,p.simulink,fullfile(p.root,'scripts'));
 9  if exist('sqlite','file') == 0
10      error('ABACUS:Dependency','Install Database Toolbox to use the native SQLite interface.');
11  end
12  for name = ["results","plots","tables","reports","ml_models"]
13      if ~isfolder(p.(name)), mkdir(p.(name)); end
14  end
15  db_initialize(databaseFile);
16  db_seed(databaseFile);
17  p.database_file = databaseFile;
18  end
```

simulink/abacus_sfun.m

```
 1  function abacus_sfun(block)
 2  %ABACUS_SFUN Level-2 MATLAB S-function wrapping the SAME steady-state backend.
 3  % Input port: [engineID, geometric altitude m, Mach, throttle]; dialog: DB path.
 4  % Output: [jet thrust N, equivalent thrust N, shaft kW, fuel kg/s, TSFC, BSFC, eta_th, valid].
 5  % No continuous/discrete engine dynamics or code generation is claimed.
 6  block.NumDialogPrms=1; block.DialogPrmsTunable={'Nontunable'};
 7  block.NumInputPorts=1; block.NumOutputPorts=1;
 8  block.SetPreCompInpPortInfoToDynamic; block.SetPreCompOutPortInfoToDynamic;
 9  block.InputPort(1).Dimensions=4; block.InputPort(1).DatatypeID=0;
10  block.InputPort(1).Complexity='Real'; block.InputPort(1).DirectFeedthrough=true;
11  block.OutputPort(1).Dimensions=8; block.OutputPort(1).DatatypeID=0;
12  block.OutputPort(1).Complexity='Real'; block.SampleTimes=[-1,0];
13  block.SimStateCompliance='DefaultSimState'; block.RegBlockMethod('Outputs',@abacus_outputs);
14  end
15  function abacus_outputs(block)
16  %ABACUS_OUTPUTS Evaluate an equilibrium point; invalid active outputs halt simulation.
17  u=block.InputPort(1).Data; dt=DT_Coordinator(block.DialogPrm(1).Data);
18  cleanup=onCleanup(@() delete(dt));
19  r=dt.run(u(1),flight_condition(u(2),u(3),u(4)),'L1',false);
20  if ~r.Valid, error('ABACUS:SimulinkPoint','%s',strjoin(r.Warnings,' | ')); end
```

```
block.OutputPort(1).Data=[r.Thrust_N;r.EquivalentThrust_N;r.ShaftPower_kW;r.FuelFlow_kgs;...
    r.TSFC_kgNs;r.BSFC_kgkWh;r.Eta_thermal;double(r.Valid)];
end
```

simulink/build_simulink_models.m

```
function models = build_simulink_models(databaseFile,outputFolder)
%BUILD_SIMULINK_MODELS Create three connected .slx models using the shared S-function.
% Inputs: optional DB path/output folder. Outputs: model names and real .slx files.
% Requires licensed Simulink; build and execution status are separate. Ref: ABACUS stack.
p=project_paths(); if nargin<1, databaseFile=p.database_file; end
if nargin<2, outputFolder=p.simulink; end
if ~license('test','Simulink'), error('ABACUS:SimulinkRequired','Install/enable Simulink to build models.'); end
if ~isfolder(outputFolder), mkdir(outputFolder); end
load_system('simulink'); models={'ABACUS_Turbojet','ABACUS_Turbofan','ABACUS_Turboprop'}; ids=[1,4,9];
for i=1:3
    model=models{i};
    if bdIsLoaded(model), error('ABACUS:ModelOpen','Close %s before rebuilding to protect unsaved edits.',model); end
    new_system(model); cleanup=onCleanup(@() close_system(model,0));
    workspace=get_param(model,'ModelWorkspace'); assignin(workspace,'ABACUS_DATABASE',char(databaseFile));
    add_block('simulink/Sources/Constant',[model,'/OperatingPoint'],...
        'Value',sprintf('[%d;0;0;1]',ids(i)),'Position',[35,95,160,135]);
    add_block('simulink/User-Defined Functions/Level-2 MATLAB S-Function',[model,'/SteadyStateEngine'],...
        'FunctionName','abacus_sfun','Parameters','ABACUS_DATABASE','Position',[225,70,420,165]);
    add_block('simulink/Sinks/To Workspace',[model,'/Result'],...
        'VariableName','abacus_output','SaveFormat','Array','Position',[480,90,600,140]);
    add_line(model,'OperatingPoint/1','SteadyStateEngine/1'); add_line(model,'SteadyStateEngine/1','Result/1');
    set_param(model,'SolverType','Fixed-step','Solver','FixedStepDiscrete','FixedStep','1',...
        'StopTime','0','ReturnWorkspaceOutputs','on');
    annotation=Simulink.Annotation(model,'Academic steady-state evaluation. No transient engine dynamics.');
    annotation.Position=[35,20];
    save_system(model,fullfile(outputFolder,[model,'.slx'])); clear cleanup
end
end
```

simulink/run_simulink_model.m

```
function output = run_simulink_model(modelName,operatingPoint,databaseFile)
%RUN_SIMULINK_MODEL Build if needed, set [ID,h,M,theta], and execute one equilibrium.
% Inputs: model name, 4-element numeric point, optional DB path. Output: SimulationOutput.
% No state integration or performance claims until this command actually runs.
p=project_paths(); if nargin<1, modelName='ABACUS_Turbojet'; end
if nargin<2, operatingPoint=[1;0;0;1]; end
if nargin<3, databaseFile=p.database_file; end
validatestring(modelName,{'ABACUS_Turbojet','ABACUS_Turbofan','ABACUS_Turboprop'});
validateattributes(operatingPoint,{'numeric'},{'vector','numel',4,'finite','real'});
file=fullfile(p.simulink,[modelName,'.slx']);
if ~isfile(file), build_simulink_models(databaseFile); end
load_system(file); cleanup=onCleanup(@() close_system(modelName,0));
input=Simulink.SimulationInput(modelName);
input=input.setVariable('ABACUS_DATABASE',char(databaseFile),'Workspace',modelName);
input=input.setBlockParameter([modelName,'/OperatingPoint'],'Value',mat2str(operatingPoint(:)));
output=sim(input);
```

```
17  end
```

tests/TestAtmosphere.m

```
 1  function tests = TestAtmosphere
 2  %TESTATMOSPHERE Analytic/rounded geopotential checkpoints, not claimed ICAO certification.
 3  tests=functiontests(localfunctions);
 4  end
 5  function testSeaLevel(testCase)
 6  a=ISA_model(0); verifyEqual(testCase,a.T_K,288.15,'AbsTol',1e-10);
 7  verifyEqual(testCase,a.P_Pa,101325,'AbsTol',1e-8); verifyEqual(testCase,a.Density_kgm3,1.225,'
    RelTol',.001);
 8  verifyEqual(testCase,a.SoundSpeed_ms,340.294,'RelTol',.001);
 9  end
10  function testGeopotentialCheckpoints(testCase)
11  % Rounded hydrostatic analytical checkpoints; corrupted PDF mixed-altitude cells excluded.
12  h=[0,2000,5000,8000,11000,15000,20000,25000,32000];
13  T=[288.15,275.15,255.65,236.15,216.65,216.65,216.65,221.65,228.65];
14  P=[101325,79495,54020,35600,22632,12045,5475,2511,868];
15  for i=1:numel(h)
16      a=ISA_model(h(i),0,'geopotential'); verifyEqual(testCase,a.T_K,T(i),'AbsTol',1e-8);
17      verifyEqual(testCase,a.P_Pa,P(i),'RelTol',.001);
18  end
19  end
20  function testLayerContinuity(testCase)
21  for h=[11000,20000]
22      left=ISA_model(h-1e-5,0,'geopotential'); right=ISA_model(h+1e-5,0,'geopotential');
23      verifyEqual(testCase,left.P_Pa,right.P_Pa,'RelTol',1e-7);
24      verifyEqual(testCase,left.T_K,right.T_K,'AbsTol',1e-6);
25  end
26  end
27  function testGeometricConversion(testCase)
28  a=ISA_model(11000); b=ISA_model(a.GeopotentialAltitude_m,0,'geopotential');
29  verifyLessThan(testCase,a.GeopotentialAltitude_m,11000); verifyEqual(testCase,a.T_K,b.T_K,'Abs
    Tol',1e-9);
30  verifyEqual(testCase,b.Altitude_m,11000,'AbsTol',1e-8);
31  end
32  function testMachStagnation(testCase)
33  a=ISA_model(0,2); verifyEqual(testCase,a.Tt_K,518.67,'AbsTol',1e-8);
34  verifyGreaterThan(testCase,a.Pt_Pa,a.P_Pa); verifyGreaterThan(testCase,a.DynamicViscosity_Pas,
    0);
35  end
36  function testNegativeAltitude(testCase)
37  verifyError(testCase,@() ISA_model(-1),'ABACUS:ERR_ATM_01');
38  end
39  function testUpperBound(testCase)
40  verifyError(testCase,@() ISA_model(32001),'ABACUS:ERR_ATM_01');
41  verifyError(testCase,@() ISA_model(32001,0,'geopotential'),'ABACUS:ERR_ATM_01');
42  end
43  function testInvalidMachAndThrottle(testCase)
44  verifyError(testCase,@() flight_condition(0,-.1,1),'ABACUS:ERR_MACH_01');
45  verifyError(testCase,@() flight_condition(0,0,1.01),'ABACUS:ERR_IN_01');
46  end
```

tests/TestDatabase.m

```
 1  function tests = TestDatabase
 2  %TESTDATABASE Execute real native SQLite constraints, provenance and transaction checks.
 3  tests=functiontests(localfunctions);
 4  end
 5  function setup(testCase)
 6  [testCase.TestData.db,testCase.TestData.file]=new_test_database();
```

```
 7  end
 8  function teardown(testCase)
 9  delete(testCase.TestData.db); delete(testCase.TestData.file);
10  end
11  function testAllTablesAndEngines(testCase)
12  db=testCase.TestData.db; t=db.read('Engines'); verifyEqual(testCase,height(t),11);
13  names={'Manufacturers','Engine_Types','Engines','Engine_Thermo_Params','Parameter_Provenance',
    ...
14      'Aircraft','Engine_Aircraft_Map','Performance_Data','Mission_Profiles','References','Simul
    ation_Log'};
15  for i=1:numel(names), verifyTrue(testCase,istable(db.read(names{i}))); end
16  end
17  function testSeedIdempotent(testCase)
18  db_seed(testCase.TestData.file); verifyEqual(testCase,height(testCase.TestData.db.read('Engine
    s')),11);
19  verifyEqual(testCase,height(testCase.TestData.db.read('Performance_Data')),13);
20  end
21  function testProvenanceAudit(testCase)
22  for id=1:11
23      e=db_get_engine(testCase.TestData.db,id); verifyGreaterThan(testCase,height(e.Provenance),
    10);
24      verifyTrue(testCase,all(string(e.Provenance.Confidence_Level)=="Low"));
25  end
26  end
27  function testForeignKeyFailure(testCase)
28  db=testCase.TestData.db;
29  t=table(900,"Boundary test",999,1,'VariableNames',{'Engine_ID','Designation','Manufacturer_ID'
    ,'Type_ID'});
30  failed=false; try, db.append('Engines',t); catch, failed=true; end
31  verifyTrue(testCase,failed); verifyEqual(testCase,height(db.read('Engines')),11);
32  end
33  function testLiteralTextRoundTrip(testCase)
34  % Deliberately hostile-looking text must remain a typed value, never SQL syntax.
35  value="O'Brien'); DROP TABLE Engines; --";
36  t=table(900,value,"Test",'VariableNames',{'Manufacturer_ID','Name','Country'});
37  db=testCase.TestData.db; db.append('Manufacturers',t); all=db.read('Manufacturers');
38  verifyEqual(testCase,string(all.Name(all.Manufacturer_ID==900)),value); verifyEqual(testCase,h
    eight(db.read('Engines')),11);
39  end
40  function testAtomicRollback(testCase)
41  db=testCase.TestData.db; before=height(db.read('Manufacturers')); failed=false;
42  try, db.atomic(@() write_then_fail(db)); catch, failed=true; end
43  verifyTrue(testCase,failed); verifyEqual(testCase,height(db.read('Manufacturers')),before);
44  end
45  function write_then_fail(db)
46  % Two-row transaction fixture: first write valid, then a deliberate failure.
47  db.append('Manufacturers',table(900,"Transaction test","Test",'VariableNames',{'Manufacturer_I
    D','Name','Country'}));
48  error('ABACUS:IntentionalTestFailure','Exercise atomic rollback.');
49  end
50  function testImmutableReference(testCase)
51  db=testCase.TestData.db; failed=false;
52  try, execute(db.Connection,'UPDATE Performance_Data SET Thrust_N=1 WHERE Data_ID=1'); catch, f
    ailed=true; end
53  verifyTrue(testCase,failed); t=db.read('Performance_Data'); verifyEqual(testCase,t.Thrust_N(t.
    Data_ID==1),52800);
54  end
55  function testMissingProvenanceRejected(testCase)
56  db=testCase.TestData.db; execute(db.Connection,'DELETE FROM Parameter_Provenance WHERE Engine_
    ID=1 AND Parameter_Name=''OPR''');
57  verifyError(testCase,@() db_get_engine(db,1),'ABACUS:MissingProvenance');
58  end
59  function testInvalidEngineID(testCase)
60  verifyError(testCase,@() db_get_engine(testCase.TestData.db,99999),'ABACUS:InvalidEngine');
```

```
61  end
62  function testSchemaConstraints(testCase)
63  db=testCase.TestData.db; failed=false;
64  try, execute(db.Connection,'UPDATE Engine_Thermo_Params SET Eta_turbine=-1 WHERE Engine_ID=1')
    ; catch, failed=true; end
65  verifyTrue(testCase,failed);
66  failed=false; try, execute(db.Connection,'UPDATE Mission_Profiles SET W_Thrust=1 WHERE Mission
    _ID=1'); catch, failed=true; end
67  verifyTrue(testCase,failed);
68  end
```

tests/TestEngines.m

```
 1  function tests = TestEngines
 2  %TESTENGINES Test analytic components, mass/work balances and architecture boundaries.
 3  % These are internal model checks; passing them does not prove measured engine accuracy.
 4  tests=functiontests(localfunctions);
 5  end
 6  function setupOnce(testCase)
 7  [testCase.TestData.db,testCase.TestData.file]=new_test_database();
 8  db_seed_piston_extension(testCase.TestData.file);
 9  end
10  function teardownOnce(testCase)
11  delete(testCase.TestData.db); delete(testCase.TestData.file);
12  end
13  function testIdealCompressionCheckpoint(testCase)
14  c=project_config(); s=compress_stage(300,100000,2^(c.gamma_air/(c.gamma_air-1)),1,c.cp_air,c.g
    amma_air);
15  verifyEqual(testCase,s.Tt_K,600,'AbsTol',1e-8); verifyEqual(testCase,s.Work_Jkg,300*c.cp_air,'
    RelTol',1e-12);
16  end
17  function testNozzleChoked(testCase)
18  c=project_config(); n=nozzle_model(1000,250000,100000,10,c.gamma_gas,c.R_gas,1);
19  verifyTrue(testCase,n.Choked); verifyEqual(testCase,n.Mach,1,'AbsTol',1e-10);
20  verifyGreaterThan(testCase,n.PressureThrust_N,0); verifyEqual(testCase,n.T_K,2000/2.33,'AbsTol
    ',1e-8);
21  verifyLessThan(testCase,abs(n.EnergyResidual_W),1e-6);
22  end
23  function testNozzleUnchoked(testCase)
24  c=project_config(); n=nozzle_model(1000,150000,100000,10,c.gamma_gas,c.R_gas,.98);
25  verifyFalse(testCase,n.Choked); verifyLessThan(testCase,n.Mach,1); verifyEqual(testCase,n.P_Pa
    ,100000,'AbsTol',1e-8);
26  end
27  function testNozzleBoundaryContinuity(testCase)
28  c=project_config(); n=nozzle_model(900,300000,100000,5,c.gamma_gas,c.R_gas,.98);
29  left=nozzle_model(900,100000*n.NPR_critical*(1-1e-8),100000,5,c.gamma_gas,c.R_gas,.98);
30  right=nozzle_model(900,100000*n.NPR_critical*(1+1e-8),100000,5,c.gamma_gas,c.R_gas,.98);
31  verifyEqual(testCase,left.GrossThrust_N,right.GrossThrust_N,'RelTol',1e-6);
32  end
33  function testNoExpansionRejected(testCase)
34  verifyError(testCase,@() nozzle_model(900,90000,100000,5,1.33,287.4,.98),'ABACUS:ERR_NOZ_02');
35  end
36  function testAllCoreEnginesSLS(testCase)
37  for id=1:11
38      e=db_get_engine(testCase.TestData.db,id); r=simulate_engine(e,flight_condition(0,0,1));
39      verifyTrue(testCase,r.Valid,strjoin(r.Warnings,' | ')); verifyGreaterThan(testCase,r.Equiv
    alentThrust_N,0);
40      verifyLessThan(testCase,abs(r.Diagnostics.MassResidual_kgs),1e-8);
41      verifyLessThan(testCase,abs(r.Diagnostics.ShaftResidual_W),1e-6);
42      verifyLessThan(testCase,abs(r.Diagnostics.BurnerResidual_W),1e-6);
43  end
44  end
45  function testTurbofanMassAndFanDemand(testCase)
```

```
e=db_get_engine(testCase.TestData.db,4); r=TF_Model(e,flight_condition(0,0,1)); d=r.Diagnostic
s;
verifyEqual(testCase,d.BypassFlow_kgs/d.CoreFlow_kgs,e.Params.BPR,'RelTol',1e-12);
verifyGreaterThan(testCase,d.FanWork_Jkg,0); verifyTrue(testCase,ismember(18,r.Stations.Statio
n));
verifyEqual(testCase,d.AirFlow_kgs,e.Params.MassFlow_kgs,'RelTol',1e-12);
end
function testTurbopropPowerAndStaticCase(testCase)
e=db_get_engine(testCase.TestData.db,9); r=TP_Model(e,flight_condition(0,0,1));
verifyTrue(testCase,isfinite(r.EquivalentThrust_N)); verifyGreaterThan(testCase,r.ShaftPower_k
W,0);
verifyTrue(testCase,ismember(7,r.Stations.Station)); verifyTrue(testCase,isnan(r.Thrust_N));
verifyEqual(testCase,r.BSFC_kgkWh,r.FuelFlow_kgs*3600/r.ShaftPower_kW,'RelTol',1e-12);
verifyGreaterThan(testCase,r.Diagnostics.CoreThrust_N,0);
end
function testPropellerContinuousAndFinite(testCase)
velocities=[0,1e-9,16.999999,17,17.000001]; thrust=zeros(size(velocities));
for i=1:numel(velocities), p=propeller_model(1e6,.8,1.225,3,velocities(i),340,1500); thrust(i)
=p.Thrust_N; end
verifyTrue(testCase,all(isfinite(thrust))); verifyEqual(testCase,thrust(1),thrust(2),'RelTol',
1e-7);
verifyEqual(testCase,thrust(3),thrust(5),'RelTol',1e-6);
end
function testKaveriWarningAndType(testCase)
e=db_get_engine(testCase.TestData.db,8); r=simulate_engine(e,flight_condition(0,0,1));
verifyEqual(testCase,r.Data_Type,'Simulated_Estimated'); verifyTrue(testCase,any(contains(r.Wa
rnings,'NOT MANUFACTURER-CONFIRMED')));
end
function testAltitudeAndMachEnvelope(testCase)
e=db_get_engine(testCase.TestData.db,9);
r=simulate_engine(e,flight_condition(9500,0,1,'geopotential')); verifyFalse(testCase,r.Valid);
r=simulate_engine(e,flight_condition(0,.56,1)); verifyFalse(testCase,r.Valid);
end
function testAfterburnerRejected(testCase)
e=db_get_engine(testCase.TestData.db,1); r=simulate_engine(e,flight_condition(0,0,1,'geometric
',true));
verifyFalse(testCase,r.Valid); verifyEqual(testCase,r.Error_Code,'ABACUS:AfterburnerUnsupporte
d');
end
function testBadEfficiencyAndTemperature(testCase)
e=db_get_engine(testCase.TestData.db,1); e.Params.Eta_compressor=-1;
verifyError(testCase,@() TJ_Model(e,flight_condition(0,0,1)),'ABACUS:Efficiency');
e=db_get_engine(testCase.TestData.db,1); e.Params.T_t4_K=400;
r=simulate_engine(e,flight_condition(0,0,1)); verifyFalse(testCase,r.Valid);
end
function testPistonPowerTorqueAndAltitude(testCase)
e=db_get_engine(testCase.TestData.db,12); sea=Piston_Model(e,flight_condition(0,0,1));
high=Piston_Model(e,flight_condition(3000,0,1));
verifyLessThan(testCase,high.ShaftPower_kW,sea.ShaftPower_kW);
verifyEqual(testCase,sea.Diagnostics.Torque_Nm*e.Params.Rated_RPM*2*pi/60,sea.ShaftPower_kW*10
00,'RelTol',1e-12);
verifyLessThan(testCase,sea.Eta_thermal,sea.Diagnostics.IdealOttoEfficiency);
end
function testZeroThrottleState(testCase)
e=db_get_engine(testCase.TestData.db,4); r=simulate_engine(e,flight_condition(0,0,0));
verifyTrue(testCase,r.Valid); verifyEqual(testCase,r.FuelFlow_kgs,0); verifyEqual(testCase,r.E
quivalentThrust_N,0);
end
function testChangedParametersInvalidateFingerprint(testCase)
e=db_get_engine(testCase.TestData.db,1); old=e.Fingerprint; e.Params.OPR=e.Params.OPR+1;
verifyNotEqual(testCase,model_fingerprint(e),old);
end
```

tests/TestGUI.m

```
function tests = TestGUI
%TESTGUI Optional real eight-tab walkthrough and warning capture; never a static mockup.
tests=functiontests(localfunctions);
end
function testEightTabsAndKaveriWarning(testCase)
assumeTrue(testCase,usejava('desktop'),'MATLAB desktop unavailable; GUI walkthrough NOT EXECUTED.');
[db,file]=new_test_database(); delete(db); app=EngineAnalyser_v1(file);
cleanup=onCleanup(@() close_app_fixture(app,file));
verifyEqual(testCase,numel(app.Controls.TabGroup.Children),8);
app.Controls.Engine.Value=8; gui_action(app,'engine'); gui_action(app,'physics');
verifyTrue(testCase,contains(app.Controls.Banner.Text,'NOT MANUFACTURER-CONFIRMED'));
verifyFalse(testCase,isempty(app.Controls.Metrics.Data));
p=project_paths(); folder=fullfile(p.reports,'gui_walkthrough'); if ~isfolder(folder), mkdir(folder); end
tabs=app.Controls.TabGroup.Children;
for i=1:numel(tabs)
    app.Controls.TabGroup.SelectedTab=tabs(i); drawnow;
    exportapp(app.UIFigure,fullfile(folder,sprintf('tab_%d.png',i)));
end
end
function close_app_fixture(app,file)
%CLOSE_APP_FIXTURE Release GUI and DB before removing the fixture database.
delete(app); delete(file);
end
```

tests/TestIntegration.m

```
function tests = TestIntegration
%TESTINTEGRATION Exercise the real DB -> physics -> DT -> grid/comparison/report path.
tests=functiontests(localfunctions);
end
function setupOnce(testCase)
[db,file]=new_test_database(); delete(db);
testCase.TestData.file=file; testCase.TestData.dt=DT_Coordinator(file);
end
function teardownOnce(testCase)
delete(testCase.TestData.dt); delete(testCase.TestData.file);
end
function testPersistenceAndLog(testCase)
dt=testCase.TestData.dt; before=height(dt.Database.read('Performance_Data'));
r=dt.run(1,flight_condition(0,0,1),'L4',true);
verifyTrue(testCase,r.Valid); t=dt.Database.read('Performance_Data'); verifyEqual(testCase,height(t),before+1);
verifyEqual(testCase,string(t.Data_Type(end)),"Simulated_Estimated");
logs=dt.Database.read('Simulation_Log'); verifyTrue(testCase,contains(string(logs.Outputs_JSON(end)),'CoreNozzle'));
end
function testFailedPointLogsWithoutPerformance(testCase)
dt=testCase.TestData.dt; before=height(dt.Database.read('Performance_Data')); n=height(dt.Database.read('Simulation_Log'));
r=dt.run(9,flight_condition(10000,0,1),'L4',true); verifyFalse(testCase,r.Valid);
verifyEqual(testCase,height(dt.Database.read('Performance_Data')),before);
verifyEqual(testCase,height(dt.Database.read('Simulation_Log')),n+1);
end
function testLogImmutable(testCase)
dt=testCase.TestData.dt; dt.run(1,flight_condition(0,0,1),'L4',true); failed=false;
try, execute(dt.Database.Connection,'DELETE FROM Simulation_Log'); catch, failed=true; end
verifyTrue(testCase,failed);
end
function testArchitectureGridCounts(testCase)
```

```
db=testCase.TestData.dt.Database;
for id=[1,6], e=db_get_engine(db,id); g=performance_grid(e); verifyEqual(testCase,g.CandidateCount,1134); end
for id=[4,9], e=db_get_engine(db,id); g=performance_grid(e); verifyEqual(testCase,g.CandidateCount,1008); end
end
function testGridValidityMaskAndTemperatureFields(testCase)
dt=testCase.TestData.dt; e=db_get_engine(dt.Database,9);
t=evaluate_grid(e,[0,9500],[0,.7],1,dt.Database,false,'L3');
verifyEqual(testCase,height(t),4); verifyEqual(testCase,nnz(t.Valid),1);
verifyTrue(testCase,all(isnan(t.FuelFlow_kgs(~logical(t.Valid))))); verifyGreaterThan(testCase,t.T_t4_K(1),0);
end
function testCompatibleComparison(testCase)
t=compare_engines(testCase.TestData.dt,[1,2],flight_condition(0,0,1),'Thrust_N');
verifyEqual(testCase,height(t),2); verifyGreaterThanOrEqual(testCase,t.Thrust_N(1),t.Thrust_N(2));
end
function testIncompatibleComparisonRejected(testCase)
verifyError(testCase,@() compare_engines(testCase.TestData.dt,[1,9],flight_condition(0,0,1),'TSFC_kgNs'),'ABACUS:IncompatibleMetric');
end
function testAllMissionPresets(testCase)
for id=1:7
    s=EngineSelector(testCase.TestData.dt,id,[],false);
    verifyTrue(testCase,isfield(s,'Eliminated')); verifyGreaterThan(testCase,strlength(s.Message),0);
    if ~isempty(s.Ranking), verifyTrue(testCase,all(strlength(s.Ranking.EliminationReason)==0)); end
end
end
function testUnverifiedReferencesCannotPass(testCase)
t=validate_references(testCase.TestData.dt.Database); verifyFalse(testCase,any(t.Status=="PASS"));
verifyEqual(testCase,height(t),13);
end
function testReportingContainsProvenance(testCase)
r=testCase.TestData.dt.run(8,flight_condition(0,0,1),'L1',false);
folder=tempname; mkdir(folder); cleanup=onCleanup(@() rmdir(folder,'s'));
file=generate_report(r,table(),struct(),table(),folder); html=fileread(file);
verifyTrue(testCase,contains(html,'NOT MANUFACTURER-CONFIRMED')); verifyTrue(testCase,contains(html,'Parameter provenance'));
verifyTrue(testCase,isfile(fullfile(folder,'ABACUS_session_data.json')));
end
function testOutputUnitConversion(testCase)
r=testCase.TestData.dt.run(1,flight_condition(0,0,1),'L1',false);
a=format_result_metrics(r,'SI'); b=format_result_metrics(r,'Imperial');
verifyEqual(testCase,a.Value(a.Quantity=="Thrust_N"),r.Thrust_N/1000,'RelTol',1e-12);
verifyEqual(testCase,b.Value(b.Quantity=="Thrust_N"),r.Thrust_N/4.4482216152605,'RelTol',1e-12);
verifyEqual(testCase,a.Value(a.Quantity=="TSFC_kgNs"),r.TSFC_kgNs*3.6e6,'RelTol',1e-12);
end
function testHTMLTextEscaped(testCase)
verifyEqual(testCase,html_escape('<script>&"'), '&lt;script&gt;&amp;&quot;');
end
```

tests/TestML.m

```
function tests = TestML
%TESTML Test split leakage, fit/predict APIs, metrics, acceptance gate and domain holes.
% Analytic response fixtures are labelled mathematical tests, never engine performance data.
tests=functiontests(localfunctions);
end
```

```
function X = analytic_grid()
%ANALYTIC_GRID Deterministic unique inputs for algorithm unit tests only.
[m,h,t]=ndgrid(linspace(0,1,9),linspace(0,10000,9),linspace(.5,1,6)); X=[m(:),h(:),t(:)];
end
function testSplitDeterministicAndDisjoint(testCase)
X=analytic_grid(); a=ml_stratified_split(X,42); b=ml_stratified_split(X,42);
verifyEqual(testCase,a.Train,b.Train); verifyFalse(testCase,any(a.Train&a.Test | a.Train&a.Validation | a.Test&a.Validation));
verifyTrue(testCase,all(a.Train|a.Validation|a.Test)); verifyEqual(testCase,sum(a.ActualFractions),1,'AbsTol',1e-12);
verifyLessThan(testCase,max(abs(a.ActualFractions-[.7,.15,.15])),.035);
for group=unique(a.Group)'
    idx=a.Group==group; verifyTrue(testCase,any(a.Train&idx)); verifyTrue(testCase,any(a.Validation&idx)); verifyTrue(testCase,any(a.Test&idx));
end
end
function testDuplicateInputsRejected(testCase)
X=analytic_grid(); X(end,:)=X(1,:); verifyError(testCase,@() ml_stratified_split(X),'ABACUS:MLDuplicate');
end
function testMetricsKnownErrors(testCase)
t=ml_metrics([1;2;3],[2;3;4],{'AnalyticResponse'});
verifyEqual(testCase,t.RMSE,1,'AbsTol',1e-12); verifyEqual(testCase,t.MAE,1,'AbsTol',1e-12);
verifyEqual(testCase,t.R2,-.5,'AbsTol',1e-12);
end
function testConstantAndZeroTargets(testCase)
t=ml_metrics(ones(4,1),ones(4,1),{'Constant'}); verifyTrue(testCase,isnan(t.R2));
t=ml_metrics([0;1;2],[0;1;2],{'ZeroIncluded'}); verifyEqual(testCase,t.MAPE_Count,2); verifyEqual(testCase,t.MAPE_Percent,0);
end
function testPolynomialTrainPredict(testCase)
X=analytic_grid(); y=2+3*X(:,1)+1e-4*X(:,2)+X(:,3).^2; s=ml_stratified_split(X);
model=ml_fit_candidate(X(s.Train,:),y(s.Train),'Poly',1); prediction=ml_apply_candidate(model,X(s.Test,:));
verifyLessThan(testCase,max(abs(prediction-y(s.Test))),1e-4);
end
function testFourCandidateAPIs(testCase)
assumeTrue(testCase,license('test','Statistics_Toolbox'),'Statistics Toolbox unavailable; four-family execution not performed.');
X=analytic_grid(); X=X(1:5:end,:); y=2+X(:,1)+X(:,2)/10000+X(:,3); old=rng; clean=onCleanup(@() rng(old)); rng(42,'twister');
for family={'Poly','RF','GPR','ANN'}
    model=ml_fit_candidate(X,y,family{1},1); prediction=ml_apply_candidate(model,X(1:3,:));
    verifySize(testCase,prediction,[3,1]); verifyTrue(testCase,all(isfinite(prediction)));
end
end
function testMLCellRejectsHolesAndExtrapolation(testCase)
b.Grid=struct('Altitude_m',[0,1000],'Mach',[0,.5],'Throttle',[.5,1]); b.ValidityMask=true(2,2,2);
f=flight_condition(500,.25,.75,'geopotential'); verifyTrue(testCase,ml_domain_valid(b,f));
b.ValidityMask(2,2,2)=false; verifyFalse(testCase,ml_domain_valid(b,f));
verifyFalse(testCase,ml_domain_valid(b,flight_condition(1500,.25,.75,'geopotential')));
end
function testMissingModelAndGate(testCase)
[db,file]=new_test_database(); cleanup=onCleanup(@() close_fixture(db,file));
e=db_get_engine(db,1); e.ID=2147483647;
[~,available,message]=ml_predict(e,flight_condition(0,0,1)); verifyFalse(testCase,available);
verifyTrue(testCase,contains(message,'physics')); verifyError(testCase,@() require_validation(e),'ABACUS:ValidationRequired');
end
function close_fixture(db,file)
%CLOSE_FIXTURE Close the temporary database before removing it on all paths.
delete(db); delete(file);
end
```

tests/TestMission.m

```
function tests = TestMission
%TESTMISSION Deliberate algorithm boundary fixtures; numbers are not engine data.
tests=functiontests(localfunctions);
end
function [t,m] = boundary_fixture()
%BOUNDARY_FIXTURE Two hypothetical constraint candidates used only for algorithm tests.
t=table([1;1],["Turbojet";"Turbojet"],[1;1],[100;200],[.001;.002],[NaN;NaN],[NaN;NaN],[100;200],...
    'VariableNames',{'Valid','Architecture','Ceiling_OK','EquivalentThrust_N','TSFC_kgNs','ShaftPower_kW','BSFC_kgkWh','DryWeight_kg'});
m=struct('Altitude_m',0,'Mach',0,'Ceiling_m',0,'Architectures','Turbojet','Min_Thrust_N',0,...
    'W_Thrust',.2,'W_Efficiency',.2,'W_Fuel',.2,'W_Altitude',.2,'W_TWR',.2);
end
function testNormalisationDirections(testCase)
verifyEqual(testCase,normalise_metric([100;200],true),[0;1]); verifyEqual(testCase,normalise_metric([100;200],false),[1;0]);
end
function testTieAndSingleMetric(testCase)
[s,tied]=normalise_metric([100;100]); verifyEqual(testCase,s,[.5;.5]); verifyTrue(testCase,tied);
verifyEqual(testCase,normalise_metric(100),.5);
end
function testAllEliminated(testCase)
[t,m]=boundary_fixture(); m.Min_Thrust_N=300; [mask,reasons]=apply_hard_constraints(t,m);
verifyFalse(testCase,any(mask)); verifyTrue(testCase,all(contains(reasons,'Min_Thrust_N')));
end
function testOnlyOneSurvives(testCase)
[t,m]=boundary_fixture(); m.Min_Thrust_N=150; verifyEqual(testCase,apply_hard_constraints(t,m),[false;true]);
end
function testHighScoreCannotOverrideConstraint(testCase)
[t,m]=boundary_fixture(); m.Max_DryWeight_kg=150; m.W_Thrust=1; m.W_Efficiency=0; m.W_Fuel=0; m.W_Altitude=0; m.W_TWR=0;
verifyEqual(testCase,apply_hard_constraints(t,m),[true;false]);
end
function testArchitectureAndCeiling(testCase)
[t,m]=boundary_fixture(); t.Architecture(1)="Turboprop"; t.Ceiling_OK(2)=false;
verifyFalse(testCase,any(apply_hard_constraints(t,m)));
end
function testWeightsRejected(testCase)
[~,m]=boundary_fixture(); m.W_Thrust=.9; verifyError(testCase,@() validate_mission(m),'ABACUS:MissionWeights');
end
function testMixedSFCRejected(testCase)
[~,m]=boundary_fixture(); m.Architectures='Turbojet,Turboprop'; verifyError(testCase,@() validate_mission(m),'ABACUS:MissionMetric');
end
```

tests/TestSimulink.m

```
function tests = TestSimulink
%TESTSIMULINK Optional real model build/simulation and MATLAB backend parity check.
tests=functiontests(localfunctions);
end
function testBuiltModelsMatchBackend(testCase)
assumeTrue(testCase,license('test','Simulink'),'Simulink unavailable; model execution NOT EXECUTED.');
[db,file]=new_test_database(); delete(db); folder=tempname; mkdir(folder);
cleanup=onCleanup(@() close_sim_fixture(file,folder)); models=build_simulink_models(file,folder); ids=[1,4,9];
dt=DT_Coordinator(file); dbCleanup=onCleanup(@() delete(dt));
```

```
for i=1:3
    load_system(fullfile(folder,[models{i},'.slx']));
    result=sim(models{i},'ReturnWorkspaceOutputs','on'); data=result.get('abacus_output');
    expected=dt.run(ids(i),flight_condition(0,0,1),'L1',false);
    verifyEqual(testCase,data(end,2),expected.EquivalentThrust_N,'RelTol',1e-9);
    verifyEqual(testCase,data(end,4),expected.FuelFlow_kgs,'RelTol',1e-9);
    close_system(models{i},0);
end
end
function close_sim_fixture(file,folder)
%CLOSE_SIM_FIXTURE Close only project test models, then remove temporary artifacts.
for name={'ABACUS_Turbojet','ABACUS_Turbofan','ABACUS_Turboprop'}
    if bdIsLoaded(name{1}), close_system(name{1},0); end
end
if isfile(file), delete(file); end
if isfolder(folder), rmdir(folder,'s'); end
end
```

tests/new_test_database.m

```
function [db,file] = new_test_database()
%NEW_TEST_DATABASE Reproduce schema/seed in a temporary file, never the production DB.
% Output: DB handle and path. Caller closes handle before deleting the file.
file=[tempname,'.db']; db_initialize(file); db_seed(file); db=DB_Interface(file);
end
```

verify_project.m

```
function report = verify_project(databaseFile)
%VERIFY_PROJECT Check installation, duplicate public function names and DB integrity.
% Input: optional DB path. Output: PASS/FAIL/WARNING table; no simulation claims.
p=project_paths(); addpath(p.root,genpath(p.matlab),p.gui,p.simulink);
if nargin<1, databaseFile=p.database_file; end
checks=strings(0,1); statuses=strings(0,1); detail=strings(0,1);
required={'README.md','LICENSE','CHANGELOG.md','database/schema.sql','database/seed_data.sql',
...
    'database/engine_analysis.db','database/engine_test.db','gui/EngineAnalyser_v1.m',...
    'run_all_tests.m','run_project.m','simulink/build_simulink_models.m'};
for i=1:numel(required)
    checks(end+1)="File: "+required{i};
    if isfile(fullfile(p.root,required{i})), statuses(end+1)="PASS"; else, statuses(end+1)="FA
IL"; end
    detail(end+1)="Presence check only";
end
checks(end+1)="MATLAB release";
if ~verLessThan('matlab','23.2'), statuses(end+1)="PASS"; else, statuses(end+1)="FAIL"; end
detail(end+1)=version;
for entry={'Database_Toolbox','Statistics_Toolbox','Simulink'}
    checks(end+1)="License: "+entry{1}; present=license('test',entry{1});
    if present, statuses(end+1)="PASS"; else, statuses(end+1)="WARNING"; end
    detail(end+1)="License availability is not proof of functional execution.";
end
files=dir(fullfile(p.root,'','*.m')); names=string({files.name});
checks(end+1)="Unique public MATLAB filenames";
if numel(unique(names))==numel(names), statuses(end+1)="PASS"; else, statuses(end+1)="FAIL"; e
nd
detail(end+1)=string(numel(files))+" MATLAB source files";
try
    db=DB_Interface(databaseFile); cleanup=onCleanup(@() delete(db));
    t=db.pragma('integrity_check'); checks(end+1)="SQLite integrity";
    if istable(t), value=string(t{1,1}); else, value=string(t{1}); end
    if value=="ok", statuses(end+1)="PASS"; else, statuses(end+1)="FAIL"; end
```

```
    detail(end+1)=value;
    fk=db.pragma('foreign_key_check'); checks(end+1)="SQLite foreign keys";
    if isempty(fk), statuses(end+1)="PASS"; else, statuses(end+1)="FAIL"; end
    detail(end+1)="Read-only foreign_key_check";
    engines=db.read('Engines');
    for i=1:height(engines), db_get_engine(db,engines.Engine_ID(i)); end
    checks(end+1)="Parameter provenance"; statuses(end+1)="PASS"; detail(end+1)="Every populated numerical engine parameter checked against its provenance value.";
catch exception
    checks(end+1)="Database readiness"; statuses(end+1)="FAIL"; detail(end+1)=string(exception.message);
end
report=table(checks,statuses,detail,'VariableNames',{'Check','Status','Diagnostic'});
disp(report); writetable(report,fullfile(p.reports,'installation_verification.csv'));
end
```